

Colony Counter

Development of an Integrated Computer Vision
System for Semi-Automated Microbial Colony Analysis

Master Thesis June 2026

Authors

Anton Egeskov Grier
Frederik Valdemar Holck Reimert
Lasse Schnell Danielsen

Supervisor

Nicki Skafte Detlefsen



Approval

This thesis has been prepared over six months at the Department of Applied Mathematics and Computer Science, at the Technical University of Denmark, DTU, in partial fulfilment for the degree Master of Science in Engineering, MSc Eng. The work represents 30 ECTS points from each member for a total of 90 ECTS. The project was started on 12th of January and was completed 12th of June. It is assumed that the reader has a basic knowledge in the areas of computer science. The work was completed by the following members:

Frederik Valdemar Holck

Reimert - s203520



Signature

12 June 2026

Date

Lasse Schnell Danielsen - s203512

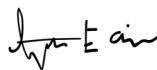


Signature

12 June 2026

Date

Anton Egeskov Grier - s203557



Signature

12 June 2026

Date

Abstract

Manual colony counting is time-consuming, operator-dependent, and difficult to scale. This thesis presents ColonyCounter, a semi-automated system for microbial colony counting that combines a web-based user interface with automatic Petri dish cropping and deep-learning-based colony detection. The system is designed around a human-in-the-loop workflow, allowing automated predictions to be inspected, filtered, and corrected by the user.

The system was developed iteratively. A user study and think-aloud evaluation shaped the platform, which evolved from a prototype into a publicly hosted service at colonycounter.dk supporting image upload, project organisation, data persistence, user authentication, and multi-format batch export. For colony detection, both CNN- and transformer-based architectures were evaluated. The final pipeline uses a density-aware Dual RF-DETR strategy: each image is first processed with full-image inference, and dense plates are rerun using tiled inference. On a held-out external benchmark of 108 agar plates across three agar types and two phone models, ColonyCounter achieved the strongest practical counting accuracy ($CA@10/\pm 1$) among all tested tools, including the recently released Colonia and Colony Grounded SAM2. Performance was strongest on well-represented agar types and densities but decreased on Chrome agar and on dense plates.

To reduce variability at the point of image capture, a low-cost foamboard imaging apparatus was developed with controlled illumination and an optional backlight. The complete workflow was evaluated on 12 plates spanning three density levels, photographed with and without the apparatus by three independent users. The apparatus-based workflow with manual correction reached 100% $CA@10/\pm 1$ with a mean absolute error of 2.1 colonies and took 7.5 minutes, compared with 8.2 minutes for manual counting. The time advantage was concentrated on high-density plates, where average per-plate counting time fell from 104 to 64 seconds.

Overall, the results show that ColonyCounter can substantially reduce the time required for microbial colony counting while preserving the accuracy needed for practical laboratory use. However, for out-of-distribution colony appearances, dense plates or difficult imaging conditions, manual correction remains the primary safeguard, and the findings support a practical assisted-counting workflow rather than a fully automatic replacement for human judgement.

Acknowledgements

We would like to extend our sincere gratitude to the microbiology labs at DTU Bioengineering for their collaboration and support throughout this thesis. A special thanks to **Mille Rask Sander**, research assistant, **Asta Zeuner**, graduate student, and **Vedis Marie Kühnau Arntzen**, graduate student, for acquiring agar plates, preparing colonies, and patiently testing our solution across multiple iterations, including participation in the initial user study and the Iteration 1 think-aloud evaluation. Their hands-on laboratory expertise and honest feedback were invaluable in shaping the system.

We also thank group leaders **Lone Gram**, **Mogens Kilstrup**, and **Ahmad Zeidan** for participating in the user study during Iteration 0 and for providing insight into real-world colony counting workflows. We are grateful that the final system is actively used in Mogens Kilstrup's laboratory for assay work.

We would also like to thank our friends, families, and partners for their continued support throughout the project.

Lastly, we would like to thank **Nicki Skafte Detlefsen** for supervising the project and providing valuable guidance throughout the process. We are grateful for the feedback and encouragement that helped shape this work.

Contents

Preface	ii
Abstract	iii
Acknowledgements	iv
1 Introduction	1
1.1 Research Questions	3
2 Background and Related Work	5
2.1 Manual Counting	5
2.2 Automated Counting	6
2.2.1 Hardware-Based Tools	6
2.2.2 Software-Based Tools	7
2.2.3 Deep Learning Approaches	8
2.3 Data	14
2.3.1 The AGAR Dataset	14
2.3.2 The Makrai et al. Veterinary Bacterial Colony Dataset	15
2.3.3 The Moucka et al dataset	16
2.3.4 DS1 and DS2	17
2.3.5 Limitations of Existing Datasets	17
3 Iteration 0: Baseline System	19
3.1 User Study	20
3.2 Platform Design	22
3.2.1 System Architecture	23
3.2.2 Detection Pipeline	23
3.3 Colony Detection Model	25
3.3.1 Method	25
3.3.2 Results	26
3.4 Performance Evaluation	26
3.4.1 Method	26
3.4.2 Results	26
3.5 Conclusion	28
4 Iteration 1: System Development	30
4.1 Think-Aloud Evaluation	31
4.2 Platform Changes	32
4.2.1 Security and Hosting	32
4.2.2 Workflow Features	33
4.2.3 Interface and Usability	35
4.3 Petri Dish Detection	37

4.3.1	Method	37
4.3.2	Results	38
4.3.3	Conclusion	38
4.4	End-to-end Processing Time	38
4.4.1	Method	38
4.4.2	Results	39
4.5	Conclusion	40
5	Iteration 2: Object Detection Model	43
5.1	Data	44
5.1.1	Method	44
5.1.2	Results	45
5.1.3	Data statistics	46
5.1.4	Conclusion	48
5.1.5	Final data splits	48
5.2	Improving the performance of the Initial YOLO model	48
5.2.1	Method	48
5.2.2	Results	50
5.2.3	Conclusion	53
5.3	RF-DETR	53
5.3.1	Method	53
5.3.2	Results	55
5.3.3	Conclusion	57
5.4	Extending RF-DETR Beyond 300 Colonies	57
5.4.1	Method	58
5.4.2	Results	60
5.4.3	Conclusion	63
5.5	Routing Architecture	63
5.5.1	Routing Approaches	65
5.5.2	Evaluation on Validation I	66
5.5.3	Final Evaluation on Validation II	68
5.5.4	Conclusion	70
5.6	Model Calibration, Post-processing, and Failure Analysis	70
5.6.1	Method	70
5.6.2	Results	72
5.6.3	Summary of Results	76
5.7	Final Evaluation on Held-Out External Benchmark	76
5.7.1	Method	77
5.7.2	Results	79
5.8	Conclusion	84
6	Imaging apparatus prototype	86
6.1	The Prototype	86

6.2	Conclusion	88
7	Full System Evaluation	89
7.1	Method	89
7.2	Results	90
7.2.1	Manual reference count agreement	90
7.2.2	Primary counting performance by colony density	91
7.2.3	Effect of backlighting	92
7.2.4	Effect of manual correction	93
7.2.5	Workflow timing	93
7.2.6	Accuracy-time tradeoff	95
7.3	Conclusion	96
8	Future Work and Preliminary Investigations	97
8.1	Laboratory Specific Models	97
8.2	Synthetic Data Generation	97
8.2.1	Overlap Augmentation	98
8.2.2	Full Plate Synthesis	98
8.2.3	Summary and outlook	99
8.3	Colony Classification	99
8.3.1	Summary and outlook	101
9	Discussion and Conclusion	102
9.1	Feature Delivery	103
9.2	Research Questions in Retrospect	104
	Bibliography	108
A	Appendix	113
A.1	Contributions	113
A.2	Statement of AI use	113
A.3	Data	114
A.3.1	BRUKER Family	114
A.4	Website Examples	115
A.5	Image Augmentations and Hyperparameters: YOLO	115
A.5.1	Colony Detection	115
A.6	Hybrid RF-DETR SAHI Experiment	116
A.7	Routing Model: Full Val I Results	117
A.8	Routing Model: Implementation Details	118
A.8.1	Otsu Router	118
A.8.2	ConvNeXt-Atto Regression Router	119
A.8.3	YOLOX-Nano Router	119
A.9	External Final Evaluation Supplementary Tables	120
A.9.1	Freestyle smartphone-image performance	120

A.9.2	Paired freestyle bootstrap comparisons	121
A.9.3	Freestyle performance by colony-count bin	121
A.9.4	Freestyle performance by agar type	121
A.9.5	Cropping comparison	122
A.9.6	Apparatus-image performance	122
A.9.7	OpenCFU and Online-Colony-Counter.com subset baselines	122
A.10	Colony Classification	122
A.10.1	Methods	123
A.10.2	Results	125
A.10.3	Implementation	126
A.10.4	Summary and Outlook	126
A.11	Imaging Box Process	127
A.11.1	Methods	127
A.12	Full-System Evaluation Supplementary Material	133
A.12.1	Manual reference count agreement	133
A.12.2	Primary counting performance	133
A.12.3	Density subgroup performance	133

1 Introduction

The biotechnology sector has expanded rapidly over recent decades, driven by advances in molecular biology, microbiology, and synthetic biology. These developments have enabled the large-scale production of pharmaceuticals, biofuels, enzymes, and other biologically derived products. As the complexity and scale of biological experiments have increased, laboratories are required to process large numbers of samples and experimental conditions, creating a strong demand for high-throughput and reproducible experimental workflows. Modern biotechnology research increasingly relies on large experimental datasets to explore extensive parameter spaces in areas such as strain engineering, fermentation optimization, and drug discovery [1].

Traditionally, many experimental procedures in microbiology and biotechnology laboratories have relied on manual operations performed by trained personnel. Tasks such as liquid handling, microbial plating, incubation management, colony enumeration, and data recording have historically been carried out manually. While these approaches remain effective, they are often labour-intensive and susceptible to variability introduced by human operators. As laboratory throughput requirements grow, manual workflows can become a limiting factor for experimental productivity and consistency [2].

To address these challenges, laboratory automation has become increasingly integrated into biotechnology workflows. Automated systems can combine robotic liquid handling, automated incubators, and integrated analytical instrumentation to execute complex experimental protocols with minimal human intervention. These platforms enable high-throughput experimentation and allow laboratories to explore a larger experimental design space than would be feasible using manual techniques alone. Automated workflows can therefore accelerate the development of biological processes and generate large, standardized datasets that are suitable for computational modelling and machine learning approaches [1].

In microbiology laboratories specifically, automation has been shown to improve workflow efficiency and standardization. Total laboratory automation systems can integrate specimen processing, incubation, imaging, and analysis into a continuous pipeline. Studies have demonstrated that such systems can significantly reduce turnaround times while improving the consistency and reproducibility of microbiological analyses [3]. Automation also contributes to improved standardization of procedures and can reduce variability introduced by manual sample handling [4].

Despite these technological advances, several common microbiological tasks remain largely dependent on manual inspection and interpretation. One notable example is the enumeration of microbial colonies on agar plates.

Microbial colony counting

Microbial colony counting is a fundamental task in microbiology laboratories, used to estimate the concentration of viable microorganisms in a sample, expressed as colony-forming units per millilitre (CFU/mL). The technique is central to both fundamental microbiology research and a wide range of applied disciplines, including the assessment of microbial contamination in food, water, and

dairy products, the evaluation of preservative efficacy, and quality control in pharmaceutical and medical device manufacturing [5]. Despite its widespread application, manual counting remains the prevailing practice and is both time-consuming and prone to human error and operator bias [6].

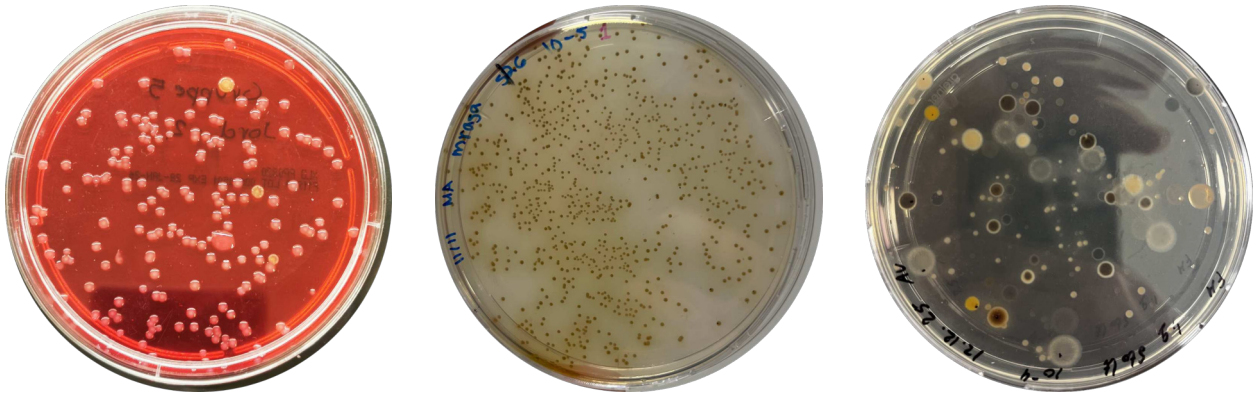


Figure 1.1: Examples of agar plates used for microbial colony counting, illustrating variation in colony density, colony morphology, agar appearance, and imaging conditions.

Automating colony detection and counting, however, is a non-trivial problem. A key challenge is the large variability in colony appearance. Colonies can differ significantly in size, shape, colour, texture, and opacity depending on the microbial species, growth medium, incubation time, and environmental conditions (Figure 1.1). This variability makes it difficult to design detection methods that generalise well across different experimental setups.

In addition, colonies frequently grow in close proximity and merge at higher concentrations, forming overlapping clusters that are difficult to separate into individual units. Even for human experts, determining whether such formations represent single or multiple colonies can be ambiguous. This biological ambiguity complicates both algorithmic segmentation and the definition of reliable ground truth for supervised learning approaches.

The imaging conditions further contribute to the complexity of the task. Agar plates often exhibit non-uniform backgrounds due to lighting gradients, reflections, condensation, bubbles, or imperfections in the agar. These artefacts can resemble small colonies or obscure true ones, particularly when colonies are faint or underdeveloped. The reflective surface of the Petri dish introduces additional problems, including glare on the dish surface, reflections of the microbial colonies in the lid, and distortions near the plate boundary, each of which can interfere with detection algorithms. Differences in imaging hardware and acquisition protocols across laboratories further reduce the robustness and transferability of automated solutions. Purpose-built imaging apparatus designs has has been shown to mitigate these artefacts effectively [7]. However current solutions are often expensive. An example of this is from Reshape Biotech, where, while the price of imaging devices are not publicly available, just the rack for holding the device costs 9,999 USD [8].

Together, these challenges define the core technical requirements this thesis must address: morphological variability across microbial colonies and imaging conditions, the reliable detection of colonies on high density plates where colonies often cluster, and the handling of imaging artefacts introduced by standard laboratory equipment. This thesis supports two complementary contribu-

tions: a hardware-agnostic deep learning detection framework designed to maintain high performance under realistic laboratory imaging conditions, and a dedicated low-cost imaging prototype to provide controlled, consistent illumination at the point of image capture. Both components are integrated within a human-in-the-loop interface that enables rapid verification and correction by the researcher.

1.1 Research Questions

This work builds on a previously conducted project, where the scope of the system was explored and a proof-of-concept product was developed. The full scope and overview of the previous work is explained in Chapter 3. In this section we explore the research questions, which serve as the foundation of the thesis.

Main Research Question

Building on an established proof-of-concept, how can a semi-automated colony counting system be improved to achieve robust and generalisable colony detection within an accessible workflow, across diverse conditions encountered in real laboratory environments?

This question is investigated through three sub-questions, each targeting a distinct aspect of the system and addressed across the work presented in this thesis.

Q1: Detection

To what extent can a deep learning detection pipeline be developed and optimised for robust, generalisable colony counting across colony morphologies, densities, and imaging conditions encountered in laboratory practice?

While detection performance is a central concern, the system through which researchers interact with those detections must also meet practical requirements for speed and ease of use in routine laboratory workflows.

Q2: Platform

How can a platform for a semi-automated colony counting system be built to support an efficient and accessible counting workflow for researchers and laboratory practitioners?

A further source of variability lies upstream of the detection model: the conditions under which images are captured.

Q3: Imaging Apparatus

To what extent does a purpose-built, low-cost imaging prototype reduce imaging variability and improve the reliability of the automated colony counting workflow?

Uncontrolled photographing conditions of agar plates introduce variability such as glare, reflections and blur that are difficult to compensate for in software alone. This question examines whether a dedicated imaging enclosure, designed to provide uniform and diffuse illumination, can reduce this variability and make the counting workflow more consistent across laboratory environments.

Together, these questions define the scope of this thesis. The end goal is an integrated, hardware-agnostic framework for semi-automated colony detection that any researcher can use with standard laboratory or consumer hardware, without specialised equipment or technical expertise.

Thesis Structure

Rather than designing the full system upfront and evaluating it once at the end, the work proceeds through a series of self-contained iterations. Each iteration defines a concrete problem, implements and evaluates a solution, and uses the findings to motivate the scope of the next iteration. This structure reflects the empirical nature of the research: what to build next is determined by what the current results reveal.

Each iteration is presented with its own introduction, methods, results, and conclusion. This makes it possible to follow the reasoning behind each design decision and understand how the system evolves over time. The Background chapter following this introduction provides the technical foundations necessary to understand the methods used across all iterations.

2 Background and Related Work

To establish a common understanding of the current field, this chapter aims to cover any previous and related work performed by other researchers and private companies. We also introduce the datasets used in coming iterations and the deep learning model architectures we have worked with.

2.1 Manual Counting

Manual enumeration of colony-forming units (CFUs) remains the standard practice in microbiological laboratories. To facilitate the process, microbiologists typically use mechanical click-counters or specially designed lamps equipped with a magnifying lens, background illumination and a pressure sensor to track the count (Figure 2.1).

The acceptable count range for reliable colony enumeration varies by method and regulatory context. ISO 7218:2024, which provides general requirements for microbiological examinations in the food chain, specifies that plates containing no more than 300 total colonies should be selected for counting [9]. The same standard defines 10 colonies as the generally accepted lower limit of reliable determination [9]. This range was first systematically established by Breed and Dotterer in 1916, who empirically derived the colony density interval that minimises counting errors from serial dilution plate counts of market milk samples [10]. The lower bound reflects the statistical unreliability of sparse counts, where minor sampling variation or ambient contamination introduces disproportionately large relative errors. The upper bound reflects the onset of confluency, where adjacent colonies merge and become impossible to separate, causing systematic undercounting alongside increasing operator fatigue. However, plates outside these recommended intervals may still be counted or estimated in practice. This may occur when no dilution produces a count within the recommended range, or when the applicable method, workflow, or analytical purpose permits an approximate rather than fully reliable count.

Counting reliably depends heavily on the individual analyst. Studies have demonstrated that manual counting is prone to errors, fatigue, and subjective bias. Fowler et al. quantified this variability against the accuracy thresholds specified in the Standard Methods for the Examination of Dairy Products, which set a within-analyst limit of 5% and a between-analyst limit of 10%. In practice, analysts reproduced their own counts with a coefficient of variation of 7.71%, and reproduced another analyst’s count on the same plate at 18.19% [11], both exceeding the defined acceptability limits and indicating a level of inter-operator disagreement that can introduce substantial inconsistency across laboratories.

Additionally, the number of colonies on a plate has a significant effect on the time it takes to count them. Fruin et al. found that counting a single plate in the 30–300 colony range takes approximately 41 seconds on average, rising to 52 seconds for plates in the 40–400 colony range [12]. For laboratories processing hundreds of plates per day, this time burden is far from trivial. Therefore, microbiologists sometimes divide the plate into sections, such as quadrants, counting only one section and extrapolating to estimate the total colony count [6]. This practice assumes perfect spatial uniformity of the microbial colonies across the plate. In reality, this is rarely the

case, as the plating method, whether pour-based or spread-plated, frequently results in uneven microbial placement. This causes the extrapolated result to be potentially skewed and unreliable.

Taken together, the inter-operator variability, the time burden at high plate densities, and the use of unreliable approximation practices constitute the primary limitations of manual colony counting and motivate the development of automated tools.



(a) Hand-held mechanical tally counter.



(b) Colony counting lamp with magnifier.

Figure 2.1: Traditional methods for microbial colony counting. These manual techniques, a mechanical tally counter (a) [13] and a magnifying colony counting lamp (b) [14], are commonly used for enumerating colony-forming units on culture media in a petri dish.

2.2 Automated Counting

Efforts to automate colony counting have progressed through three broad phases. Hardware-integrated instruments emerged first, followed by software-based image analysis, and more recently by deep learning approaches.

2.2.1 Hardware-Based Tools

The first practical automatic colony counter was introduced by Mansberg in 1957, and by the 1970s commercial systems such as the 3M Model 620 and the Artek series had emerged, using internal video cameras and analogue contrast thresholding. A systematic evaluation of four such instruments found that not one met the acceptability criterion of 90% of counts falling within $\pm 10\%$ of the corresponding manual reference count, with documented failure modes including sensitivity to non-biological artifacts such as air bubbles and agar scratches and a need for instrument-specific correction factors that varied with colony morphology and plate orientation [15]. These limitations drove the development of more capable platforms.

Modern commercial hardware systems such as Interscience Scan [16], QCount [5], ColonyQuant [17], and Reshape [18] provide end-to-end automated counting as integrated, closed-source platforms. These systems house the Petri dish within a dedicated imaging enclosure, eliminating ambient

lighting variability. Internally, programmable LED illumination with configurable top and bottom lighting modes and a calibrated high-resolution camera ensures consistent image acquisition. Together, these components provide the controlled, reproducible imaging conditions that underpin system accuracy and account for a substantial portion of the procurement cost.

These tools provide end-to-end solutions but are limited by high purchasing cost and licensing fees, restricting accessibility. The price of an automated hardware system typically falls in the range of \$5,000 to \$20,000, a range that remains broadly representative of current market pricing [19, 20], and they generally require bench space within crowded microbiological laboratories and regular calibration and service. For many laboratories, adoption therefore represents a trade-off between the financial investment and the ongoing time cost of manual counting. Even when available, many microbiologists remain sceptical of automated systems and still opt to manually validate or adjust results, preferring their own judgement over the automated output [19]. Independent evaluation of a commercial hardware counter found that fully automated counting produced a mean deviation of 59.7% from manual reference counts, improving to 1.8% only after manual operator review and correction [21]. However, incorporating this correction step extended the total counting time to 104 seconds per plate, compared with 70 seconds for manual counting alone, negating the anticipated time savings. The controlled imaging environment is therefore the defining technical property of these systems.

2.2.2 Software-Based Tools

For the software based methods, the vast majority of the early and existing tools rely on digital image processing (DIP) and classical computer vision techniques to identify colonies. The classical computer vision pipeline typically involves converting a high-resolution colour image into a simplified grayscale representation, followed by the application of mathematical filters to enhance contrast between the microbial biomass and the agar background. Segmentation is then achieved by applying a threshold to separate foreground colonies from the agar background. A popular choice in this field is Otsu's method [22]. This algorithm aims to find the optimal threshold that best separates foreground and background [23]. It achieves this by iterating through all possible pixel intensity values to find a threshold that minimizes the intra-class variance of the background versus foreground pixels, while maximizing the inter-class variance. After a binary segmentation mask is generated, other morphological operations are typically applied to refine the masks. To separate overlapping colonies, these tools often utilize geometric assumptions, such as microbial colonies most commonly being round. An algorithm such as the Circular Hough Transform [24] is often utilized, which uses a voting procedure to detect circles within a predefined radius interval [23, 25, 26]. Alternatively, the classic Watershed algorithm treats the grayscale image intensity as a topological elevation map, mathematically "flooding" the map from identified regional minima to build artificial boundaries (watershed lines) between adjacent biological clusters [27].

A widely known example of a software solution is the open-source tool OpenCFU. This tool uses DIP to construct a score map to increment morphologically valid areas based on user defined thresholds [23]. Other notable tools that apply different algorithms to detect colonies include ImageJ [28] and EBIImage, but many other alternatives exist [25, 29, 30, 31, 32].

While these approaches can perform well under highly controlled conditions, where colonies are uni-

formly distributed, approximately circular, well-contrasted against the agar, and non-overlapping, they have significant fundamental disadvantages. Classical computer vision algorithms require precise manual fine tuning of various hyperparameters (e.g threshold levels, expected circularity ratios, and minimum/maximum colony pixel sizes). Without the tedious manual calibration, even small changes of agar media, colony morphology, or environmental changes can confuse the algorithm, leading to unreliable results.

To overcome the shortcomings of rigid hyperparameter tuning, tools emerged that integrate basic machine learning into the pipeline. AutoCellSeg replaces manual calibration with an interactive phase where the user clicks representative colonies, then uses a feedback-based watershed algorithm to adaptively separate clusters [33]. CFUCounter similarly requires a user-selected reference colony, extracts colour and size features to configure thresholds, and applies the Density-Based Spatial Clustering of Applications with Noise (DBSCAN) algorithm to group and separate touching colonies [34]. While these hybrid approaches reduce manual calibration effort, they still require per-image user interaction, break down at high colony densities, and have been validated only on small monoculture datasets [33, 34].

2.2.3 Deep Learning Approaches

Recent developments in automated colony counting have shifted towards deep learning, with convolutional neural networks (CNNs) now representing the dominant paradigm. Using machine learning, the task often takes the form of an object-detection or image-segmentation problem. Unlike classical algorithms, that rely on a sequence of mathematical rules to define what is a colony, CNNs can learn to spot colonies of varying sizes, colours and shapes. By processing thousands of annotated images during a computationally intensive training phase, CNNs learn highly complex, non-linear hierarchical feature representations directly from raw image pixel data, without threshold tuning [35]. This shift was made possible by the growing availability of large, publicly annotated colony image datasets and modern GPU hardware.

When assays require exact morphological metrics, such as the total surface area of a colony or precise perimeter analysis for motility studies, segmentation networks are used to assign a class label to every pixel. An early approach was to use a U-net [36], an encoder-decoder architecture that uses skip connections to preserve spatial detail. However, U-net performs semantic segmentation, meaning it classifies pixels as `colony` or `background` without distinguishing between colonies. This leads to a problem where overlapping colonies become fused into a single colony-blob.

To overcome this, some researchers integrated secondary classification networks as a post processing step that splits clusters [37]. Others opted for Mask R-CNN for instance segmentation [38], allowing it to learn to segment individual colonies.

While segmentation models can provide pixel-level detail, laboratories and industrial workflows typically prioritize high-speed inference and reliable enumeration over morphological precision. For such applications, object detection architectures are often preferred because they directly localize and count individual objects using bounding boxes, typically requiring lower annotation effort and computational cost than full semantic or instance segmentation pipelines [39].

Early frameworks for object detection, such as Region-Based Convolutional Neural Networks (R-

CNN), utilized a decoupled approach where traditional computer vision algorithms, specifically Selective Search, were employed to propose approximately 2,000 candidate regions of interest. Each proposal was independently processed through a Convolutional Neural Network (CNN) for feature extraction, followed by a Multi-Layer Perceptron (MLP) for classification. This methodology introduced significant computational redundancy, as the requirement for 2,000 forward passes per image resulted in massive overhead and high inference latency [40].

The computational limitations of such proposal-based pipelines motivated the development of more efficient detection frameworks. Among these, the *You Only Look Once* (YOLO) family has emerged as one of the most widely adopted approaches, particularly where inference speed is a central design requirement [39]. More recently, transformer-based detectors have introduced an alternative detection paradigm with complementary strengths. Both families are examined in the following sections.

YOLO Architecture

In 2015, the introduction of the YOLO architecture fundamentally shifted the computer vision paradigm by reframing object detection as a single-pass regression problem rather than a localized classification task [39]. The YOLO framework operates by partitioning the input image into an $S \times S$ grid. For each grid cell, the network predicts multiple bounding boxes, each defined by spatial coordinates (x, y, w, h) and an associated confidence score. A grid cell is responsible for detecting objects whose centroids fall within its boundaries.

In addition to bounding-box predictions, each grid cell outputs a vector of conditional class probabilities, $(P(\text{Class}_i \mid \text{Object}))$, forming a class-probability map over the $S \times S$ grid. These probabilities are predicted once per grid cell and are shared across all bounding boxes generated by that cell.

Each bounding box also outputs a confidence score, which is trained to estimate

$$P(\text{Object}) \times \text{IoU}_{\text{pred}}^{\text{gt}}$$

where $(\text{IoU}_{\text{pred}}^{\text{gt}})$ is the intersection over union between the predicted box and the ground-truth box. During inference, the network directly predicts this confidence score. The final class-specific confidence for a bounding box is then computed as

$P(\text{Class}_i \mid \text{Object}) \times \text{Confidence}$, which can be interpreted as an estimate of

$$P(\text{Class}_i \mid \text{Object}) \times P(\text{Object}) \times \text{IoU} * \text{pred}^{\text{gt}}$$

This score reflects both the likelihood of the object belonging to class (i) and the predicted quality of the bounding-box localisation.

The model is optimized using a composite loss function comprising five terms: two losses for bounding box center coordinates and dimensions, one loss term for object confidence in cells containing objects, one term penalizing confidence predictions in cells without objects, and one term for classification errors in the conditional class probabilities.

To manage the spatial redundancy introduced by grid-based predictions, YOLO originally em-

ployed Non-Maximum Suppression (NMS) as a post-processing heuristic. NMS filters overlapping detections by comparing the IoU of all proposed boxes. When the IoU of two boxes from the same class exceeds a predefined threshold, the box with the lower confidence score is discarded.

Since its introduction in YOLO9000 [41], the YOLO architecture has gradually refined how bounding boxes are predicted from feature maps. A key change after the original YOLO design was the introduction of anchor-based detection. In anchor-based versions, predictions are made directly from feature maps produced by the backbone network. Each feature map can be interpreted as a grid where every spatial location corresponds to a region of the input image. At each grid location, the model predicts several anchor boxes, which are predefined bounding box templates with fixed aspect ratios and sizes. For example, if a detection head operates on a feature map of size 13×13 and predicts 5 anchors at each spatial location, the total number of candidate bounding boxes produced by that feature map is $13 \times 13 \times 5 = 845$.

Each anchor prediction includes offsets that adjust the anchors center coordinates, width, and height, as well as an objectness score and class probabilities. During inference, these candidate boxes are filtered using confidence thresholds and non-maximum suppression to produce the final detections. During training, each ground-truth object is assigned to the anchor whose shape and position yield the highest Intersection over Union (IoU), and only this matched anchor is optimized to regress toward that object, while the remaining anchors are treated as negatives.

Later models such as YOLOv8 adopt an anchor-free design, in which the network directly predicts object centers and regresses bounding box coordinates without relying on predefined anchor shapes, thereby reducing the number of hyperparameters and eliminating the need to compute IoU between ground-truth boxes and anchor priors during assignment [42].

In more recent development, YOLOv10, further removes the traditional Non-Maximum Suppression (NMS) post-processing step, which has quadratic worst-case complexity $O(n^2)$ due to pairwise IoU comparisons between predicted boxes [43]. YOLOv10 achieves this by employing a dual-head training strategy: a one-to-many head generates multiple candidate predictions per object to stabilize learning, while a one-to-one head enforces a unique prediction for each ground-truth instance. At inference time, only the one-to-one head is used, enabling the network to output a single, high-confidence bounding box per object and thereby bypass NMS entirely, resulting in a fully end-to-end and more computationally efficient detection pipeline. The Ultralytics framework utilizes only the one-to-one head if the detection limit is set to ≤ 300 . If more than 300 detections are required the model uses the many-to-one head and employs NMS regardless.

Building upon this, Ultralytics have released the model YOLO26. [44] Beyond the NMS-free inference paradigm introduced in YOLOv10, YOLO26 focuses on simplifying the detection head and improving training efficiency. YOLO26 implements a new loss function that shifts the weight during training from the one-to-many head in the beginning of training the one-to-one head at the end. Two further improvements include the Small-Target-Aware Label Assignment (STAL) which prioritizes small targets during training and the implementation of hybrid MuSGD optimizer, which combines the Large-Language Model, loss Muon and the standard Stochastic Gradient Descent. [45]

DETR-Based Models

While YOLO-style detectors have traditionally relied on CNN based architectures, recent developments in real-time object detection have given rise to Detection Transformer (DETR) models, such as LW-DETR, RT-DETR, D-FINE, and RF-DETR [46, 47, 48, 49]. These models build upon the foundation established by DETR which fundamentally redefines earlier detection models by replacing dense anchor-based prediction and post-hoc non-maximum suppression, as often used in models such as YOLO, with a unified end-to-end architecture that treats object detection as a direct set prediction problem [50].

The original DETR

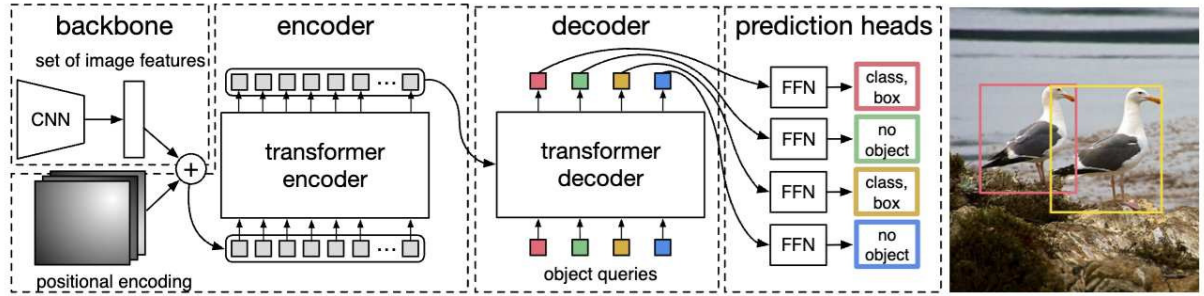


Figure 2.2: DETR model architecture [50]. The model consists of a backbone, a transformer encoder, a transformer decoder and FFN prediction heads.

The DETR architecture consists of 4 primary blocks: a CNN backbone, a transformer encoder, a transformer decoder, and a set of prediction heads (Figure 2.2) [50].

The CNN backbone first processes the input image and produces a single feature map, which is then reduced in channel dimension via a 1×1 convolution and flattened into a sequence of tokens. Since the transformer is permutation-invariant and does not inherently encode spatial structure, positional encodings are added to this token sequence at the input of each attention layer. This allows the encoder to use multi-head self-attention to model global dependencies across the entire image while at the same time still having spatial information. These encoding are then fed to the transformer decoder, in addition to learnable object queries. Each query therefore ends up with information about all other queries, and context about the image. Finally, each query is passed to a FFN which predicts a class label, and a bounding box [50].

During training the model uses bipartite matching (with the Hungarian Matching algorithm) to assign each prediction with a ground truth detection. This matching is determined by a cost function that evaluates predicted class probability, and the bounding box loss between the predicted bounding box and the GT bounding box. Based on this matching between GT and predictions the loss is calculated as a linear combination between the negative log-likelihood for the class predictions and the bounding box loss [50].

While the DETR model was revolutionary for its end-to-end simplicity, it suffers from notable computational and convergence inefficiencies due to several factors.

The first factor is the quadratic complexity of self-attention in the encoder w.r.t. to the number of tokens. This means that the encoder is computationally expensive for higher-resolution inputs with

a high number of tokens. The original DETR is therefore effectively limited to lower-resolution feature maps, which lack the spatial detail necessary to reliably detect small objects, which is a significant limitation for colony object detection [51].

The second factor is slow convergence. In the original DETR, object queries and attention modules are both initialized randomly, which results in queries having no notion of what objects look like or where they might be, and the attention modules assign nearly uniform weights across all image pixels. Through training, the queries must learn to represent meaningful object concepts while the attention weights must learn to focus on a sparse set of relevant spatial locations. This is a slow process and is part of the reason why DETR requires around 500 epochs to converge on COCO, roughly 10 to 20 times slower than Faster R-CNN [51].

RF-DETR architecture

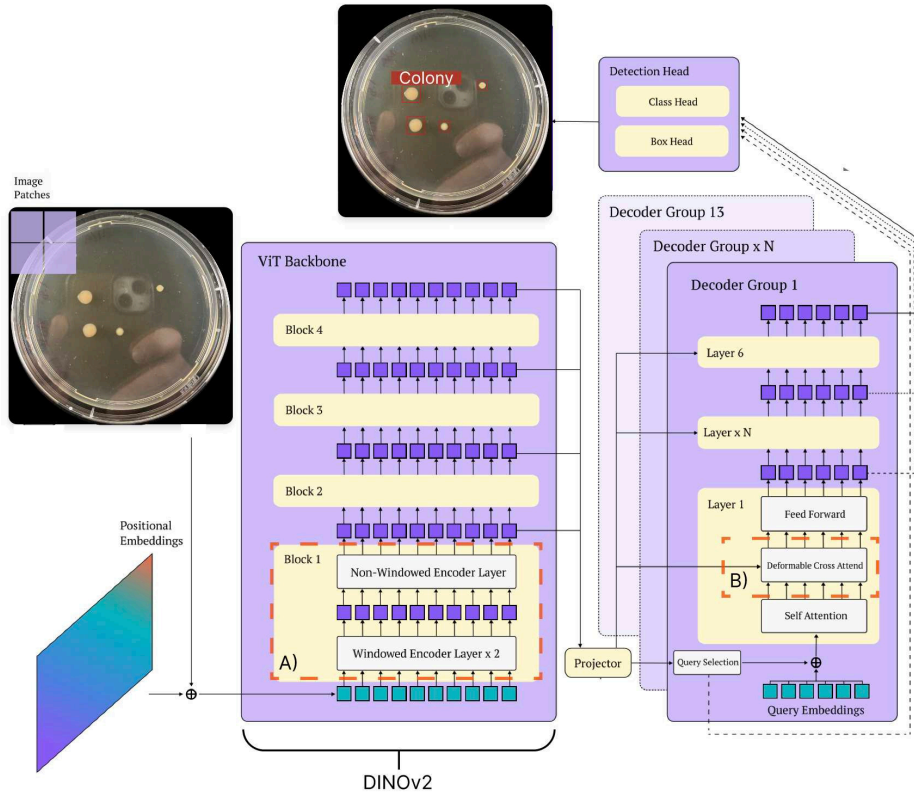


Figure 2.3: Overview of the RF-DETR architecture, for the ViT Backbone RF-DETR uses a DINOv2 backbone A) interleaved window and global attention layers, B) deformable cross-attention. [49].

RF-DETR belongs to the DETR family of object detectors and builds particularly on LW-DETR, from which it adopts much of its architecture with additional modifications. It was designed to improve performance and overcome the above-mentioned efficiency bottlenecks of the original DETR model, making the model feasible for real-time object detection [46, 49]. In this section we describe some of the main elements of the RF-DETR architecture (see Figure 2.3 for an overview).

RF-DETR follows the DETR framework but replaces the traditional CNN backbone and separate transformer encoder with a unified Vision Transformer (ViT) encoder. The images is split up into patches, each linearly projected into a patch embedding, combined with positional encodings, and

fed as a sequence of tokens into the ViT encoder [46]. Here RF-DETR uses a DINOv2 backbone [52], a ViT pretrained with self-supervised learning on internet-scale data. This provides richer visual representations and improves detection performance, particularly on small datasets and out-of-distribution domains [49].

To reduce the computational cost of the quadratic scaling of self-attention, RF-DETR uses interleaved window and global attention layers (Figure 2.3, A). Rather than applying self-attention across all tokens globally in every layer, some layers instead divide the feature map into non-overlapping local windows and apply self-attention within each window separately. Since self-attention scales as $O(n^2)$ with sequence length, operating on smaller windows dramatically reduces the number of computations required [46].

To address the slow convergence of attention weights, RF-DETR adopts deformable cross-attention in the decoder [46] (Figure 2.3, B). Rather than requiring each object query to attend to all pixels, deformable attention only attends to a small fixed set of local sampled points in the feature map. The location of these sampling points is calculated from the query (and a linear layer) meaning that where it should "look" in the feature map is itself learnable [51]. Since the attention is sparse from the beginning of training, the model avoids the slow transition from near-uniform to sparse attention that limits the original DETR.

For further optimization RF-DETR also introduces a weight-sharing neural architecture search (NAS). Here they train a single network that at every training iteration randomly samples a different configuration - varying image resolution, patch size, number of decoder layers, number of query tokens, and number of windows per windowed attention block. Because all configurations share the same underlying weights, the model is forced to learn representations that generalize across all of them. This random sampling of architectures also acts as a regularizer during training [49]. After training, they then evaluate all configurations without retraining, producing a continuous accuracy-latency Pareto curve from a single training run, which they then use to decide the optimal model architecture parameters for each variation of the model (e.g.. Small, Medium, Large).

Why do we use RF-DETR?

RF-DETR was selected as one of the detection models for this project because several of its characteristics are well suited to the task of colony counting on agar plates.

First, bacterial colonies are often small, overlapping, and visually similar to artifacts in the agar, such as impurities or debris, which places high demands on small- and medium-object detection performance. On the COCO benchmark, RF-DETR demonstrates particularly strong performance on small and medium-sized objects compared with other real-time object detectors [49].

Second, agar plate images represent a specialized visual domain that differs substantially from standard natural-image datasets such as COCO. Therefore, the detector should demonstrate an ability to generalize beyond COCO-like data distributions. The RF100-VL benchmark provides additional evidence of this capability. Unlike COCO, which contains 80 object categories from a relatively constrained set of visual domains, RF100-VL evaluates object detection across 100 diverse real-world datasets covering 564 object classes [53]. RF-DETR remains competitive on this benchmark, suggesting that its performance transfers reasonably well beyond the distribution

represented by COCO [49].

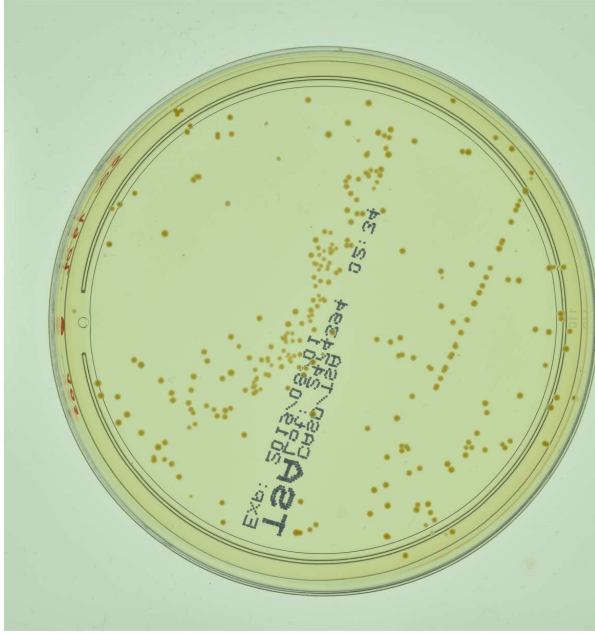
Third, practical laboratory deployment requires inference speeds that support real-time or near-real-time feedback. RF-DETR is designed as a real-time object detector and achieves competitive inference speeds on modern GPU hardware [49]. However, the latency results reported in the literature are primarily measured on GPUs, whereas the system developed in this project is intended to perform inference on CPUs. This distinction is important because transformer-based models such as RF-DETR typically benefit substantially from GPU acceleration, while convolutional models, such as YOLO, may be more efficient in CPU-based settings. Therefore, RF-DETR’s actual CPU latency will be evaluated in the project experiments.

2.3 Data

The performance of deep neural networks is highly dependent upon the availability of large, accurately annotated training datasets. Without being trained on diverse images with various species and image conditions, the models cannot successfully learn robust generalized features. To resolve this critical bottleneck and facilitate the standardization of algorithmic benchmarking, the academic community has developed and published several expansive, open-source image databases. In this section we don’t include datasets with segmentation masks (eg. the BRUKERCOLONY, BRUKERAGAR and BRUKERDUST datasets [54, 55, 56]) as their pixel-wise annotation format presents a fundamental limitation for object detection training: in regions of high colony density, overlapping colonies merge into contiguous mask regions that cannot be reliably decomposed into individual bounding boxes (See Appendix Figure A.1).

2.3.1 The AGAR Dataset

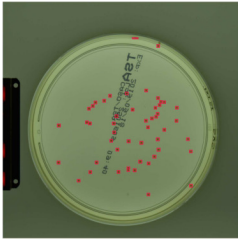
The Annotated Germs for Automated Recognition (AGAR) dataset [57] is a publicly available collection of 18,000 high-resolution images of agar plates. This dataset was the first of this kind and size, and was designed to be used for training deep learning models. It includes five different microorganisms, chosen to represent a range of morphologies across Gram positive, Gram negative, and fungal groups. The dataset includes images of isolated single cultures and selected paired mixed cultures. The images were captured in standardized setups using two different cameras. The AGAR dataset provides precise bounding box annotations for 336,442 individual colonies, labelled by expert microbiologists [57] (Figure 2.4).



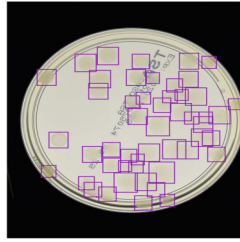
(a) Original high-resolution agar plate.



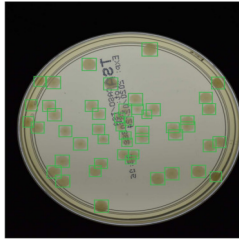
(b) Corresponding bounding box annotations.



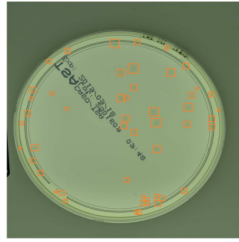
(c)



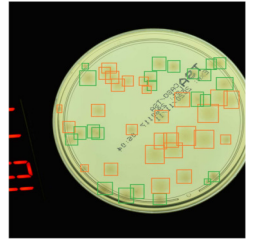
(d)



(e)



(f)



(g)

Figure 2.4: Examples from the AGAR dataset. The top row displays a typical raw image (a) alongside its expert-labelled bounding box annotations (b). The bottom row (c-g) illustrates the diversity of the dataset across varying lighting conditions, background complexities, and colony densities.

2.3.2 The Makrai et al. Veterinary Bacterial Colony Dataset

This annotated dataset was published in 2023 by Makrai et al. in *Scientific Data*, providing a collection of 369 digital images of bacterial cultures spanning 24 clinically significant bacterial species of veterinary importance [58] (Figure 2.5). Collectively, the dataset contains 56,865 manually annotated colonies, each labeled with a bounding box and species identifier. The included species were selected to represent a broad taxonomic and morphological range, cultured across several distinct solid media types, including blood, chocolate, nutrient, and MacConkey agars. To promote generalisability and reflect authentic laboratory imaging conditions, photographs were deliberately acquired using three different consumer mobile devices (LG Nexus 5X, iPhone 6, and HUAWEI P30 Lite) against both black and white backgrounds, introducing meaningful variation in image quality, color balance, and lighting. The dataset was validated by expert bacteriologists, with duplicate images removed and bounding box placements independently verified. While smaller in scale than AGAR, the Makrai et al. dataset is distinguished by its broad taxonomic coverage and its use

of consumer mobile devices, which more closely reflect the imaging conditions found in routine laboratory environments than dedicated camera setups.

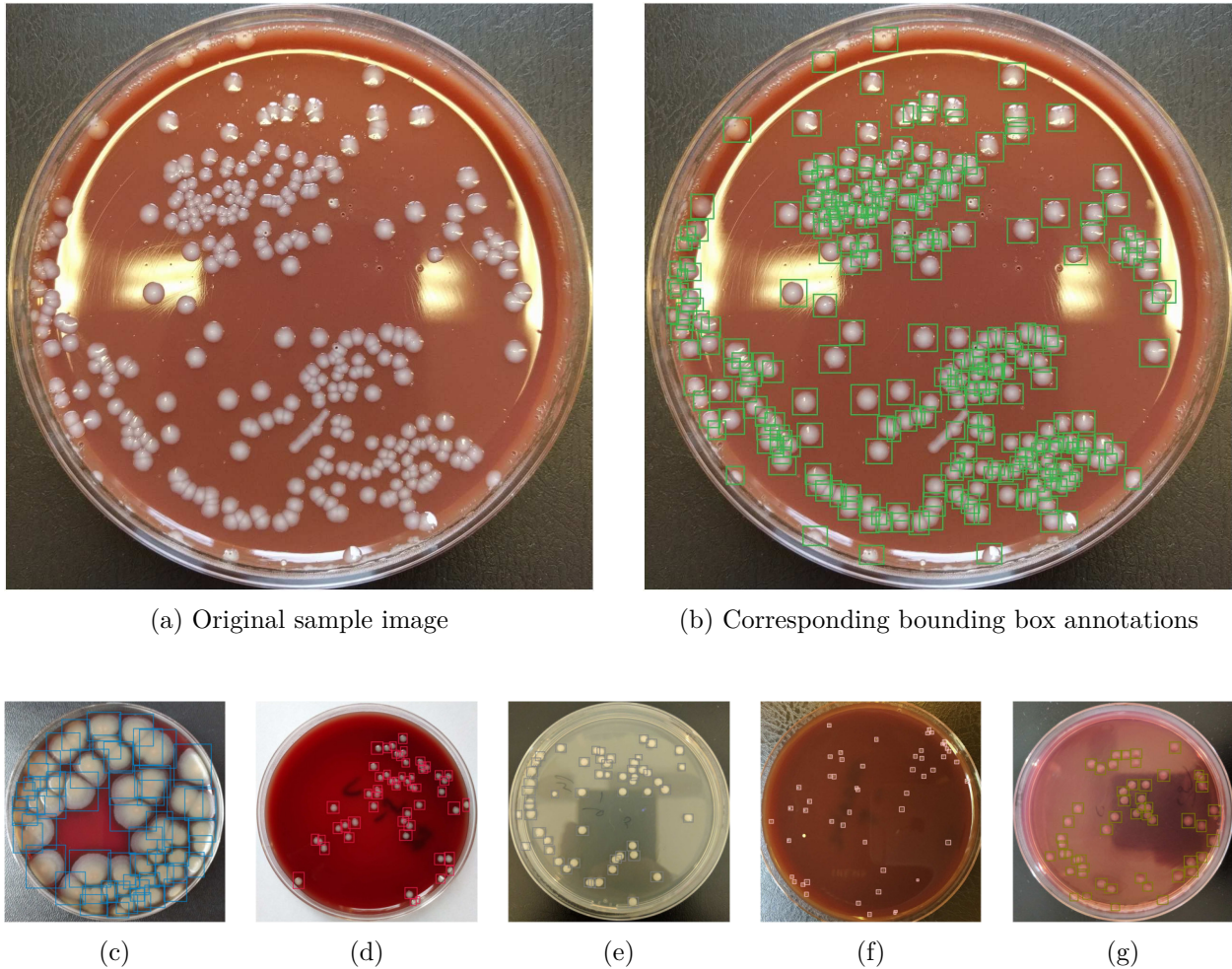


Figure 2.5: Overview of the Makrai et al. Veterinary Bacterial Colony Dataset. The top row displays a typical raw image (a) alongside its bounding box annotations (b). The bottom row (c-g) illustrates the diversity of the dataset across different samples and conditions.

2.3.3 The Moucka et al dataset

The Moucka et al dataset contains 108 agar plates, and was made to assess the performance of automatic colony counting tool against manual counting [59]. The plates contain *E. coli* grown on three agar types: Blood agar, LB agar, and Chrome agar (Figure 2.6). Plates were imaged under two acquisition conditions: freestyle imaging, where the plate was manually held, and apparatus imaging, where the plate was imaged inside a handcrafted wooden apparatus designed to standardise illumination and reduce reflections (Figure 2.6). For each acquisition condition, images were captured using both an iPhone 11 Pro Max and a Samsung Galaxy A52. The iPhone images are generally of higher visual quality and of higher resolution (approximately 12 MP for the iPhone images and 3 MP for the Samsung images.) For each plate, the number of colonies was manually counted, but colony objects were not annotated on the images.

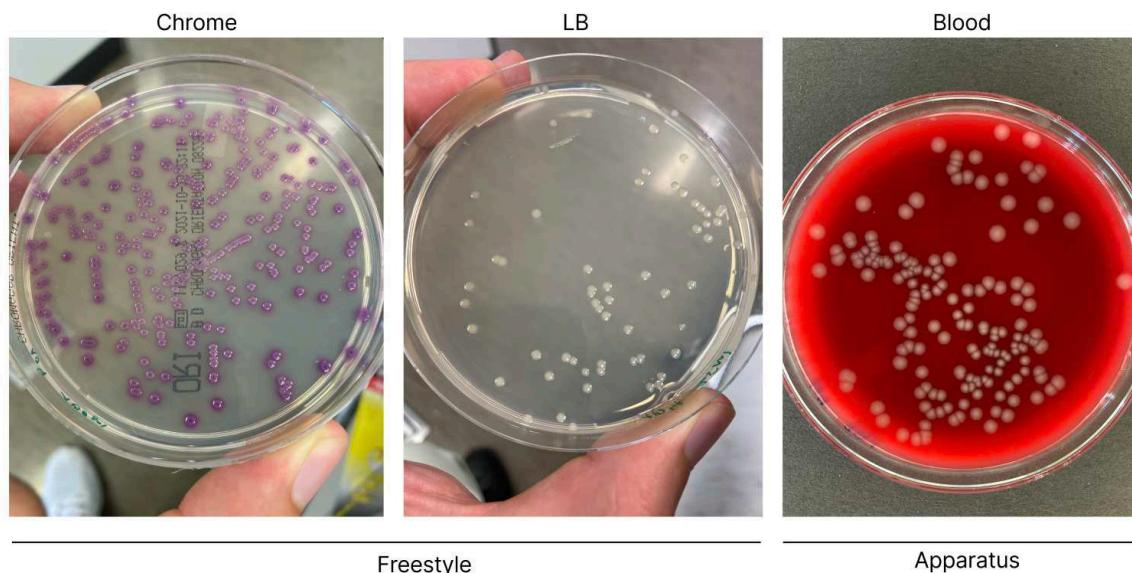


Figure 2.6: Examples of iPhone freestyle images for Chrome and LB plates, and example of Iphone apparatus image of Blood agar from the Moucka et al dataset

2.3.4 DS1 and DS2

Beyond the datasets described above, two additional object-detection datasets, referred to in this thesis as DS1 and DS2, were obtained from Roboflow Universe [60, 61]. The versions used contain 760 and 2,700 images, respectively. Both include less controlled photographs with substantial variation in lighting, perspective, background, and image quality. However, their biological composition, image-acquisition procedures, and annotation processes are sparsely documented.

2.3.5 Limitations of Existing Datasets

Although existing datasets provide valuable resources for model development and evaluation, they leave important gaps in biological diversity, imaging variability, and annotation detail relevant to this project. The AGAR dataset offers the largest collection of object-level colony annotations among the reviewed datasets, but its images were acquired using standardized camera setups under controlled laboratory conditions [57]. As a result, it provides limited representation of the variability expected in less controlled smartphone-based imaging, such as variation in lighting, viewing angle, focus, distance, background, and camera quality.

The Makrai et al. dataset partly addresses this acquisition gap by using smartphone images. However, it contains only 369 images, all collected within a single laboratory and depicting single-species cultures [58]. It therefore offers limited representation of mixed or heterogeneous samples, where multiple microorganisms may produce colonies with different or overlapping visual appearances within the same image. Additionally, it does not capture variation across different laboratories.

The Moucka et al. dataset adds further smartphone images and acquisition conditions, but provides only manual plate-level counts rather than object-level annotations for individual colonies [59]. It can therefore be used to compare automatic counting tools against manual counts, but it cannot be used directly for supervised training of object-detection models, which require colony-level locations such as bounding boxes, centroids, or segmentation masks.

DS1 and DS2 contain more varied and less controlled images, but their limited documentation makes it difficult to establish their biological coverage, acquisition conditions, and annotation reliability [60, 61]. Their suitability for model training is therefore assessed experimentally in Iteration 2.

Together these limitations indicate a need for well-documented, object-level annotated data collected under more varied biological and imaging conditions. Such data are needed both for training robust colony-detection models and for evaluating how well they generalise across laboratories and acquisition settings.

3 Iteration 0: Baseline System

The work of Iteration 0, was conducted in the Autumn semester of 2025, for the course 02830 Advanced project in digital media engineering

The motivation for this work originates directly from personal laboratory experience. As students of the Biotechnology B.Sc. programme at the Technical University of Denmark (DTU), we encountered manual colony counting as a routine but time-consuming task in laboratory courses and in research groups. As discussed in Section 2.1, manually counting colonies on Petri dishes is tedious, introduces observer variability, and scales poorly when large numbers of plates must be processed. At the time, we questioned why no effective automated solution appeared to be in widespread use for a task that seemed so well-defined.

The work of Iteration 0, was conducted in the Autumn semester of 2025, for the 02830 Advanced project in digital media engineering. We reviewed existing tools and found that while automated colony counters do exist, they are predominantly expensive proprietary instruments designed for industrial laboratory workflows, or rudimentary image processing scripts lacking robustness under real-world imaging conditions. No easily accessible, hardware-agnostic system appeared to adequately address the needs of a typical university research laboratory. This finding led us to conduct an initial user study with stakeholders from microbiology research groups at DTU. The study identified the core requirements for an accessible colony counting system and directly informed the design of our baseline prototype. These requirements are stated below and are referred to throughout this chapter.

Iteration Requirements

R0.1 The system must run on any modern computer regardless of operating system or hardware.

R0.2 The system must support interactive verification of automated detections, such as adding, removing, and filtering individual colony detections.

R0.3 Counting accuracy must be sufficient for use in scientific documentation across all plate densities encountered in practice.

R0.4 The system must reduce the time required to count colonies compared to fully manual counting.

This chapter begins with the user study that established these requirements and shaped the feature scope. We then describe the architecture and detection pipeline of the prototype. Two empirical studies follow. The first evaluates the colony detection model across datasets of increasing difficulty, and the second compares manual, AI-only, and AI-assisted counting workflows in a controlled laboratory setting.

3.1 User Study

We conducted a user study to characterise participants’ current workflows, identify barriers preventing the adoption of existing automated tools, and determine which features would be most valuable in a new system. Participants included research group leaders, a research assistant, and graduate students, as listed in Table 3.1.

Table 3.1: Overview of User Study Participants

Role	Participant(s)
Research Group Leaders	Lone Gram, Mogens Kilstrup, Ahmad Zeidan
Research Assistant	Mille Rask Sander
Graduate Students	Vedis Arntzen, Asta Zeuner

The interviews confirmed that manual colony counting is widely perceived as a bottleneck in laboratory workflows. Participants identified four primary barriers to adopting existing automated solutions: the high cost of hardware-based counters, insufficient accuracy for use in scientific documentation, poor integration with existing laboratory workflows, and an inability to handle the diversity of colony morphologies encountered in practice. Participants had varying prior experience with existing tools. One participant had used classical image analysis software in earlier work, while a colleague had briefly tested a mobile application for colony detection before abandoning it due to poor detection performance. In Lone Gram’s research group, a Reshape[18] machine was available but used exclusively for timelapse documentation rather than colony counting. Despite its automated counting capability, the absence of any means to verify or manually adjust detections made it unsuitable for results intended for scientific documentation. No available solution fitted naturally into the day-to-day workflow.

The most consistently expressed requirement was speed and ease of use. The system must reduce counting time without imposing a learning curve or requiring specialised equipment. Another finding was that participants were uncomfortable with fully automated counting. Rather than wanting the system to operate without oversight, participants considered manual verification a requirement for any result to be used in scientific documentation. Specifically, they required the ability to add and remove individual detections and to filter detections by size or confidence threshold, correcting the model where it missed colonies or produced false positives. This was not framed as an optional refinement step, but as a necessary part of the workflow. This requirement had a direct influence on our system design, rather than optimising purely for model accuracy, we built the prototype around an interactive verification interface as its central feature.

The interviews surfaced 20 candidate features, listed in Table 3.2. To prioritise these, we constructed an effort-impact matrix. We assessed each feature on two dimensions, the estimated implementation effort and the expected impact on the users’ workflow. We assigned scores collectively through consensus, using a relative scale in which we evaluated features against each other rather than against any fixed reference.

Table 3.2: Candidate features identified during the user study.

Category / Feature	Description
Detection & Counting	
Count (High Quality)	Detect and count colonies on high quality images captured under perfect lighting conditions, where each colony is visually clear and distinct on top of the agar media.
Count (Medium Quality)	Extend detection to images with moderate artefacts such as slight uneven illumination and a few reflections.
Count (Low Quality)	Robust detection on images of low quality, affected by glare, blur, reflections or other challenging conditions.
Auto Cropping	Automatically detect and crop the Petri dish boundary from the raw image.
Classification	
Classify Colour	Categorise colonies by their pigmentation.
Classify Morphology	Categorise colonies by their morphological features like texture and shape.
Interface & Usability	
Filters	Filter detections based on size and confidence.
Interactive Tools	Manually add, remove, or adjust individual colony detections on the canvas.
Mobile UI	A responsive interface optimised for use on smartphones and tablets.
Custom Styling	User-configurable visual appearance of colony bounding boxes.
Data & Workflow	
Data Persistence	Save and retrieve past counting sessions and results across visits.
Project Management	Organise plates into named experiments or projects for batch processing.
Export Images	Download annotated images with overlaid colony detections.
CFU Calculator	Convert colony counts to colony-forming units per millilitre (CFU/mL) given volume and dilution factor.
Growth Curves	Generate time-series plots from repeated counts of the same culture.
Benchling	Integration with the Benchling laboratory information management system.
Hardware & Advanced	
Photo Box	A lightweight imaging enclosure providing improved light conditions for image capture.
Multi Image Anti Glare	Merge multiple exposures of the same plate to eliminate glare computationally.
Outlier Detection	Identify outlier detections on the plate.
OCR	Optical character recognition to read marker text on plates.

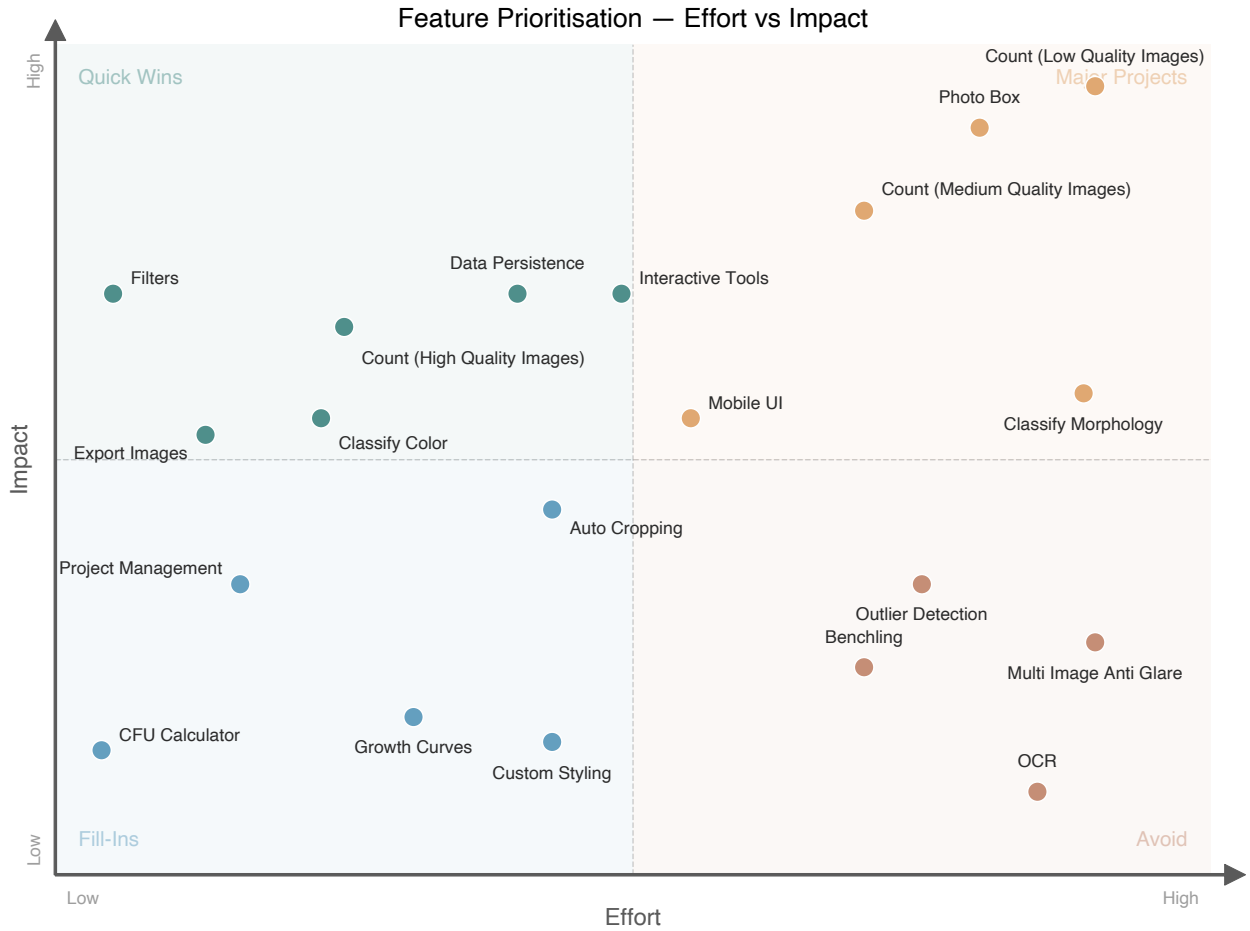


Figure 3.1: Effort-impact matrix for candidate features we identified during the user study.

The matrix (Figure 3.1) organises features into four quadrants: *Quick Wins* (low effort, high impact), *Major Projects* (high effort, high impact), *Fill-Ins* (low effort, low impact), and *Avoid* (high effort, low impact). Counting on high-quality images, detection filters, data persistence, and image export fell into the Quick Wins quadrant and formed the core implemented feature set of our prototype. Colour classification also fell in the Quick Wins quadrant and was prioritised, but we did not implement it in this iteration. Interactive Tools scored high impact at moderate implementation effort, placing it on the boundary between Quick Wins and Major Projects. Given its non-negotiable status from the interviews, we elevated it to the prototype scope. We recognised high-effort features including the photo box enclosure, counting on medium and low quality images, and colony morphology classification as high-impact but deferred them as Major Projects for subsequent iterations.

3.2 Platform Design

We shaped the architecture of the prototype directly from the requirements identified in the user study. This section describes the technology choices, detection pipeline, and training strategy that formed our baseline system.

3.2.1 System Architecture

Three constraints shaped our technology choices. The system needed to run on any computer regardless of operating system or hardware, support real-time interactive annotation for manual verification, and process images with low latency. We chose a web-based architecture to address the cross-platform requirement by delivering the application through a browser, removing any dependency on a specific operating system or local environment. Our remaining choices followed from this decision and from the specific demands of the interactive verification interface. Table 3.3 summarises the technologies we selected and the rationale behind each choice.

Table 3.3: Technology stack for our baseline prototype and the rationale behind each choice.

Component	Technology	Rationale
Frontend framework	React + Next.js	Reactive component model required for real-time annotation canvas. Next.js provides routing and server-side rendering without additional infrastructure.
Annotation canvas	Konva.js	Shape-based rendering to an HTML5 canvas handles large numbers of overlapping annotations efficiently without DOM overhead.
Backend API	FastAPI (Python)	Lightweight REST API, shares a language with the inference pipeline, and we have previous experience with FastAPI.
Detection model	YOLOv9c	Favourable balance of inference speed and detection accuracy. GPL-3.0 licence permits open distribution.
Segmentation model	FastSAM	Fast inference suitable for interactive use. Text-prompt interface enables automatic dish cropping without labelled segmentation data.

We built the frontend using the React framework with Next.js. A reactive component model was a practical requirement for the annotation canvas, where users add, remove, and adjust individual colony detections in real time. We attempted to achieve this with plain HTML and JavaScript, but it introduced substantial complexity and lowered performance. Next.js additionally provided routing, API route handling, and server-side rendering out of the box, reducing the amount of infrastructure code we needed to write.

We implemented the backend in Python using FastAPI. Keeping the backend in Python avoided a language boundary with the inference pipeline, which relies on PyTorch and Ultralytics. We selected FastAPI over Flask and Django based on our prior team experience and its suitability for serving model predictions as a lightweight API.

3.2.2 Detection Pipeline

Our detection pipeline consisted of two sequential models, as illustrated in Figure 3.2. First, we used a FastSAM [62] model to segment the Petri dish from the raw image and produce a cropped region of interest. We selected FastSAM over the original SAM model primarily for its inference speed, which is necessary for responsive interaction in a web application. It also supports text prompting, which allowed the system to automatically crop the plate using the prompt *whole petri*

dish. Furthermore, FastSAM generalised much better to varied dish sizes, lighting conditions, and plate types without requiring any task-specific fine-tuning or labelled segmentation data.

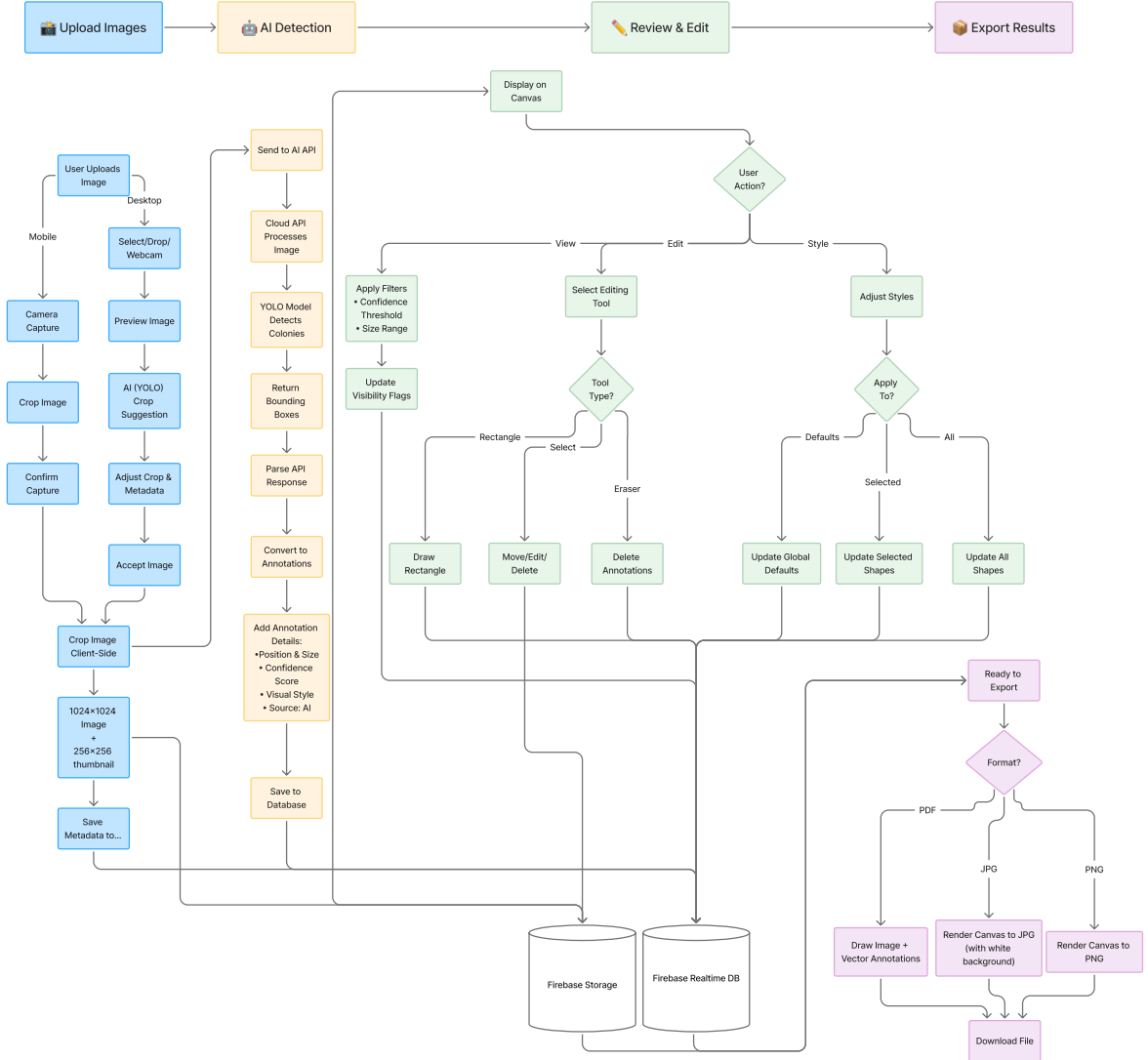


Figure 3.2: Overview of the semi-automated workflow we implemented in the baseline prototype, illustrating the four-stage pipeline from image acquisition through AI-assisted annotation to export.

Secondly, we applied a fine-tuned YOLOv9c [63] object detection model to the cropped image to localise and count individual colonies. We chose YOLOv9c for its favourable balance between inference speed and detection accuracy, with the compact variant offering sufficient capacity for colony detection without excessive computational overhead.

Users could then review the detections on our interactive annotation canvas and add, remove, or filter colonies before exporting the result. We implemented the canvas using Konva.js via the `react-konva` bindings, which integrated cleanly with the React component tree. Konva’s shape-based rendering model meant that each annotation could be treated as an independent object with its own event handlers, making drag, click, and hover interactions straightforward to implement. It

also handles large numbers of overlapping annotations efficiently by rendering to an HTML5 canvas rather than the Document Object Model, which is important for high-density plates where several hundred detections may be displayed simultaneously. Figure 3.3 shows the resulting interface.

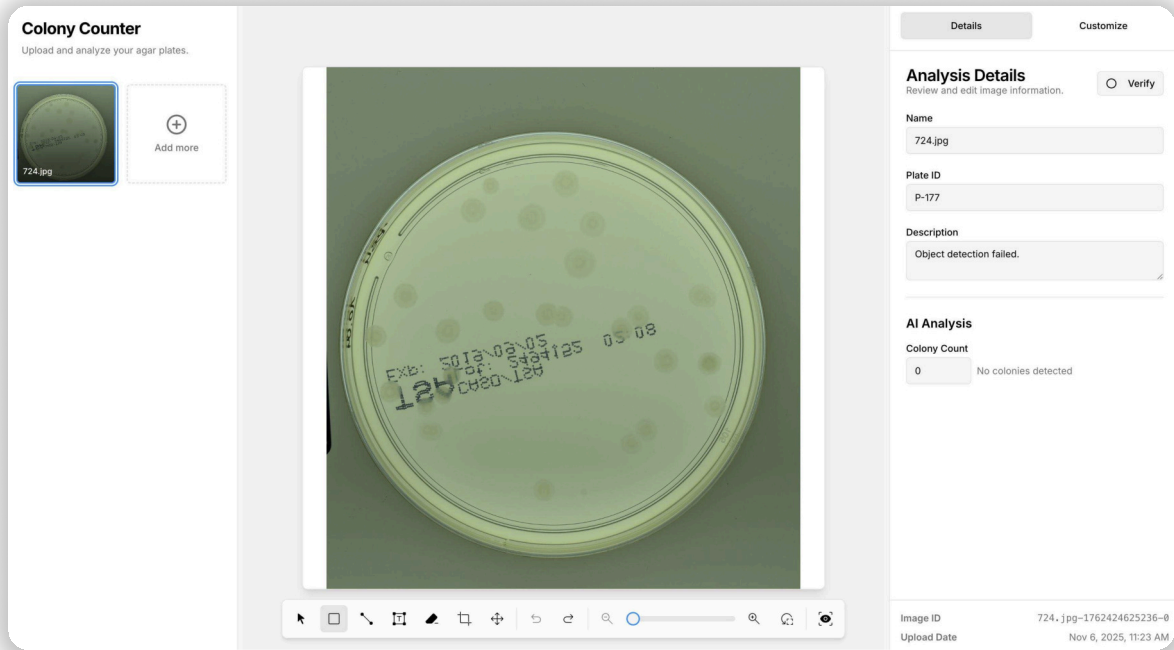


Figure 3.3: The baseline prototype interface. The left panel lists uploaded plates, the central canvas displays the selected image with the annotation toolbar for adding and adjusting detections, and the right panel reports the analysis details and AI colony count.

3.3 Colony Detection Model

We trained the colony detection model using a curriculum learning strategy and evaluated it across three public datasets of increasing imaging difficulty.

3.3.1 Method

To train a colony detection model capable of generalising across the imaging variability typical of university laboratories, we adopted a curriculum learning approach. We first trained the model on the AGAR dataset [57], a standardised benchmark for automated colony counting, with heavy augmentation applied to simulate varied imaging conditions. We subsequently exposed the model to two additional datasets, DS1 [60] and DS2 [61], with greater imaging variability taken from Roboflow. These datasets had images of much lower quality and were intended to broaden the learned feature domain and reduce overfitting to the specific acquisition setup of the AGAR data.

We then evaluated the trained model on all three datasets. The AGAR dataset ($n = 18,000$) consists of standardised, high-quality images captured under controlled laboratory conditions. DS1 ($n = 760$) and DS2 ($n = 2,700$) exhibit substantially greater variance in perspective, lighting, and colony morphology, with DS1 representing the most challenging imaging conditions of the three. We evaluated both the curriculum-trained model (`with_curr`) and a conventionally trained baseline (`no_curr`) across all three datasets. We quantified performance using exact match accuracy,

weighted accuracy, mean absolute error (MAE), and the coefficient of determination (R^2).

3.3.2 Results

Table 3.4 reports the performance of the curriculum-trained and conventionally trained YOLOv9c models on each of the three evaluation datasets.

The results show a clear generalisation gap across datasets. The model performed well on the AGAR dataset, captured under controlled conditions, but performance degraded substantially on the more heterogeneous DS2 and DS1 datasets, with DS1 proving particularly challenging. This cross-dataset drop reflects the sensitivity of the model to imaging conditions outside the AGAR distribution, and defines the core problem that our subsequent iterations set out to address. Curriculum learning improved performance on DS1 and DS2. However, since the test sets were drawn from the same distributions as the training data, it is difficult to draw firm conclusions about how either model would generalise to unseen real-world images.

Table 3.4: Detection performance of the curriculum-trained and conventionally trained YOLOv9c models across three datasets of increasing difficulty.

Dataset	Model	Exact Acc (%)	Weighted Acc (%)	MAE	R^2
AGAR	no_curr	69.42	97.02	0.74	0.994
	with_curr	59.19	95.97	1.01	0.991
DS2	no_curr	7.69	78.67	18.29	0.864
	with_curr	40.00	89.96	8.61	0.934
DS1	no_curr	21.62	22.87	56.74	-0.119
	with_curr	21.62	56.08	32.31	0.367

3.4 Performance Evaluation

We evaluated the end-to-end system with three participants to compare manual counting, AI-only counting, and AI-assisted counting with interactive verification.

3.4.1 Method

Each participant counted ten agar plates spanning four dilution levels (10^{-8} to 10^{-5}) using three methods in sequence: fully manual counting, AI-only counting, and AI-assisted counting with interactive verification and correction. Each participant’s own manual count served as the ground-truth reference for their respective AI results. We assessed counting accuracy as the mean signed error (method – Manual) per density group. We recorded processing time per method for each participant and compared across methods using a paired t-test.

3.4.2 Results

We evaluated the end-to-end system across ten agar plates with varying colony densities, including one plate with an exceptionally high colony count of approximately 900 colonies. The counted results can be viewed in Table 3.5

In terms of accuracy, both methods performed well across standard-density plates. On the very sparse, sparse, and medium groups, the AI-only error was within a few colonies of the manual count in all cases. The Dense plate was the exception: on a crowded plate where colonies overlap and

merge, the model systematically undercounted by an average of 291 colonies, and manual verification reduced this to 85 colonies. This suggests that, in this study, our human-in-the-loop design added substantial value on the most challenging plates. In addition, one of the ten plates required manual correction of the automated dish-cropping boundaries, indicating that the FastSAM-based segmentation step was not fully reliable across all imaging conditions.

Table 3.5: Colony count accuracy by plate density. Each density group contains n plates, each counted by three operators. Manual counts (mean $\pm$ SD) serve as the ground-truth reference. Δ is the mean $\pm$ SD of all per-operator differences pooled across operators and plates.

Density (colonies)	n	Manual	Δ AI Only	Δ AI + Manual
Very sparse (1–6)	3	3 ± 2	$+1 \pm 2$	0 ± 0
Sparse (7–20)	3	10 ± 2	$+3 \pm 3$	0 ± 1
Medium (80–110)	3	92 ± 8	-1 ± 6	-3 ± 3
Dense (>600)*	1	905 ± 102	-291 ± 183	-85 ± 121

* For the Dense group ($n = 1$ plate), Δ reflects inter-operator variability only.

In terms of processing time, fully manual counting took an average of 6.7 minutes per ten plates. The AI-only workflow was by far the fastest at approximately 1.2 minutes, but this speed came at the cost of accuracy. The AI+Manual verification workflow took an average of 5.2 minutes, a modest reduction compared to fully manual counting, but with higher accuracy. The time difference between Manual and AI-only was statistically significant ($p < 0.05$), as was the difference between AI-only and AI+Manual ($p < 0.01$), while Manual and AI+Manual did not differ significantly ($p = 0.21$). This illustrates the core trade-off: fully automated counting is fast but unreliable on challenging plates, whereas manual verification recovers that accuracy at the cost of time.

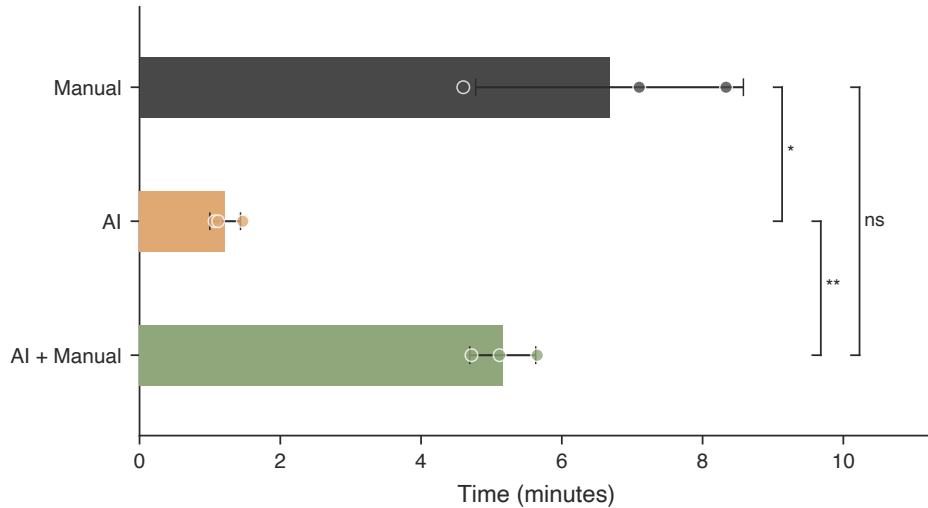


Figure 3.4: Total time to process all ten plates per method, for three independent operators. Individual operator times are shown as points. Significance brackets show pairwise t-test results (* $p < 0.05$, ** $p < 0.01$, ns: not significant).

Several aspects of this evaluation design limit the strength of the conclusions that can be drawn. The three methods were performed in a fixed sequence by each participant, which may introduce practice and memory effects that inflate the measured performance of the AI+Manual condition.

With only three participants, statistical results should be interpreted with caution. The Manual versus AI-only comparison in particular, at $p = 0.037$, sits right at the edge of the significance threshold and should not be treated as a robust finding given the small sample. The time we recorded for each method covered only the active counting and verification steps. We did not include the time required to photograph the plates, which means the reported times underestimate the total workflow duration. We selected the ten plates used in the study from the plates available to us at the time of testing. While we chose plates spanning multiple dilution levels, the number of plates we had access to was limited, which reduces how much can be generalised from the results.

The accuracy results carry an additional structural limitation. Any difference in accuracy between AI-only and AI+Manual is driven entirely by the single Dense plate, and no general conclusion about model behaviour at high colony counts can be drawn from a single observation. The manual counts on that plate also exhibited substantial inter-operator variation ($SD = 102$ on a mean of 905, approximately 11%), meaning the ground truth itself is uncertain at that density and any accuracy figures for that plate carry uncertainty beyond what is attributable to the model alone.

3.5 Conclusion

In this iteration, we established the technical feasibility of a low-cost, semi-automated colony counting system and assessed the extent to which the four system requirements were met.

R0.1: Cross-Platform Access

• Satisfied

The system must run on any computer regardless of operating system or hardware.

Our web-based interface runs in any browser without installation or specialist hardware. We handle inference on a backend server, meaning the client device itself has no compute requirement.

R0.2: Interactive Verification

• Supported

The system must support interactive verification of automated detections, including adding, removing, and filtering individual colony detections.

Our integrated Konva.js canvas allows users to easily add, delete, or adjust detections. Interactive filters let users filter AI detections by confidence score or size thresholds. On the high-density plate, AI-only undercounted by an average of 291 colonies, while AI+Manual reduced this to 85 colonies, suggesting that interactive verification added substantial value on the most challenging plates in this study.

R0.3: Counting Accuracy

• Partially Met

Counting accuracy must be sufficient for use in scientific documentation across all plate densities encountered in practice.

Our system performed well on standard-density plates, with errors of a few colonies or less across the very sparse, sparse, and medium groups. Performance degraded on the high-density plate where colony overlap caused systematic undercounting. However, the system has not yet been evaluated under more challenging and diverse real-world conditions, which for example can include glare, blur or large variations in agar type.

R0.4: Counting Speed

• Partially Met

The system must reduce the time required to count colonies compared to fully manual counting.

Our AI+Manual workflow processed ten plates in 5.2 minutes compared to 6.7 minutes for fully manual counting, a reduction that was not statistically significant ($p = 0.21$). The AI-only workflow was faster at approximately 1.2 minutes but at an unacceptable cost to accuracy. These timing results should be interpreted with caution: we used only three participants in a fixed method order, which may introduce practice effects, and we did not include the time required to photograph the plates.

These results define the scope of the following iterations. Addressing **R3** requires improving model robustness across colony densities and imaging conditions, through evaluation of new architectures, expanded training data, and dedicated imaging hardware. Addressing **R4** requires reducing the time cost of the verification step, either by improving initial detection quality to minimise the number of corrections needed, or by refining the interface to make corrections faster.

4 Iteration 1: System Development

In Iteration 0, we established that a semi-automated colony counting workflow is technically feasible. The cross-platform access and interactive verification requirements were fully satisfied by the web-based architecture and the Konva.js annotation canvas. The counting accuracy and processing speed requirements were only partially met. Performance degraded on high-density plates, and the reduction in counting time did not reach statistical significance. We identified two further technical limitations in the detection pipeline. The FastSAM-based Petri dish detection succeeded in approximately 9 out of 10 cases, leaving a persistent failure mode for routine use. Inference latency, while sufficient for interactive use, was not optimised for a deployed service.

Following Iteration 0, we conducted a formative think-aloud evaluation with three laboratory users, which identified complementary usability gaps in export functionality, session organisation, onboarding, and annotation coverage, as described in the following section.

Together, these technical gaps and user-identified limitations define the scope of this iteration. Our objectives are to replace the Petri dish detection model, reduce end-to-end processing time, deploy the system as a secure public service, complete the laboratory workflow, and improve the interface for first-time and experienced users.

Iteration Requirements

R1.1 Automatic Petri dish detection must be faster and more reliable than the FastSAM-based implementation used in Iteration 0.

R1.2 The end-to-end processing time from image upload to count result must be reduced compared to the Iteration 0 baseline.

R1.3 The system must be securely hosted and accessible to external users without requiring local installation, with appropriate authentication and usage controls.

R1.4 The system must support the complete laboratory workflow from image upload through annotation and verification to structured result export.

R1.5 The interface must be sufficiently intuitive for a first-time user to complete a counting workflow without instruction, and must provide efficient interaction mechanisms for experienced users.

This chapter begins with the formative think-aloud evaluation that shaped the iteration scope. We then describe the platform changes introduced in this iteration, followed by two empirical studies addressing the Petri dish detection and processing speed requirements. The chapter concludes with an assessment of the remaining requirements.

4.1 Think-Aloud Evaluation

After Iteration 0, we conducted a think-aloud evaluation with research assistant *Mille Rask Sander* and graduate students *Asta Zeuner* and *Vedis Marie Kühnau Arntzen* in a single group session. We followed participants through the full workflow, from photographing agar plates to uploading images, interacting with detection results, and exporting counts, while encouraging them to verbalise their reasoning and reactions throughout.

The evaluation identified findings across four main areas. summarized overview of the changes and the motivations of the motivations for them can be viewed in Table 4.1

Cropping pipeline While participants generally liked the cropping feature, as it provided clean images of their plates without background that could easily be used in presentations and documentation, they found the automatic Petri dish detection noticeably slow, to the point where manual cropping was sometimes perceived as a faster alternative. This supported the reliability gap we identified in Iteration 0, where the FastSAM model succeeded in 9 out of 10 cases, and motivated replacing it with a faster and more reliable detection model.

Ease of use Participants did not intuitively follow the upload-interact-export workflow without guidance. Important features such as the filters were easy to miss, and several interface elements lacked clear explanations. They explicitly requested a guided introduction tour of the website to show the intended workflow.

Interactions The participants wanted the option to mark missed colonies more quickly, noting that drawing a bounding box was slower than it needed to be for workflows where only the count matters. They also requested the ability to write text directly on the image to annotate specific colonies or regions. We identified several smaller interaction issues as well, such as unreliable undo behaviour, a copy function that did not always work as expected, and bounding box opacity that was too high by default, obscuring the colonies underneath.

Data organisation and export Participants explicitly asked for the ability to organise plates into named projects to improve management of multiple experiments. They also asked for more comprehensive export options covering entire projects rather than individual images.

The findings directly impacted the scope of this iteration. Based on the think-aloud evaluation, the gaps we identified in Iteration 0, and deployment requirements, we prioritised the following changes: replacement of the cropping model, a guided onboarding tour, custom hover states to improve button affordance, dot and text annotation tools, a project management system with filtering, search, and sorting, and an expanded export system supporting multiple output formats. We also resolved a number of interaction bugs.

Table 4.1: Overview of changes implemented in Iteration 1, grouped by motivating source.

Source / Change	Description
Iteration 0 Gaps	
Dish cropping model	Replace FastSAM with faster model architecture to improve segmentation reliability and speed.
Inference speed	Convert models to OpenVINO format and revise the dataflow to reduce processing time.
Deployment Requirement	
Security & authentication	Introduce user authentication, API rate limiting, and various security measures to enable safe public hosting.
Think-aloud Evaluation	
Project management	Organise images into named projects with filtering, search, and sorting.
Export system	Export entire projects in multiple formats including Excel, PDF, CSV, and raw images.
Onboarding tour	Guided introduction tour presented to first-time users covering the full workflow.
Annotation tools	Dot tool for rapid colony marking and text tool for labelling regions on the canvas.
Discoverability	Custom hover states and right-click context menu to surface available actions.
Bug fixes	Resolve unreliable undo, non-functional copy button, and default opacity issues.

4.2 Platform Changes

The changes described in this section address deployment requirements and the usability gaps we identified in the think-aloud evaluation. They are not subject to formal empirical evaluation within this iteration but are described here for completeness.

4.2.1 Security and Hosting

We selected Firebase for its tight integration across hosting, authentication, storage, and server-side functions, and because its free tier covered the expected usage volume for a research prototype. We deployed the system as a publicly accessible service at colonycounter.dk, hosted on Firebase Hosting with Cloud Functions for server-side rendering and the inference API running as a containerised service on Google Cloud Platform (GCP). Authentication uses Firebase Authentication with email verification gating new registrations. A Firebase ID token is exchanged at a server-side endpoint for an HttpOnly session cookie, with middleware verifying the cookie before rendering any protected content, keeping authentication server-side rather than reliant on client-side state. We protected mutation routes against cross-site request forgery. Inference endpoints require a valid bearer token and are subject to per-user rate limiting (150 requests per minute, with a daily ceiling of 1000 requests). We validate file uploads against an image type allowlist (JPEG, PNG, WebP, HEIC), a 20 MB size cap, and a dimension range of 256 to 8192 pixels. Per-user storage is capped at 2 GB, enforced server-side, and Firebase Storage and Database rules enforce owner-scoped access to prevent cross-user data access. We manually verified the security measures during development,

but did not conduct a formal penetration test by an external party. Such a test would provide stronger assurance that the deployed service is resilient to attack.

4.2.2 Workflow Features

We implemented the following features to address the workflow gaps identified in the think-aloud evaluation, completing the end-to-end laboratory workflow from image upload through annotation and verification to structured result export.

Project Management

The Iteration 0 workspace provided no mechanism for categorising plates across experiments. We chose a flat project-based hierarchy over a nested folder structure to minimise navigation depth for typical laboratory use, where experiments rarely require more than one level of organisation. We added search and sorting to facilitate navigation across larger collections, and included a plate validation flag to allow researchers to distinguish verified counts from plates still awaiting review. Figure 4.1 shows the project management interface before and after the redesign.

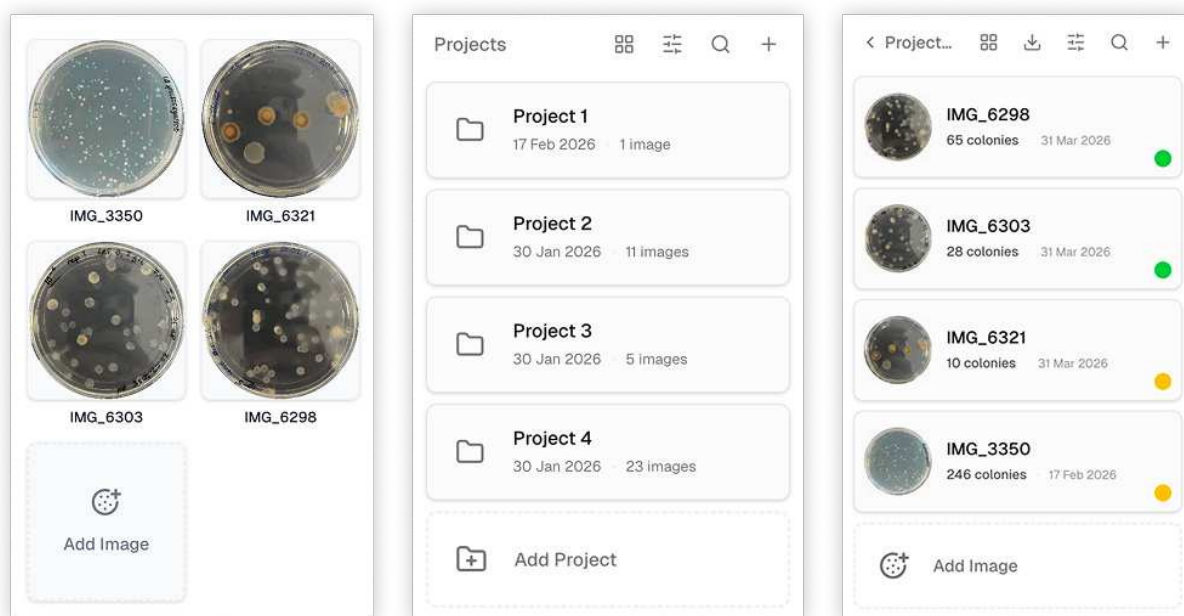


Figure 4.1: Project management before and after. Left: the Iteration 0 workspace with no project organisation. Centre and right: our new project-based hierarchy showing the project list and the image list within a project, with colony counts and per-plate validation status indicators.

Export System

Export in Iteration 0 was limited to single annotated images at a time in JPEG or PNG format, which did not support the batch documentation workflows described in the think-aloud evaluation. We redesigned the export system around a format checklist supporting both single-plate and whole-project export. Our format choices were driven by the range of downstream uses identified in the evaluation: Excel for tabular data entry into laboratory records, PDF for shareable report generation, CSV and JSON for programmatic analysis, and raw and processed image export in JPEG, PNG, and PDF format. Figure 4.2 shows the resulting export dialog.

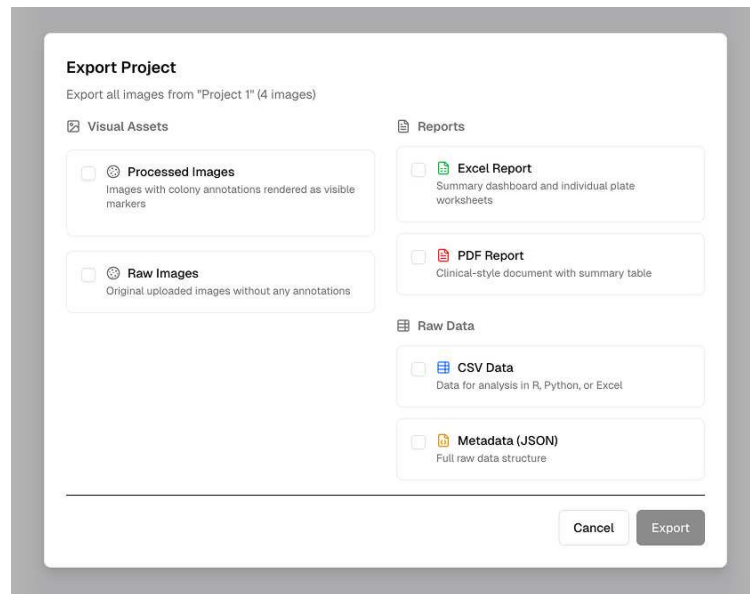


Figure 4.2: Our export dialog presenting a format checklist for single-plate and whole-project export, covering processed images, raw images, Excel report, PDF report, CSV data, and metadata.

We did not formally evaluate the workflow features with users. A think-aloud study or pre/post questionnaire targeting these features specifically would assess whether the project structure and export options align with actual laboratory documentation practices. A/B testing could additionally be used to compare alternative designs for specific features such as the export dialog.

Annotation Tools

We added two new tools to the Konva.js canvas to support counting workflows that do not require bounding box annotations.

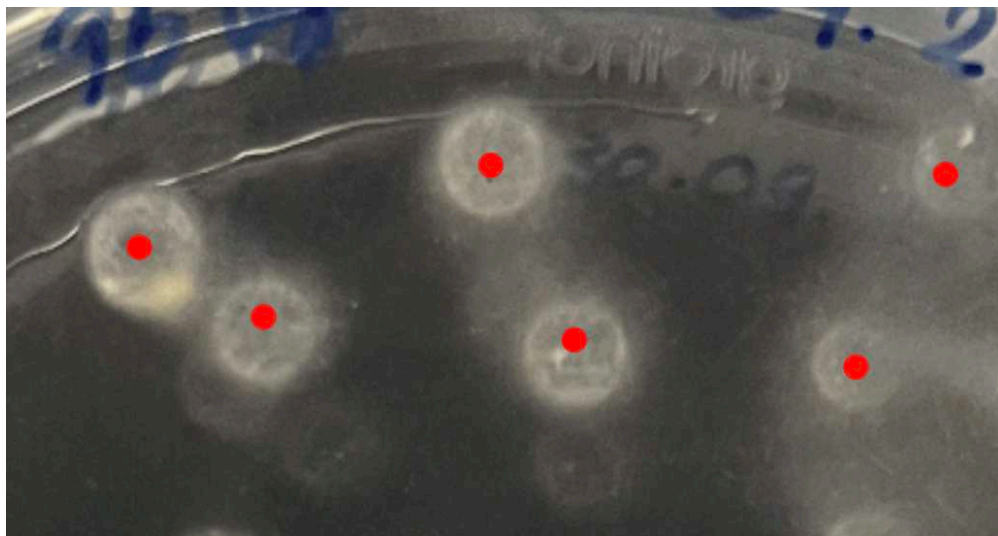


Figure 4.3: Dot annotation placed on the canvas, showing the option to place dots to quickly count colonies

The dot tool provides an alternative to bounding box annotation for workflows in which only the colony count matters. Drawing a bounding box around each missed colony was identified in the

think-aloud evaluation as unnecessarily slow when object precision is not required. The tool places a dot marker at the selected position with a single click.

Dot annotations are included in the total colony count alongside bounding box detections. Figure 4.3 shows an example of the dot annotation tool.

Text annotations allow the user to place free-form labels directly on the canvas. The Konva.js framework does not natively support editable, repositionable text. It has a built-in text layer that rasterizes content immediately to the canvas, preventing subsequent editing or repositioning. To address this, we position a floating editor overlay at the point of text creation and re-open it on double-click. Text content is stored as HTML, converted to an SVG image via a `foreignObject` element, and rendered as a Konva image node, allowing text objects to be dragged and re-edited after placement. Figure 4.4 shows the floating editor and a placed text annotation.

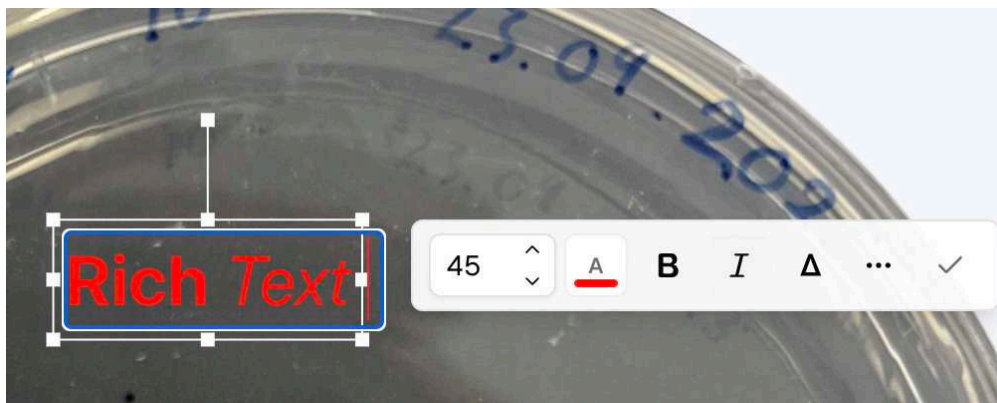


Figure 4.4: Text annotation placed on the canvas, showing our floating rich-text editor with font size, colour, bold, and italic controls. The annotation is rendered as a Konva image node, allowing it to be repositioned and re-edited after placement.

4.2.3 Interface and Usability

The following changes address the learnability and interaction gaps we identified in the think-aloud evaluation.

Onboarding Tour

We implemented a guided onboarding tour using Driver.js to address the learnability gap identified in the think-aloud evaluation. We chose Driver.js for its lightweight footprint and framework-agnostic design, which avoided adding a heavy dependency to the existing stack. The tour is presented automatically on a user's first desktop visit and not repeated once seen. An optional extended path provides a more detailed walkthrough for users who want it. Figure 4.5 shows one step of the tour, where the Driver.js overlay highlights the annotation toolbar and describes the available tools.

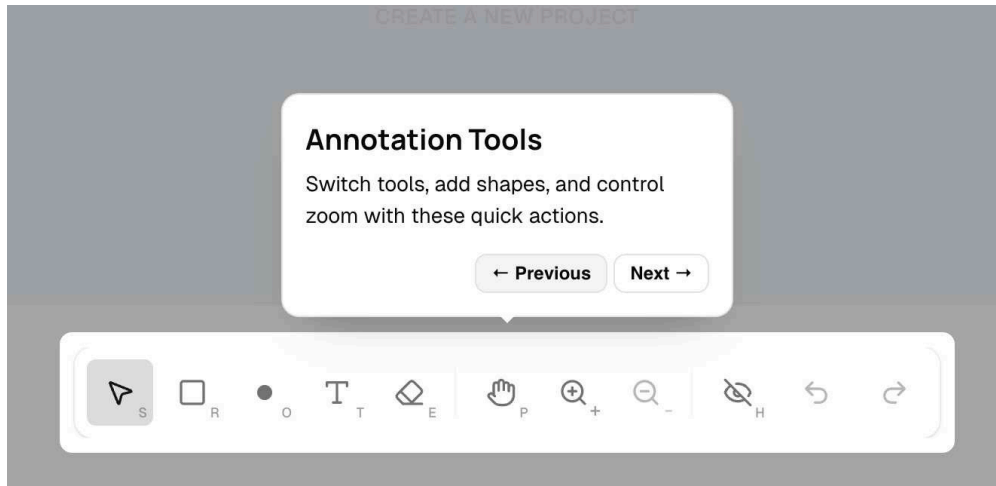


Figure 4.5: One step of our onboarding tour, implemented using Driver.js. The overlay highlights the annotation toolbar and describes the available tools, including bounding box, dot, and text annotation, zoom controls, and undo/redo.

Discoverability

We added a context-sensitive right-click menu to the annotation canvas to surface available actions without requiring users to locate toolbar controls. Right-clicking on an annotation exposes manipulation actions including copy, paste, duplicate, delete, and layer ordering.

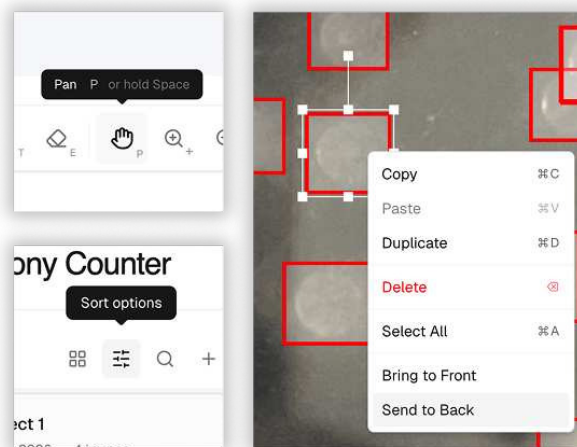


Figure 4.6: **Left:** images show examples of our custom tooltips, that allow users to hover a button or feature, to get the name and possibly shortcut if available. **Right:** Shows our context-aware right-click menu, that gives custom options to the user depending on where they right-click.

Right-clicking on the empty canvas exposes global canvas actions including undo and redo, zoom controls, annotation visibility, and canvas export. We added custom tooltips to all toolbar buttons, each displaying the action name alongside its keyboard shortcut to support discoverability for new and experienced users alike. An example of the discoverability implementations can be seen in 4.6

We did not conduct a formal evaluation of these features within this iteration. User testing, in which participants attempt the full workflow without instruction, would directly assess whether the onboarding tour and discoverability improvements achieve the learnability goal stated in R1.5.

4.3 Petri Dish Detection

This section evaluates whether the zero-shot FastSAM-based dish detection used in Iteration 0 can be replaced by a trained, YOLO-based detector that is faster, more accurate, and suitable for browser deployment.

4.3.1 Method

The FastSAM-based dish detection we used in Iteration 0 succeeded on only 9 out of 10 images. Additionally, the FastSAM model was too large to embed on the website, and was therefore running on GCP. This meant that the image of the plate had to be uploaded and cropped on GCP, and users were often able to crop the images themselves faster.

To improve both reliability and speed, our goal was to replace it with a faster and smaller architecture that could be embedded on the site and run on the client’s computer in ONNX format. Since we had only evaluated the FastSAM model on a small dataset, we also needed a proper evaluation of performance.

To train another model architecture, we required a dataset with ground truth labels for plates. No such dataset exists publicly, which was part of the reason for originally using FastSAM. We therefore created a custom dataset by annotating 295 images from various datasets and images we collected during Iteration 0. This provided a foundation for fine-tuning a small YOLO detection model trained specifically for Petri dish localisation. We split the data into an 80:20 training/validation split, with 50 images from data we collected in Iteration 0 reserved as a test set.

We trained three models. First, we trained the smallest YOLO26 model using the Ultralytics framework. After reviewing the Ultralytics licensing, we trained two YOLOX models as an alternative: tiny and nano [64]. Unlike Ultralytics’ AGPL-3.0 license, YOLOX does not require the entire project to be open-sourced. For the YOLO26 model, we applied the standard data augmentations from Ultralytics and ran training for 50 epochs. For both YOLOX models, we used the augmentations seen in Appendix A.4. All three models used pre-trained weights based on the COCO dataset[65]. We used the standard Ultralytics learning rate schedule: 3 warmup epochs, an initial learning rate of 1e-2, and a final learning rate of 1e-5. We trained all models on input images with size 416×416 , based on the default image size from the YOLOX documentation.

We evaluated final performance on the test set by measuring accuracy at IoU thresholds of 0.75, 0.85, and 0.90. We chose this metric because tight but not perfect overlap is expected: the model should produce a close-fitting box around the Petri dish without needing pixel-perfect alignment. To establish a baseline, we computed the average width and height of annotations in the training set as a fraction of image dimensions, centred in the middle of the image. The average width fraction was 0.769 and the height was 0.738, giving a fixed centre-crop heuristic. We also benchmarked inference times for all four deep learning models. To mimic the full workflow, we converted all models to ONNX format and benchmarked them on a CPU, specifically an Intel(R) Core(TM) i5-1145G7 at 2.60 GHz. We processed ten warm-up images before averaging inference time across 400 runs of the same image.

4.3.2 Results

Among the evaluated methods, YOLO26 achieves the strongest overall performance, combining the highest accuracy across all thresholds (1.00, 0.98, 0.92) with the lowest inference time among the learned models (11.85 ms). In particular, it maintains high accuracy even at IoU 0.90, indicating strong localisation precision. Both YOLOX-Nano and YOLOX-Tiny achieve perfect accuracy at IoU thresholds of 0.75 and 0.85, and slightly lower performance at IoU 0.90 (0.90). While their accuracy is comparable, their computational cost differs substantially: YOLOX-Nano requires 16.54 ms per inference, whereas YOLOX-Tiny requires 52.15 ms. The FastSAM model performs significantly worse than the detection-based approaches, with the highest inference time (1137 ms) and relatively low accuracy across all thresholds (0.48, 0.46, 0.42). Our `website_presetcrop` baseline demonstrates that a simple heuristic is insufficient, achieving the lowest inference time (1 ms) but zero accuracy at IoU 0.85. A full overview of the results can be seen in Tabel 4.2

Method	IoU 0.75	IoU 0.85	IoU 0.90	Inference Time (ms)
YOLO26	1.00	0.98	0.92	11.85
YOLOX-Nano	1.00	1.00	0.90	16.54
YOLOX-Tiny	1.00	1.00	0.90	52.15
FastSAM	0.48	0.46	0.42	1137
website_presetcrop	0.11	0.00	0.00	1

Table 4.2: Comparison of detection performance across IoU thresholds and inference time.

4.3.3 Conclusion

Based on the results of the Petri dish detection experiment, we chose to implement the YOLOX-Nano model. With its perfect detection up to IoU 0.85, we deem this model to be as good as the YOLO26. While the colony detection model should benefit from crops, it should still be trained to ignore outside disturbances, so a completely tight box is not neccessary. This fact combined with the low inference time and looser licensing, made the YOLOX-Nano the perfect candidate. The model was implemented in the frontend using WebAssembly and is able to run locally on the users computer.

4.4 End-to-end Processing Time

In Iteration 0, our upload pipeline was sequential and relied on a remote Cloud Run service for dish cropping, resulting in an end-to-end processing time of approximately 20 seconds per image. In this study, we describe three complementary changes to the pipeline and measure the resulting reduction in processing time.

4.4.1 Method

In Iteration 0, the upload pipeline was sequential: the image was first sent to a Cloud Run API for cropping, then uploaded to Firebase Storage, and finally submitted to the detection API, with each step waiting for the previous to complete. This structure introduced two main bottlenecks: the server-side network round-trip required to obtain a crop suggestion, and the sequential ordering of the storage upload and detection steps. We implemented three complementary changes to reduce end-to-end processing time.

Cropping Flow The primary speed gain in the cropping step followed directly from the model replacement described in the preceding section: running inference in the browser using `onnxruntime-web` eliminated the GCP network round-trip entirely. We cache the model session after first load, reducing latency for subsequent images in the same session.

Colony Detection We converted the backend detection model to OpenVINO format. We selected OpenVINO because it provided better CPU inference throughput than the ONNX Runtime baseline on our deployment hardware. We changed the image format sent to the detection endpoint from a base64-encoded JSON body to a compressed WebP image transmitted as multipart form data, downscaled to a maximum dimension of 1024 pixels before upload.

Upload Workflow We restructured the upload workflow for multiple images. In our earlier implementation, all images were queued through a crop review step and submitted only after the user had approved all crops, after which uploads and detection proceeded sequentially. In the revised workflow, each image begins uploading and is submitted for detection as soon as its crop is individually accepted, so the first result may be available while the user is still reviewing the remaining crops. Additionally, the Firebase Storage upload and the GCP detection API call, which previously ran sequentially for each image, now run in parallel.

Benchmarking Setup

To quantify the improvement, we measured end-to-end processing time for both versions of the pipeline. We processed a single representative image (7.8MB PNG) ten times on a computer equipped with an Apple M1 Pro (10-core CPU) and 16 GB of unified memory over the same internet connection. We timed two stages independently using console logs: the time from image selection to a crop suggestion appearing on screen, and the time from accepting the crop to the image being ready for interaction on the canvas. In the second stage, storage upload and AI analysis ran in parallel, so the measured duration reflects when both branches had completed rather than their sum. We took these measurements to illustrate the magnitude of the improvement rather than to characterise performance across all image sizes or hardware configurations.

4.4.2 Results

Figure 4.7 shows the mean processing time per stage for the old and new pipeline across our ten measurements. The crop suggestion step was reduced from 7.3s to 0.3s, driven by eliminating the Cloud Run network round-trip through browser-side inference. The upload and detection step was reduced from 13.3s to 5.4s, attributable to the OpenVINO model conversion, the WebP payload change, and the parallel upload workflow. Total end-to-end time decreased from 20.6s to 5.7s.

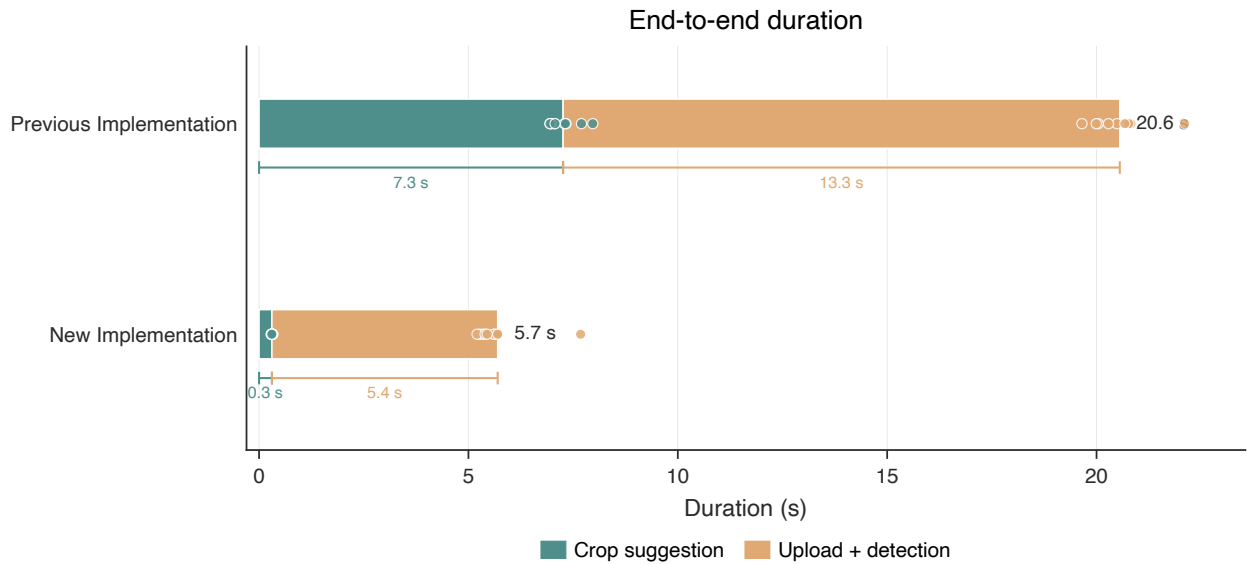


Figure 4.7: Pipeline duration for our old and new versions, split by stage. Bars show mean processing time across ten measurements of a 7.8 MB PNG image on an Apple M1 Pro. Individual measurements are shown as dots.

4.5 Conclusion

In this iteration, we transitioned the system from a local proof-of-concept to a secure, publicly hosted service. The five iteration requirements are assessed below.

R1.1: Petri Dish Detection

• Satisfied

Automatic Petri dish detection must be faster and more reliable than the FastSAM-based implementation used in Iteration 0.

The detection models we trained achieve high accuracy across all evaluated IoU thresholds, with YOLOX reaching 100%, 100%, and 90%, YOLO26 achieving 100%, 98%, and 92%, and FastSAM obtaining 48%, 46%, and 42% at IoU thresholds of 0.75, 0.85, and 0.90, respectively. All YOLO-based models demonstrate consistently strong performance, indicating reliable detection of Petri dishes across the test set. We selected YOLOX-nano for deployment given the licensing constraints of YOLO26. The achieved performance is sufficient for reliable dish detection in practical use, and browser-side inference eliminates the server round-trip. We consider this requirement satisfied.

R1.2: Processing Speed

• Satisfied

The end-to-end processing time from image upload to count result must be reduced compared to the Iteration 0 baseline.

We reduced the end-to-end processing time from 20.6 s to 5.7 s, a 72% reduction relative to the Iteration 0 baseline. Three complementary changes contributed. Moving Petri dish detection to browser-side ONNX inference eliminated the Cloud Run network round-trip. Converting the colony detection model to OpenVINO format and switching the upload payload to compressed WebP reduced the upload and detection step. Restructuring the upload workflow to process each image individually in parallel reduced overall waiting time. We consider this requirement satisfied.

R1.3: Security and Hosting

• Satisfied

The system must be securely hosted and accessible to external users without requiring local installation, with appropriate authentication and usage controls.

We deployed the system as a publicly accessible service at colonycounter.dk. User registration is gated behind email verification, and all inference endpoints require authenticated bearer tokens. We have per-user rate limiting, upload validation, per-user storage quotas, and owner-scoped Firebase rules in place. We consider this requirement satisfied.

R1.4: Workflow Completeness

• Satisfied

The system must support the complete laboratory workflow from image upload through annotation and verification to structured result export.

Our project-based organisation replaces the flat single workspace from Iteration 0, and our multi-format batch export across six formats (Excel, PDF, CSV, JSON, processed images, and raw images) covers the documentation workflows identified in the think-aloud evaluation. We addressed all workflow gaps reported by participants. We consider this requirement satisfied.

R1.5: Usability

• Partially Met

The interface must be sufficiently intuitive for a first-time user to complete a counting workflow without instruction, and must provide efficient interaction mechanisms for experienced users.

We addressed all usability gaps identified in the think-aloud evaluation in this iteration: we implemented a guided onboarding tour, a dot annotation tool for rapid colony marking, text annotation support, a right-click context menu, and keyboard shortcut tooltips. However, we did not conduct a formal usability evaluation of these features within this iteration, and their effectiveness has not been verified with users. We therefore consider the requirement partially met, and defer a formal evaluation to a subsequent iteration.

With the system now deployed and the core laboratory workflow in place, we have resolved the infrastructure limitations carried over from Iteration 0. The primary open question is counting accuracy, which we only partially addressed in Iteration 0. Iteration 2 therefore shifts focus to the detection pipeline itself, with the aim of evaluating and improving counting accuracy.

5 Iteration 2: Object Detection Model

Iteration 1 brought the system from a local proof-of-concept to a secure, publicly hosted service. But the YOLOV9C model trained for colony detection in Iteration 0 was carried forward without modification, and its limitations were not addressed. The benchmark conducted in Iteration 0 revealed a clear generalisation gap, with performance degrading substantially on images outside its training data distribution and on high-density plates where colonies overlap and merge. The test dataset used for evaluation consisted of images with morphologically similar colonies captured under good lighting conditions, meaning the model had never been tested on the diversity of colony appearances, lighting variability, glare, or challenging cases such as colonies occluded by text encountered in real laboratory environments.

Together, these limitations define the scope of this iteration. The primary objective is to develop a more robust detection model through curated training data and iterative model improvements.

Iteration Requirements

R2.1 The detection model must maintain reliable detection performance across the full range of colony densities encountered in laboratory practice, including high-density plates where colonies are closely packed or partially overlapping.

R2.2 The detection model must generalise across the range of lighting conditions encountered in real laboratory environments, including variation introduced by glare, reflections, and shadows. It should also be able to detect colonies occluded by text marker on the plate lid.

R2.3 The detection model must perform reliably across agar types and across colonies that vary in size, shape, colour, and opacity, including colonies located at the rim of the plate.

R2.4 The final pipeline must be evaluated against relevant accessible colony-counting tools and perform competitively under realistic benchmark conditions.

This chapter begins with a data section, covering the construction of new datasets, the exclusion of noisy sources, and the final data splits used throughout the modelling experiments. The modelling work then begins with a detection-head ablation study of YOLO26. RF-DETR is subsequently introduced and evaluated as a transformer-based alternative to the YOLO family. Several strategies are then explored to address its default detection ceiling of 300 objects per image. The final approach, Dual RF-DETR, runs standard full-image inference on sparse plates and uses SAHI-based inference for dense plates. A Routing Model study then evaluates lightweight approaches for assigning each plate to the appropriate detection pipeline at inference time. The chapter concludes with model calibration, post-processing, failure mode analysis, and a final comparison with competing tools on a held-out external test set.

Since YOLO26 and RF-DETR are compared throughout the chapter, they are coloured in the text and use the same colours in figures and tables when shown side by side.

5.1 Data

To improve generalisation beyond the initial datasets used in Iteration 0, two new datasets are introduced, NatureDTU and CrossLab. This section describes the construction of these datasets, reports distribution statistics, reassesses the DS1 and DS2 datasets from Iteration 0, and describes the final data splits.

5.1.1 Method

This section covers the construction of NatureDTU and CrossLab, followed by the experimental procedure used to assess the inclusion of DS1 and DS2.

NatureDTU

The NatureDTU dataset consists of two sources. The first is the publicly available Makrai et al. Veterinary Bacterial Colony Dataset described in section 2.3.2, consisting of 369 images spanning 24 bacterial species, captured on consumer mobile devices across multiple background conditions [58]. The second is an in-house collection of 211 images across 61 agar plates, each captured three times under the same conditions as a form of real-world data augmentation. The images were captured at DTU Bioengineering and consist of marine agar plates inoculated with environmental ocean samples, resulting in a diverse range of colony morphologies. Two copies of each plate were photographed using a Huawei P30 Pro and one using an iPhone 13, introducing natural variation in image quality, colour balance, and perspective. The plates contain many different microorganisms with different shapes and forms, representing challenging real-world conditions which are not present in the other available datasets. Initial annotations were generated using the YOLOV9C model developed in Iteration 0, deployed through the ColonyCounter web application, and subsequently corrected manually to resolve missed detections and false positives. Representative examples are shown in Figure 5.1.

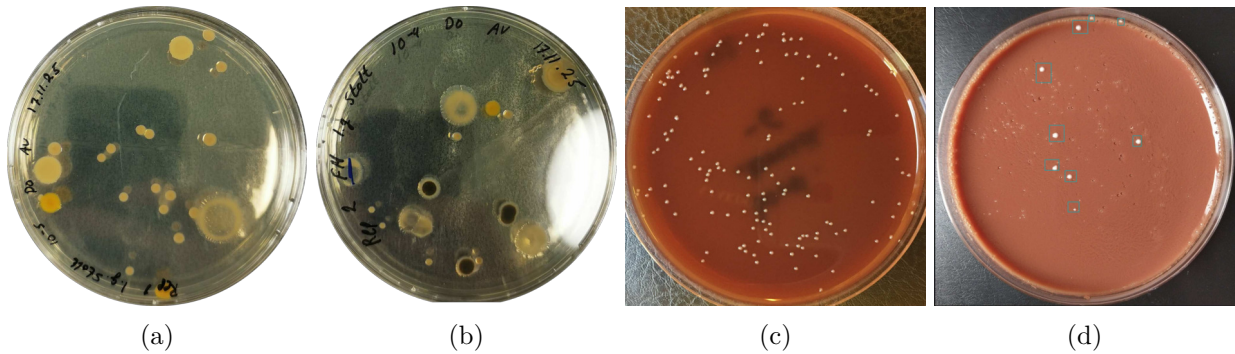


Figure 5.1: Examples from the NatureDTU dataset. (a) and (b) are images captured in-house on a mobile device, (c) is an image from the Makrai et al dataset [58], and (d) shows loosely fitting annotation boxes in an image from the Makrai et al dataset.

CrossLab

The CrossLab dataset was constructed from images acquired across different laboratories and experimental setups, providing a more domain-distant benchmark than a random split of the training data alone. The dataset consists of 65 images captured on different mobile phone cameras, covering a diverse range of colony morphologies, agar types, and bacterial species. Eight of these images originate from the same laboratory as the NatureDTU dataset. While this partial overlap means the

set is not strictly out-of-distribution, the majority of images come from sufficiently distinct domains to provide a reliable indicator of generalisation. Annotations were generated using the YOLOV9C model from Iteration 0, deployed through the ColonyCounter web application, and subsequently verified manually. Three representative examples are shown in Figure 5.2.

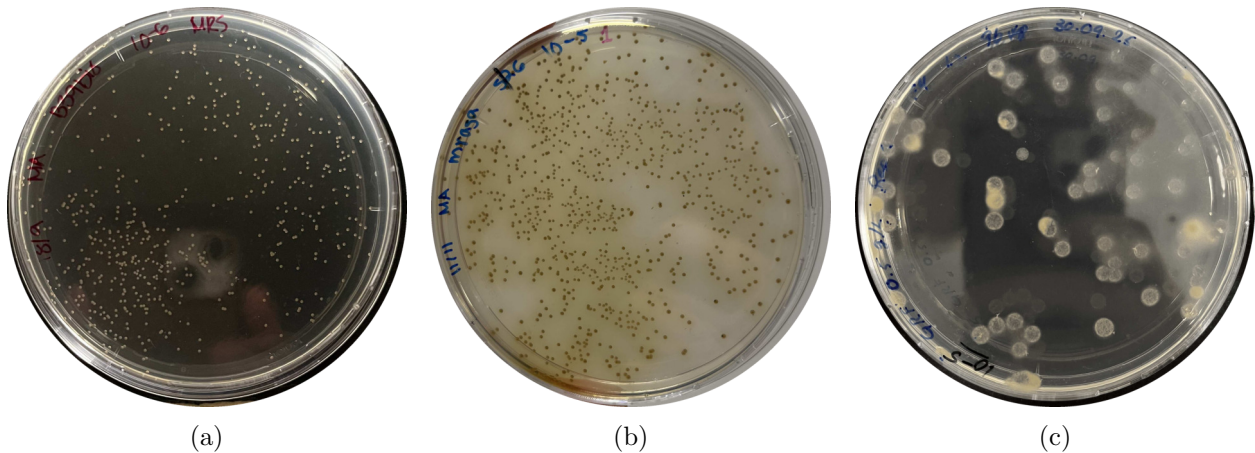


Figure 5.2: Three examples from the CrossLab dataset. The images shown here were captured in-house on mobile devices.

Assessing DS1 and DS2

To determine whether DS1 and DS2 contribute positively to model performance, two models are trained and compared: one on AGAR alone, and one on AGAR supplemented with DS1 and DS2. Both models are evaluated on the NatureDTU dataset, which was not used during training, providing a consistent and domain-distinct benchmark for comparison. The CrossLab set is deliberately excluded from this assessment, as it is reserved for final model evaluation later in this iteration. Performance is reported as mAP@50, computed by first calculating the average precision (AP) per plate and then averaging across all plates.

5.1.2 Results

This section reports the outcome of the DS1 and DS2 inclusion assessment.

Assessing DS1 and DS2

Figure 5.3 (A) shows the mAP@50 of YOLOV9C trained on AGAR alone versus AGAR supplemented with DS1 and DS2, evaluated on NatureDTU. Training on AGAR alone achieves a mAP@50 of 0.633, compared to 0.610 when DS1 and DS2 are included, a reduction of 0.023. This suggests that the additional data introduces noise rather than useful diversity. A visual inspection of both datasets reveals the likely cause. DS1 and DS2 contain images where bounding box annotations are either incorrectly placed or entirely absent for visible colonies (Figure 5.3 B,C). Exposing the model to these labels during training likely introduces contradictory supervision signals that conflict with the well-annotated AGAR data, degrading the model’s ability to localize colonies reliably on unseen images.

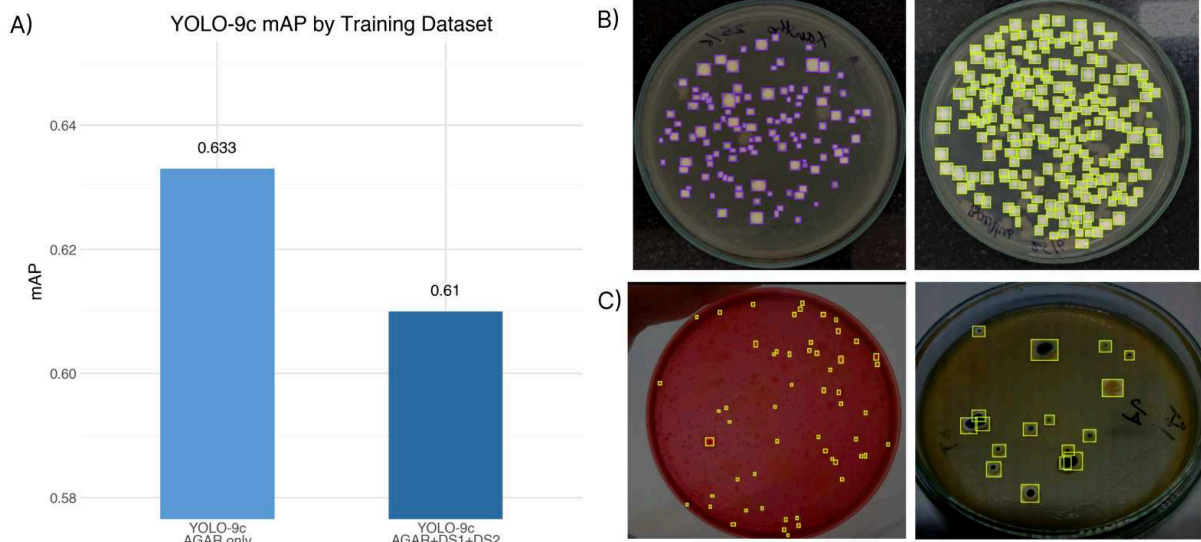


Figure 5.3: A) mAP of YOLOv9c trained on the AGAR dataset, and on the AGAR dataset + DS1+DS2. Evaluated on NatureDTU B) Images from DS2 with wrong annotations C) Images from DS1 with wrong annotations

5.1.3 Data statistics

Figure 5.4 shows a ridgeline distribution of colony counts across the three datasets, and Table 5.1 reports summary statistics. The three distributions differ substantially. The AGAR dataset is heavily skewed towards low counts, with a median of 11 and a maximum of 300, while the minimum is 0. NatureDTU and CrossLab exhibit greater variance and higher counts, with NatureDTU reaching a median of 72 and both datasets reaching a maximum of 890 colonies. The AGAR dataset is also $\sim 20\times$ larger than the other two combined. The right panel of Figure 5.4 shows the distribution of bounding box area relative to total image area. Across all datasets, bounding box area is consistently small, with objects rarely exceeding 1% of the total image area. The median relative areas for AGAR, NatureDTU, and Val II are 0.083%, 0.046%, and 0.013%, respectively.

Table 5.1: Dataset Count Summary Statistics

Dataset	N_{images}	$N_{colonies}$	mean	median	std	min	max
AGAR	18,000	336,442	25.0	11	36.2	0	300
NatureDTU	580	75,931	130.9	70	155.3	2	890
CrossLab	65	8,240	126.8	37	232.8	1	890

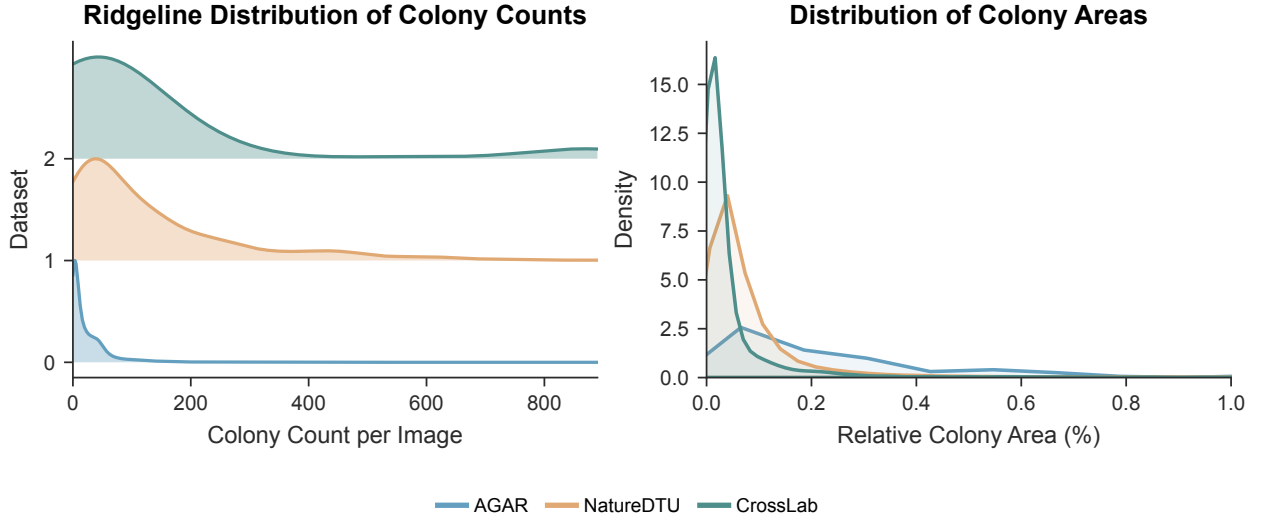


Figure 5.4: Ridgeline plot showing the count distribution of the three datasets used in Iteration 2 and line chart showing the distribution of the bounding box area relative to the whole image.

We also examined the shapes of the bounding boxes. Figure 5.5 shows the relative height and width of bounding boxes across the three datasets, along with an identity line and the corresponding R^2 value. Most boxes are relatively square. CrossLab shows the highest conformity to a square aspect ratio ($R^2 = 0.974$). The AGAR dataset exhibits a systematic pattern in annotation heights that is absent from the other two datasets. This may reflect a bias in the annotation tool used, where software default behaviour constrained boxes towards particular aspect ratios, potentially introducing a bias into the trained models.

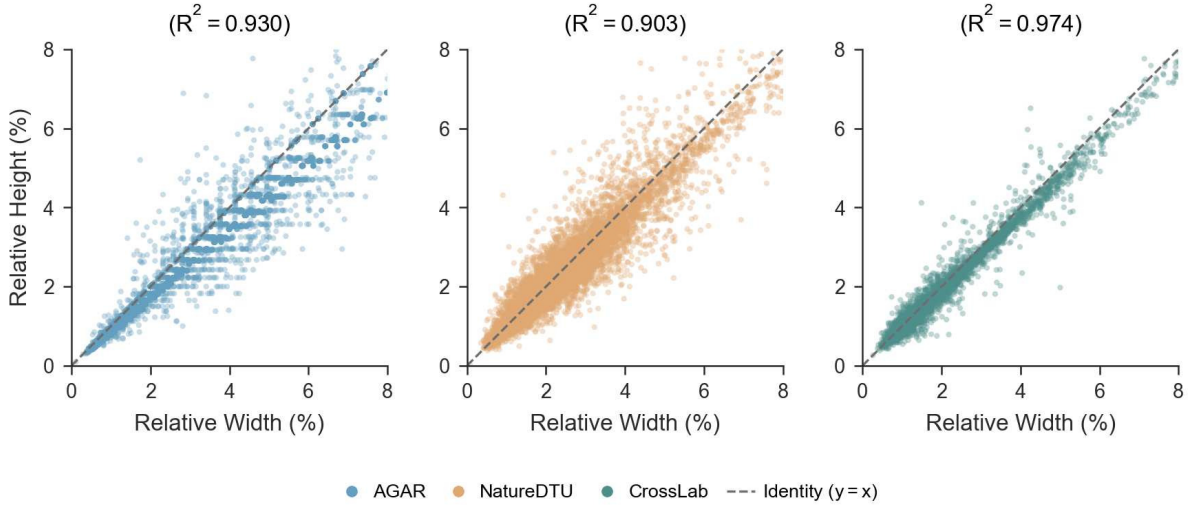


Figure 5.5: Heights vs widths of bounding boxes from the three datasets used in Iteration 2. An identity line is plotted and the R^2 value is calculated to the identity line.

NatureDTU shows the most variation ($R^2 = 0.903$), with no systematic pattern beyond the general tendency towards small object sizes shared across all three datasets. This lower conformity prompted further inspection of the NatureDTU annotations.

We observe that some annotated boxes in NatureDTU are loosely fitting, as shown in Figure 5.1d. This may account for part of the observed aspect ratio variation and has implications for evaluation at higher IoU thresholds.

5.1.4 Conclusion

The DS1 and DS2 assessment showed that supplementing AGAR with these datasets degraded performance on NatureDTU, likely due to the widespread annotation errors identified in both datasets. Including noisy labels during training introduces inconsistent supervision signals, which can actively harm generalisation rather than improve it. Both datasets are therefore excluded from further use.

The statistics reveal two notable characteristics of the remaining datasets. First, AGAR is heavily skewed towards low colony counts with a median of 11 and a maximum of 300, while NatureDTU and CrossLab better represent the higher densities encountered in practice, with medians of 72 and 37 respectively and a maximum of 890 colonies in both datasets. Second, colonies are consistently small relative to image size across all datasets, rarely exceeding 1% of the total image area. These properties motivate the final data split below, which combines AGAR’s annotated volume with NatureDTU’s domain diversity while reserving CrossLab for model comparison.

5.1.5 Final data splits

The final data configuration is summarised in Table 5.2. Here we use the AGAR dataset for pretraining and 80% of NatureDTU dataset for further training (fine-tuning), balancing annotated volume with domain diversity. The validation I set (Val I hereafter) comprises the remaining 20% of NatureDTU and is used for hyperparameter tuning. The CrossLab dataset is used as Val II, and provides the primary basis for final model comparison, offering a more reliable indicator of real-world generalisation than a random split alone. The Moucka et al. dataset described in section 2.3.3, is used as a held-out final test set and is reserved for the benchmark against competing tools.

Table 5.2: Overview of the final data split and the role of each set in the evaluation pipeline.

Set	Source	N Images	Role
Pretrain	AGAR	18,000	Model training
Train	NatureDTU (80%)	462	Model training
Val I	NatureDTU (20%)	118	Hyperparameter tuning
Val II	CrossLab	65	Model Comparison
Test	Moucka et al.	432	Final benchmark

5.2 Improving the performance of the Initial YOLO model

This section describes the experiments conducted to develop a more robust detection model than the YOLOV9C baseline from Iteration 0. We began by attempting to improve the model within the YOLO framework, by testing newer model architectures and by optimizing the configuration for our specific object detection task.

5.2.1 Method

In our previous work we employed a YOLOV9C model using the Ultralytics framework. Ultralytics have since released new iterations of the YOLO architecture including [YOLO26](#), which serves as the

YOLO baseline in this iteration.

Colonies on agar plates come in a variety of sizes, but tend to be small. As shown in the dataset statistics, the median bounding box area is below 0.1% of the total image area in all three datasets.

The standard **YOLO26** architecture performs predictions on 3 levels of feature maps independently. From the backbone detections are run on feature maps made from 3 strides, P3: stride = 8, P4: stride = 16 and P5: stride = 32. Lower-stride feature maps retain higher spatial resolution and capture fine-grained detail suited to small objects, while higher-stride maps encode more global context and contribute to detections of larger objects. Given the predominantly small colony sizes in this dataset, the choice of detection heads is therefore non-trivial.

Following a study by Chutichaimaytar et al. [66], modifying the detection heads has the potential to improve small object detection performance. We therefore performed an ablation study where we compared the **YOLO26** architecture with different detection heads, see Table 5.3. We compare 4 different configurations of detection heads (Table 5.4), with the configuration with P3, P4, and P5 being the standard model from Ultralytics. Additionally, we evaluate the previous YOLOV9C model from iteration 0, but train it on the same datasplits for a fair comparison.

Table 5.3: Detection Head Configurations, Spatial Resolution, Strides, and Receptive Fields for YOLO26 Experiment. The heads are named after the layer from the backbone which they extract features from, based on an input image of 1024×1024 .

Det. Head	Spatial Resolution	Stride	Relative detection area
P2	256×256	4	$7 \times 7 / 1024^2 \cdot 100 = 0.0046\%$
P3	128×128	8	$15 \times 15 / 1024^2 \cdot 100 = 0.021\%$
P4	64×64	16	$31 \times 31 / 1024^2 \cdot 100 = 0.092\%$
P5	32×32	32	$63 \times 63 / 1024^2 \cdot 100 = 0.38\%$

Table 5.4: Evaluated YOLO26 Architecture Combinations.

Config 1	Config 2	Config 3(Standard)	Config 4(Full Pyramid)
P2 and P3	P2, P3, and P4	P3, P4, and P5	P2, P3, P4, and P5

Training

We assess the four **YOLO26** configurations described in Table 5.4 using the large version of the **YOLO26** model. For the YOLOV9 model, we use the same compact (c) version used in Iteration 0. We pretrain all models on the Pretrain dataset and finetune on the Train dataset. For both datasets, training follows the standard Ultralytics learning rate schedule: 3 warmup epochs with an initial learning rate of 1×10^{-2} , decaying to a final learning rate of 1×10^{-5} , using the muSGD optimizer. For other hyperparameters see Table A.3.

The Ultralytics framework has a built in configuration for different data augmentation. It is implemented by specifying values for each parameter, drawing a random number from a uniform distribution within the parameter value and using that for the augmentation. This means that each training run will not be completely alike, but over a large number of epochs, the models should experience the same type of variance. In Table A.2 we present the overridden image augmentations

which we have employed, to try and simulate real world images as much as possible. We employ both colour and geometric augmentations to simulate as many different variations as possible. The complete list and meaning of parameters can be found at Ultralytics website [67].

Assessing model performance We evaluate model performance using the FiftyOne package [68] rather than the built-in evaluation methods provided by Ultralytics or Roboflow, in order to reduce evaluation bias. Performance is reported as mAP@50, computed by first calculating the average precision (AP) per plate and then averaging across all plates. This per-plate aggregation is chosen to account for the large variation in object density across plates, ensuring that images with many objects do not disproportionately influence the overall score. This also aligns with our goal of selecting a model that performs robustly across all object densities.

Assessing inference speed We assess the inference speed of the YOLO models on CPU, specifically Mac M4 Processor. Since the models are intended for CPU-based deployment, all models are converted to OpenVINO prior to evaluation. We perform 5 warmup runs, followed by 10 timed runs on one image from our Val I dataset with 150 colonies. We evaluate at resolutions 576×576 , 704×704 , 1024×1024 and 1280×1280 . We hold this up against the mAP@50 at the different scales to verify the optimal resolution for our task. The model performance is evaluated on the Val I data set.

A second validation round is performed on two models selected on the basis of Val I. One is the best performing regardless of inference speed, and the other is the model with the best trade-off of speed and performance. These two will be evaluated on their mAP@50 on the Val II dataset. While the main objective is the counting accuracy, which allows for loose fitting boxes, for downstream tasks it can be important to not include too much of the background during detection. We will therefore also investigate the performance across a range of IoU to determine how well the bounding boxes fit the ground truth. Unlike the Val I dataset, the Val II dataset has tighter fitting ground truth boxes, making it more suitable for evaluating this.

5.2.2 Results

Figure 5.6 shows the accuracy–latency trade-off for the five YOLO configurations across the evaluated input resolution. Across all models, increasing the input resolution consistently improved detection performance, albeit with diminishing returns at higher resolutions. This trend was accompanied by a substantial increase in inference time, reflecting the expected quadratic scaling of computational cost with image size. It is not shown in the figure, but higher resolutions for the models were tested, but performance dropped.

The **YOLO26** (L, p2p3) configuration demonstrated a steady and predictable trade-off between speed and accuracy. Performance improved from an mAP@50 of 0.8187 at 576 resolution to 0.8760 at 1280, representing the highest overall mAP@50 among all tested configurations. However, this came at the cost of significantly increased latency, rising from 151.2 ms to 511.0 ms.

The **YOLO26** (L, p2-p4) model showed similar behaviour, achieving strong performance at mid-to-high resolutions.

The **YOLO26** (L, p3-p5) variant exhibited competitive performance at lower computational cost. At 1024 resolution, it achieved an mAP@50 of 0.8710 with an inference time of 318.8 ms, offering

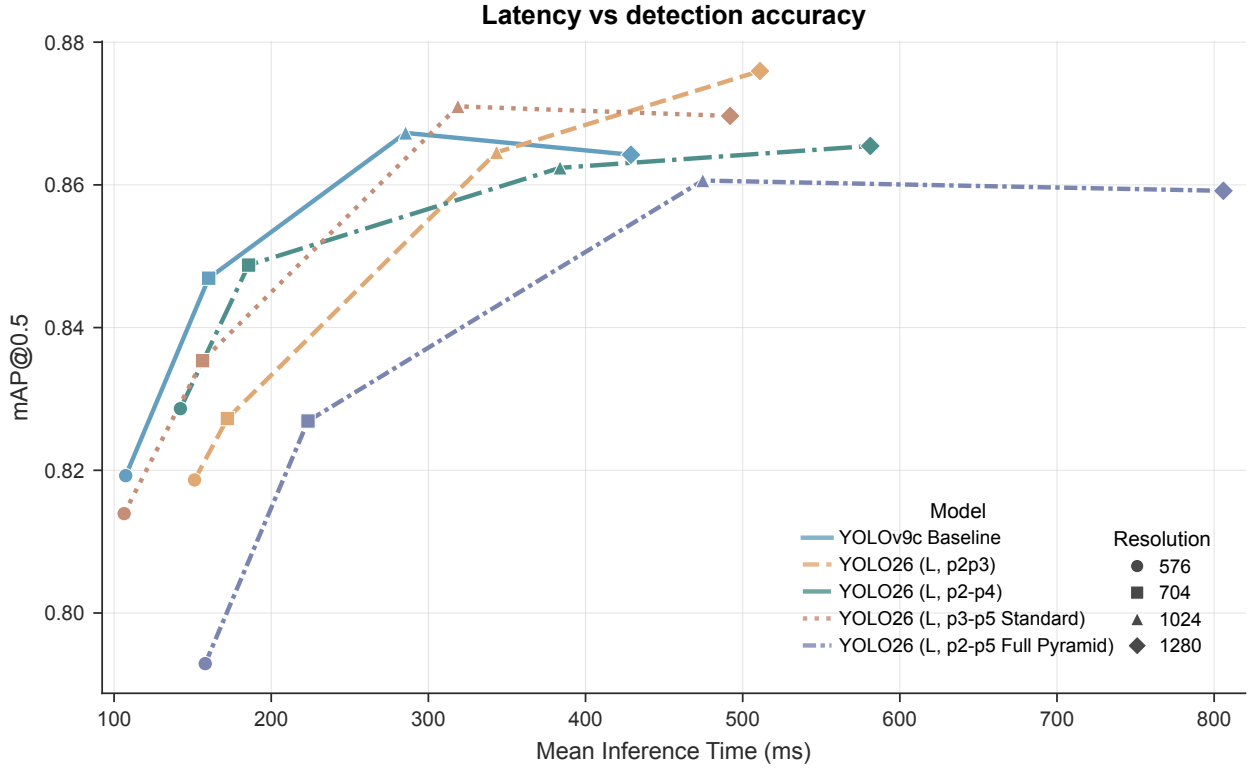


Figure 5.6: Comparison of inference time and mAP@50 of the different YOLO configurations at different resolutions on Val I.

a more efficient trade-off compared to **YOLO26** (L, p2p3). However, performance did not increase beyond this point, with slightly lower performance at 1280 resolution.

The YOLOv9c model provided a strong balance between speed and accuracy. At 1024 resolution, it achieved an mAP@50 of 0.8673 with a relatively low inference time of 285.5 ms, making it one of the most efficient configurations. While its peak performance was slightly below that of **YOLO26** (L, p2p3), it consistently offered lower latency across all resolutions.

In contrast, the **YOLO26** (Full Pyramid) configuration underperformed relative to its computational cost. Although performance improved with resolution, reaching an mAP@50 of 0.8606 at 1024, it required substantially higher inference times, particularly at 1280 resolution (805.9 ms), without achieving superior accuracy. The metrics for the best performances of each model are shown in Table 5.5.

Model	YOLOv9c	YOLO26 (L, p2p3)	YOLO26 (L, p2-p4)	YOLO26 (L, p3-p5)	YOLO26 (Full Pyramid)
mAP@50	0.8673	0.8760	0.8654	0.8710	0.8606

Table 5.5: Overall mAP@50 of each model evaluated on Val I.

All models perform almost equally on the Val I dataset, which may indicate an upper limit of detections and annotations of the dataset itself. Since there is still a slight difference, we pick out two models for further comparison using the Val II data set. The two models we chose were the **YOLO26** (L, p2p3) with an input resolution of 1280, since this had the highest mAP@50, and the **YOLO26** (L, p3-p5) at input resolution 1024, as it had substantially lower inference time,

($\Delta = 200ms$) while still maintaining high performance.

In Figure 5.7 we present the mAP@50 of **YOLO26** (L, p2p3) and **YOLO26** (L, p3-p5) across different colony count bins. Generally the two models are almost identical from ranges 0-300, with **YOLO26** (L, p2p3) outperforming slightly. A marked difference can be observed at the 300+ count range where **YOLO26** (L, p2p3) outperforms with a mAP@50 of 0.864 against the **YOLO26** (L, p3-p5) mAP@50 of 0.772.

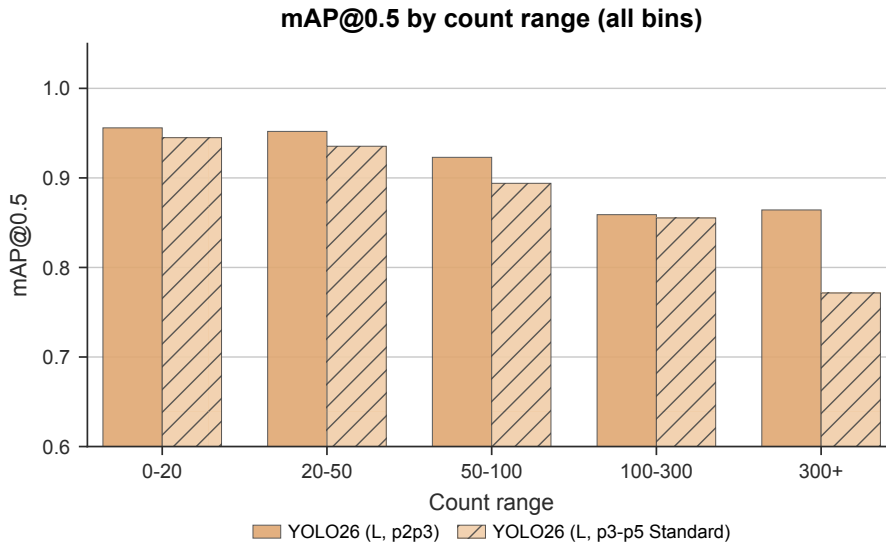


Figure 5.7: Comparison of mAP@50 of YOLO26 (L, p2p3) and YOLO26 (L, p3-p5) across different colony count bins on Val II

We further investigated whether **YOLO26** (L, p2p3) would also produce tighter bounding boxes compared to **YOLO26** (L, p3-p5). In Figure 5.8 we have plotted the performance of the two models across an IoU threshold of 0.5 to 0.8. It is clear that while they are close in performance at lower thresholds, they diverge substantially once the thresholds are raised.

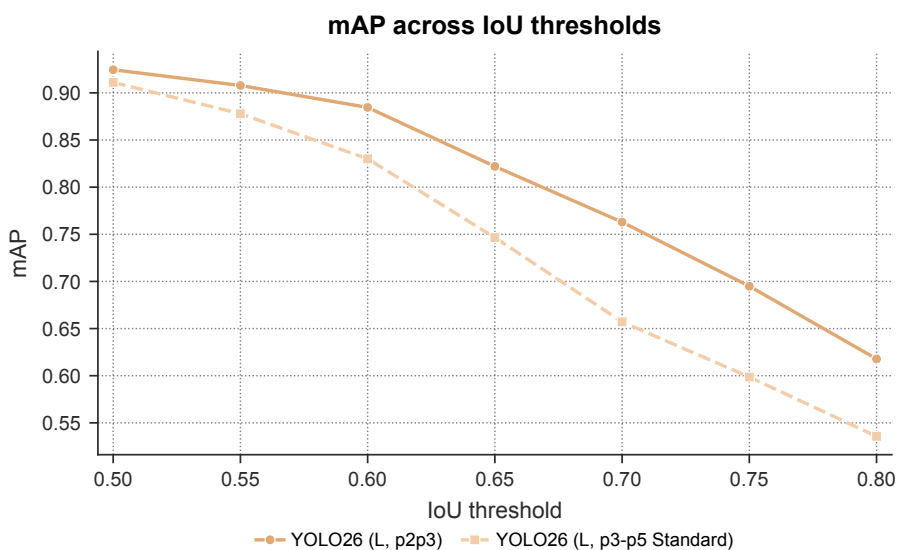


Figure 5.8: Comparison of mAP of YOLO26 (L, p2p3) and YOLO26 (L, p3-p5) across multiple IoU thresholds on Val II

5.2.3 Conclusion

The experiments demonstrate that modifying detection head configurations in **YOLO26** has a measurable, though ultimately bounded, impact on performance for detection of small colonies. Across all evaluated models, performance differences remained relatively small, suggesting that the dataset itself may impose an upper limit on achievable detection accuracy, likely due to annotation quality and inherent ambiguity in colony boundaries.

Among the tested configurations, **YOLO26** (L, p2p3) achieved the highest overall mAP@50, confirming the importance of high-resolution feature maps when targeting very small objects. Its advantage became particularly evident in high-density scenarios (300+ colonies), where it maintained substantially higher detection performance compared to alternatives. Additionally, its stronger performance at higher IoU thresholds indicates that it produces tighter and more precise bounding boxes, which is beneficial for downstream tasks requiring spatial accuracy.

However, this performance gain comes at a computational cost. The **YOLO26** (L, p3-p5) configuration offered a more favorable balance between inference speed and accuracy, achieving near-competitive mAP@50 at a notably lower latency. This makes it a strong candidate in settings where computational efficiency is a constraint and where colony count is not higher than 300.

The inclusion of all four detection heads (**YOLO26** (L, Full Pyramid)) did not yield performance improvements despite its added complexity. This may be due to overcomplicating the task. We hypothesise that the detection heads overlap too much in their spatial resolution and may compete in importance for detecting various sized objects leading to ambiguity in the final detections. Different combinations of detection heads could possibly leverage the small object detection of P_2 while maintaining a clear distinction of spatial resolutions for each detection head. We suggest that combining either P_2, P_3, P_5 or P_2, P_4, P_5 could have the potential to alleviate feature redundancy.

Overall, the results show a clear trade-off between detection performance and inference speed. The **YOLO26** (L, p2p3) configuration achieved the strongest overall performance, particularly on high-density plates and at higher IoU thresholds, indicating better localisation of small and densely packed colonies. In contrast, **YOLO26** (L, p3-p5) provided a more efficient alternative, with lower latency and only a modest reduction in mAP@50. However, since the additional latency of **YOLO26** (L, p2p3) remains acceptable in the intended workflow, we prioritise its stronger accuracy and localisation performance in the subsequent experiments. From this point onward, we refer to **YOLO26** (L, p2p3) simply as **YOLO26**.

5.3 RF-DETR

This section describes the experiments conducted to evaluate **RF-DETR** as an alternative detection model to the YOLO-based approach. **RF-DETR** is a transformer-based real-time object detector that has shown strong benchmark performance on both small object detection and out-of-distribution generalisation, making it a promising candidate for the colony counting task.

5.3.1 Method

While **RF-DETR**'s detection performance in the literature makes it an attractive candidate, transformer-based models are generally less efficient on CPU than CNN based models such as YOLO, as the

attention operations underpinning transformers are highly optimised for GPU parallelism and do not benefit from the same degree of hardware acceleration on CPU. Since the ColonyCounter application runs inference on CPU, it is therefore particularly important to characterise inference speed alongside detection performance.

We train **RF-DETR** at 576×576 and 1024×1024 resolutions and evaluate the trained models across four inference resolutions (576×576 , 704×704 , 1024×1024 , and 1280×1280) on the Val I dataset to jointly characterise detection performance and inference speed, and to identify the optimal operating point across both criteria. Inference speed and performance are benchmarked against the selected **YOLO26** model on CPU using OpenVINO-converted models. The best-performing configuration for each model is then evaluated on the Val II dataset, to assess generalisation to a more domain-distant setting.

A practical constraint of **RF-DETR** is its built-in detection limit of 300 objects per image. Images with more than 300 annotated colonies are excluded from training to respect this limit. The impact of this exclusion is examined in the results, and improving upon it is left to a later section.

Training

We use the medium variant of **RF-DETR**, as it offers a good trade-off between latency and accuracy [49]. Here we use the default built-in augmentations (random resizing, random cropping, color jittering, and horizontal flipping). Two **RF-DETR** models are trained, one at 576×576 and one at 1024×1024 . All models are pretrained by Roboflow on the COCO dataset. We pretrain our models on the Pretrain dataset and finetune them on the Train dataset (Table 5.2). For the Pretrain dataset we used a learning rate, for the non-encoder parts of model of 10^{-4} and for the Train dataset a learning rate of $5 \cdot 10^{-5}$. For both datasets we used an encoder learning rate of $1.5 \cdot 10^{-4}$. The AdamW optimizer was used across all parts of the model.

Assessing inference speed

We assess the inference speed of our **RF-DETR** models on CPU, using OpenVINO and benchmark them against **YOLO26** [44]. As described in the previous section, we perform 5 warmup runs, followed by 10 timed runs on one image from our test dataset with 150 objects (colonies). All benchmarks were run on the same Mac M4 processor used in the previous section. The models are evaluated at resolutions of 576×576 , 704×704 , 1024×1024 and 1280×1280 on plates with fewer than 300 colonies. While we normally run **YOLO26** with a detection limit of 1000, we here decrease the limit to 300 for fair comparison as **RF-DETR** has built in maximum detection limit of 300.

Assessing detection performance

Detection performance is reported as mAP@50, computed by first calculating the average precision (AP) per plate and then averaging across all plates, consistent with the evaluation protocol used for the YOLO experiments. We first evaluate all resolution configurations on Val I to identify the optimal operating point for each model with respect to both detection performance and inference speed. The best-performing configuration is then evaluated on Val II, broken down by colony count range, to assess generalisation to a more domain-distant setting and to characterise the impact of **RF-DETR**'s 300-object detection limit on performance at high colony densities.

5.3.2 Results

Figure 5.9 shows the mAP@50 versus inference time (in OpenVINO format) evaluated on Val I across four inference resolutions (576px, 704px, 1024px, 1280px) for the RF-DETR models trained at 576px and 1024px, and for YOLO26.

At 576px and 704px inference, the RF-DETR model trained at 576px performs slightly better, reaching mAP@50 scores of 0.871 and 0.879. At 1024px inference, the model trained at 1024px becomes the stronger configuration, reaching 0.891 compared with 0.875 for the model trained at 576px.

YOLO26 is substantially faster than RF-DETR at all comparable resolutions, with detection performance improving steadily with resolution and reaching a peak mAP@50 of 0.876 at 1280px. However, both RF-DETR models retain a detection performance advantage across all compared resolutions, although these differences are quite small. The largest gap is 0.05 mAP@50 at 576px inference for the RF-DETR model trained at 576px.

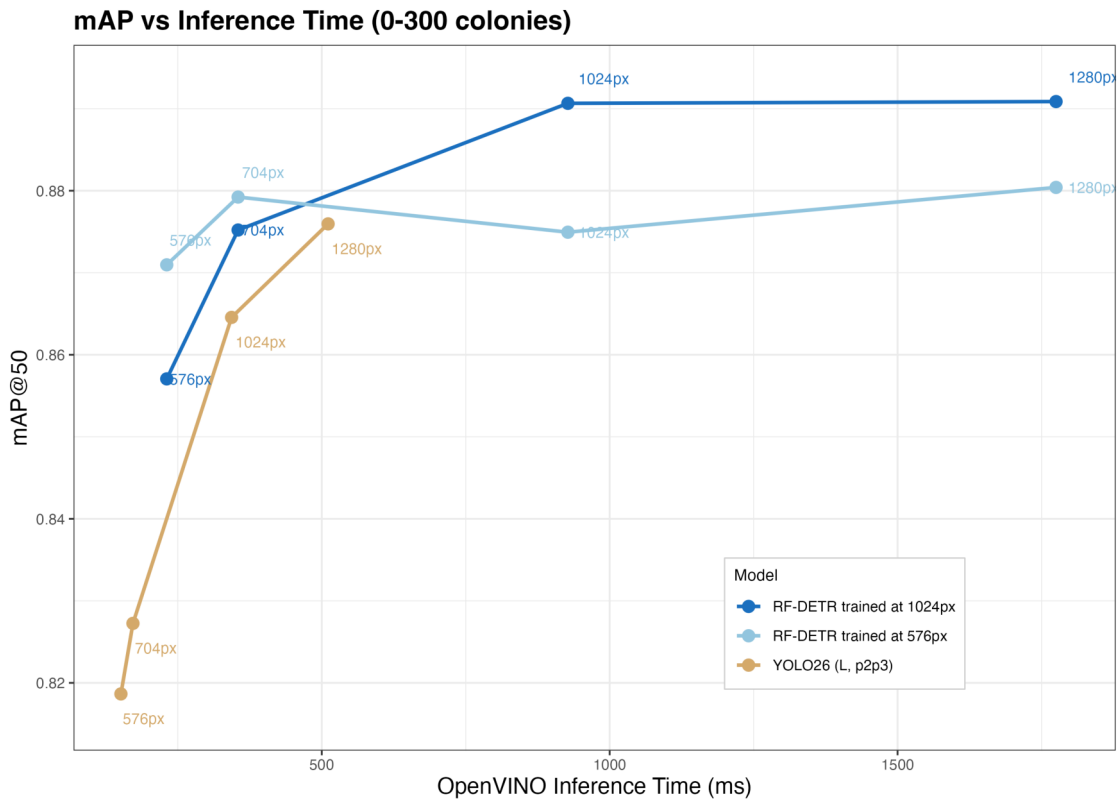


Figure 5.9: mAP@50 versus OpenVINO inference time on Val I for RF-DETR models trained at 576px and 1024px and YOLO26. Point labels indicate inference resolution.

Based on these validation results, the strongest RF-DETR configuration is therefore the model trained and evaluated at 1024px, with a peak mAP@50 of 0.891. The model trained at 576px, evaluated at 704px remains a useful speed-oriented alternative: it reaches 0.879 mAP@50 at 357 ms, only 0.012 below the best performing configuration w.r.t. to mAP@50 at roughly one third of the inference cost. We also note that there is quite a large gap w.r.t. to inference time for models evaluated at 704px and 1024px, and it might be interesting in the future to investigate whether a

more optimal model is located between these resolutions.

While the Val I set is useful for selecting the optimal inference resolution, it serves primarily as a hyperparameter tuning set and was constructed via a random split of the training data. To assess generalisation to a more domain-distant setting, the **RF-DETR** configuration trained and evaluated at 1024px is compared against **YOLO26** on Val II, broken down by colony count range (Table 5.6.)

Table 5.6: mAP@50 by range for Val II

Model	0-20 (<i>n</i> = 21)	20-50 (<i>n</i> = 10)	50-100 (<i>n</i> = 10)	100-300 (<i>n</i> = 14)	300+ (<i>n</i> = 6)	0-300 (<i>n</i> = 61)
RF-DETR (1024px)	0.97	0.98	0.94	0.90	0.39	0.95
YOLO26	0.96	0.95	0.92	0.86	0.86	0.92

The **RF-DETR** configuration trained and evaluated at 1024px consistently outperforms **YOLO26** across all density ranges within 0-300, maintaining a mAP@50 above 0.900 throughout. The advantage is smallest on sparse plates in the 0-20 range ($\Delta = 0.014$), widens considerably across the 20-50 and 50-100 ranges ($\Delta = 0.036$ and $\Delta = 0.030$), and is largest in the 100-300 range ($\Delta = 0.039$), suggesting that its relative advantage over **YOLO26** grows with colony density. Above 300 colonies, the **RF-DETR** configuration trained and evaluated at 1024px drops dramatically to a mAP@50 of 0.389 due to its hard detection limit of 300 objects per image, while **YOLO26** handles this range substantially better (0.824). This is a known limitation with the **RF-DETR**'s number of object queries to be addressed in future iterations. It should be noted that sample sizes per bin are small (*n*=6-21), which limits the strength of conclusions that can be drawn.

The 50–100 and 100–300 colony ranges are the most practically relevant, as plates below 50 colonies are quickly counted by hand and plates above 300 are conventionally considered too numerous to count. In both of these ranges, the **RF-DETR** configuration trained and evaluated at 1024px holds a clear advantage over **YOLO26**.

Comparing performance across the two validation sets (Val I & Val II) reveals an interesting pattern. Both models score higher on Val II than on Val I across the 0-300 range, which is likely explained by the fact that Val II consists of higher quality images with less reflections, glare and blur.

Figure 5.10, A shows model performance across a range of IoU thresholds. The **RF-DETR** configuration trained and evaluated at 1024px consistently outperforms **YOLO26** up to IoU 0.75, with the largest gap occurring at IoU 0.60. At IoU 0.75 the advantage reverses: **YOLO26** leads by around 4 at IoU 0.75 and 3.7 at IoU 0.80. This pattern suggests that **RF-DETR** detects more objects overall, but produces less precisely localised bounding boxes on average. The qualitative examples in Figure 5.10, B illustrate this trade-off by showing examples of **RF-DETR** bounding boxes and GT bounding boxes, with arrows pointing to objects that the **YOLO26** model misses. **RF-DETR** recovers detections on difficult objects that **YOLO26** misses, but its boxes tend to fit loosely around the ground truth. It is worth noting that the ground truth annotations themselves are not always perfectly drawn, meaning that performance differences at stricter IoU thresholds such as 0.75 and 0.80 are partly driven by annotation inconsistency rather than true localisation quality, making results at these thresholds less reliable. For most use cases at IoU 0.5-0.7, **RF-DETR** is the stronger choice. However, applications requiring tight localisation may favour **YOLO26**. Since the end goal

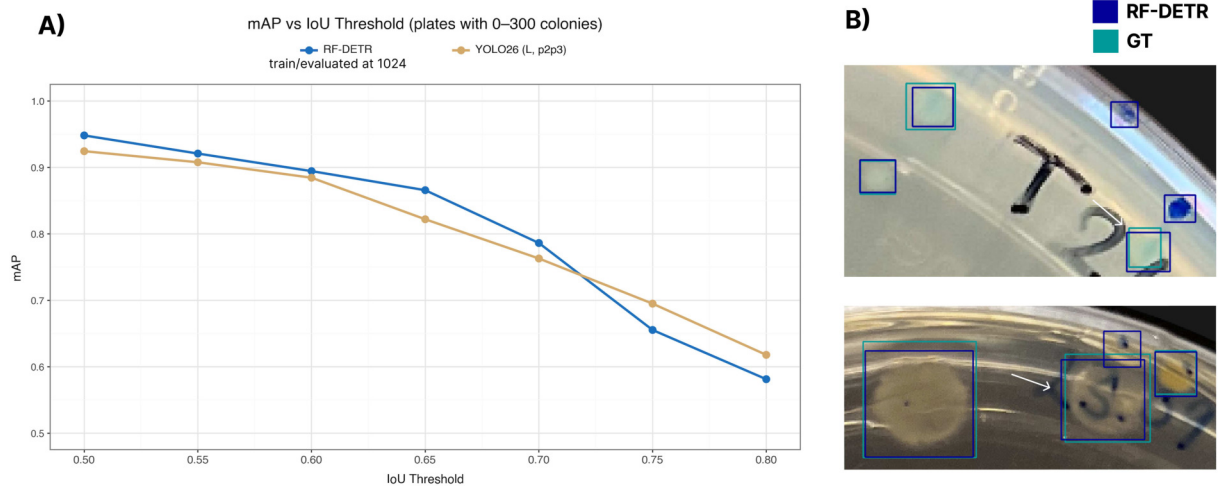


Figure 5.10: A) mAP vs IoU thresholds on Val II for the RF-DETR configuration trained and evaluated at 1024px and YOLO26. B) Example bounding boxes for RF-DETR at 0.32 confidence. Arrows point to examples where the YOLO model does not find an object but the RF-DETR model does. However, these bounding boxes for hard-to-find objects do not tightly overlap with the GT.

is colony counting rather than precise bounding boxes, loose but correct detections are preferable to missed ones, further supporting the RF-DETR configuration trained and evaluated at 1024px as the more suitable model for this task.

5.3.3 Conclusion

The RF-DETR configuration trained and evaluated at 1024px outperforms YOLO26 across all density ranges within 0-300 colonies on both validation sets at mAP@50, and is carried forward as the main baseline model, and will be referred to as Full-image RF-DETR. These results expose a critical limitation in RF-DETR's hard detection ceiling of 300 objects per image. Although plates exceeding 300 colonies are conventionally considered too numerous to count accurately, treating this as a hard cutoff is problematic in practice. A model trained on too few plates near and beyond 300 colonies will likely perform worse at the edges of the valid range, and risks undermining user trust by silently returning incomplete counts rather than flagging the result. Furthermore, we have observed from our stakeholder interviews that while 10-300 is the desired range for counting, often plates with more than 300 colonies will be counted by researchers regardless. Sometimes this is even done by dividing the plate into four equal sections, counting only one section and then using that to estimate total plate count to save time. Addressing this limitation is the focus of the following section.

5.4 Extending RF-DETR Beyond 300 Colonies

Having established RF-DETR as the stronger model across the 0-300 colony range, the remaining challenge is its hard detection limit at 300 objects. RF-DETR relies on a fixed set of object queries (default: 300), where each query corresponds to one predicted object, meaning any colonies beyond this limit are structurally undetectable, regardless of model capacity. As seen in the previous section, this manifests as a dramatic collapse in mAP@50 above 300 colonies (0.39), affecting approximately 10% of images in the dataset. In this section we evaluate different approaches to

this limitation.

5.4.1 Method

We evaluate the different approaches to the 300-query ceiling using Val I. The best performing approach is then evaluated on Val II. We present the models in Table 5.7.

Table 5.7: Overview of the models evaluated in this section.

Approach	Explanation
YOLO26	Best performing YOLO model from earlier section, included as baseline
Full-image RF-DETR	Best performing RF-DETR model from previous section, inference on full images
1000-query RF-DETR	RF-DETR model with object queries increased from 300 to 1000, inference on full images
SAHI RF-DETR	Inference is run on overlapping crops using four or nine slices
Dual RF-DETR	Uses full-image RF-DETR for plates with low colony density, uses SAHI RF-DETR for images with high colony density

Increasing the Number of Object Queries

The most direct solution to the 300-object ceiling is to increase the number of object queries, allowing the model to predict more objects per image. However, increasing the query count risks degrading performance on sparse plates, as the model must learn to suppress a large number of unused queries.

Additionally, RF-DETR only has pretrained weights initialized with 300 object queries, any increase requires training from scratch (though the DINOv2 backbone weights can still be retained), forfeiting the representational benefits of the COCO pretraining.

A higher query count also increases inference cost for every image, regardless of colony density. This means that sparse plates, which make up most of the dataset, pay the same computational penalty as the dense cases that actually require additional queries.

We evaluate a 1000 object query RF-DETR model. This was chosen as a practical upper bound for the dataset, where the densest plate contains 890 annotated colonies. This model is referred to as 1000-query RF-DETR.

SAHI Inference

Slicing Aided Hyper Inference (SAHI) is a technique designed to improve detection of small or densely packed objects by dividing an image into overlapping tiles, running inference on each tile independently, and merging the resulting predictions with overlap-based post-processing, such as NMS or related duplicate-removal methods.

Applied to dense colony plates, this allows RF-DETR to operate on sub-regions that contain fewer than 300 colonies each, effectively bypassing the query ceiling without changing the model architecture. We have visualized the SAHI slicing on a Petri dish in Figure 5.11

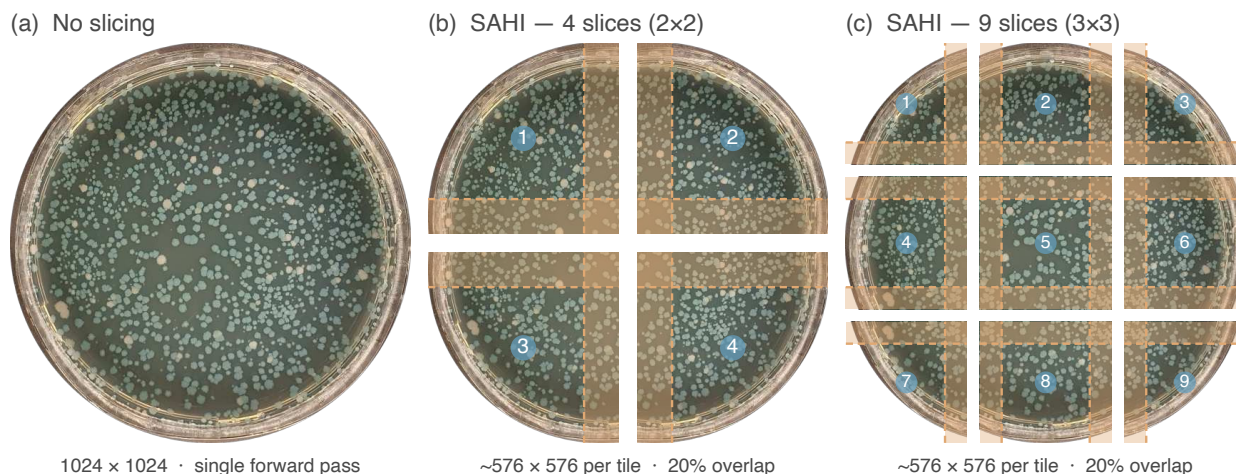


Figure 5.11: Illustration of SAHI slicing configurations on a dense agar plate image. **(a)** Full image processed in a single forward pass. **(b)** Configuration using four slices (2×2): the image is divided into four overlapping tiles, each processed independently at 576×576 . **(c)** Configuration using nine slices (3×3) with nine tiles at the same per-tile resolution. Amber regions indicate the overlap zones shared between adjacent tiles, where duplicate detections are removed during post-processing.

A limitation of SAHI is that tiling reduces the spatial context available to the model at inference. On dense plates this is less of a concern, as each tile still contains a representative sample of colonies. On sparse plates however, individual tiles may contain very few or no colonies at all, depriving the model of the broader plate-level context that informs what a colony looks like relative to its surroundings.

We evaluate SAHI [RF-DETR](#) using two slicing configurations: one using four slices from a 2×2 layout and one using nine slices from a 3×3 layout. In both cases, each tile is processed at 576px resolution. Each configuration is evaluated with $\sim 10\%$ and $\sim 20\%$ overlap. Higher overlap increases the chance that colonies near tile boundaries are fully visible in at least one tile, but also increases the number of duplicate detections that must be removed during post-processing.

A natural concern with this approach is increased inference time, as multiple forward passes are required per image. However, since each tile is a smaller crop, inference can be run at a reduced resolution of 576px rather than 1024px. As shown in Figure 5.9, inference at 576px is approximately four times faster than at 1024px, meaning that with four tiles the per-tile speed gain largely offsets the cost of processing multiple tiles, resulting in comparable overall inference time and with nine tiles, processing time only roughly doubles.

Dual RF-DETR

Rather than applying SAHI to all images uniformly, an alternative is to apply it selectively. Dense images are routed through a SAHI-based inference pipeline, while sparse images are processed as standard full-image forward passes. This directly addresses the spatial context concern raised in the previous section: by restricting SAHI to dense plates, sparse images retain full plate-level context during inference.

We evaluate this as Dual [RF-DETR](#), a two-branch setup that combines Full-image [RF-DETR](#) for sparse plates with the best-performing SAHI [RF-DETR](#) configuration for dense plates. In this section,

routing is performed using ground-truth colony counts. This isolates the detection performance of the two-branch strategy from the separate problem of estimating the routing decision, which is addressed in the following routing-model section.

As a single-model alternative to selective SAHI, we also evaluated a mixed-training Hybrid RF-DETR model trained on full images for sparse plates and crops generated using four slices for dense plates. This was intended to test whether one model could handle both inference contexts under a routed SAHI setup. However, since the Hybrid model did not improve the final strategy we do not include the results, but instead report them in Appendix A.6.

Implementation Details

All RF-DETR variants in this section use the same training schedule, learning-rate settings, and image augmentations as the Full-image RF-DETR model described in the preceding RF-DETR section, unless otherwise stated. The main differences between variants are the number of object queries, the training image format, and the inference strategy.

Model performance is evaluated using mAP@50 throughout this section. Val I is used for configuration selection, including SAHI overlap, the number of SAHI slices and the Dual RF-DETR routing threshold. Val II is reserved for the final comparison of the selected strategy.

5.4.2 Results

Increasing the Number of Object Queries

Results on the Val I set are shown in Table 5.8 for the 0-300 range and the 300+ range.

Table 5.8: 1000-query RF-DETR performance on Val I using mAP@50 with 2 decimals. The 0-300 and 300+ ranges contain n=106 and n=12 plates, respectively.

Model	0-300 (n=106)	300+ (n=12)
1000-query RF-DETR	0.87	0.78
Full-image RF-DETR	0.89	0.62
YOLO26	0.88	0.88

Within the 0-300 range, 1000-query RF-DETR scores a mAP@50 of 0.87, compared to 0.89 for Full-image RF-DETR and 0.88 for YOLO26. This degradation is consistent with the expectation that a larger query count forces the model to suppress a greater number of unused predictions on sparse images.

Above 300 colonies, performance improves considerably relative to Full-image RF-DETR (0.78 vs. 0.62), confirming that the query ceiling was the dominant bottleneck in this range. However, 1000-query RF-DETR still falls short of YOLO26 (0.88) on dense plates. Overall, increasing the query count trades a performance reduction across the 0-300 range, which constitutes approximately 90% of the dataset, for only partial recovery above 300 colonies, without reaching the level of YOLO26 in that regime. Thus, increasing the query count reduces the dense-plate failure but does not provide a strong enough tradeoff to replace the default Full-image RF-DETR model.

SAHI Inference

We evaluate SAHI RF-DETR by varying two inference settings: the number of slices and the overlap between adjacent slices. The image is either split into four slices using a 2×2 layout or into nine

slices using a 3×3 layout. For each slice count, we evaluate approximately 10% and 20% overlap on Val I.

Table 5.9: SAHI performance by slice count and overlap on Val I. Values are mAP@50 rounded to two decimals. The 0–300 and 300+ ranges contain $n=106$ and $n=12$ plates, respectively. A grey third decimal is shown where needed to distinguish close rounded values.

Slices	Overlap	0–300 (n=106)	300+ (n=12)
4	20%	0.88	0.85
	10%	0.88	0.87
9	20%	0.87	0.88 ₁
	10%	0.86	0.87
Full-image RF-DETR		0.89	0.62
YOLO26		0.88	0.87 ₈

Table 5.9 shows a clear difference between the two density regimes. For plates with fewer than 300 colonies ($n=106$), Full-image RF-DETR achieves the highest performance, outperforming all SAHI configurations. Among the SAHI configurations, using four slices performs best at both overlap settings, while using nine slices is slightly lower at both 10% and 20% overlap. At these lower densities, the RF-DETR model does not exceed its object query limit. Consequently, processing the full image in a single forward pass likely provides the broader plate-level context needed for reliable detection. This advantage of retaining spatial context also explains why using four slices outperforms the more heavily cropped setup using nine slices.

When plates exceed 300 colonies ($n=12$), Full-image RF-DETR drops sharply, a direct consequence of its query ceiling. SAHI reliably rescues performance across all configurations. Using nine slices with 20% overlap achieves the highest mAP@50, closely followed by YOLO26. Both configurations using four slices also perform strongly, though here the 10% overlap setting outperforms the 20% setting. The reason is likely structural: with only four tiles, each covers a larger area and therefore contains more colonies per tile. At 20% overlap, the expanded boundary regions may push individual tile counts closer to the 300-query ceiling, causing some detections to be dropped. The smaller crops produced when using nine slices avoid this issue, which is why higher overlap benefits that setting but not the setting using four slices.

These results indicate that SAHI should not be applied uniformly, but is valuable for plates above the query ceiling. The best SAHI RF-DETR configuration for dense plates uses nine slices with 20% overlap. From this point onward, SAHI RF-DETR refers to this selected configuration unless otherwise stated.

Dual RF-DETR

We evaluate Dual RF-DETR, which combines the strongest model for each density regime. Based on the previous results from the Val I dataset, this pairs Full-image RF-DETR for sparse plates with SAHI RF-DETR for dense plates.

Table 5.10 breaks down performance across finer density bins to determine the optimal routing threshold between the two models.

Table 5.10: mAP50 scores by model and bin range for Val I. Values are rounded to two decimals. SAHI RF-DETR denotes the selected configuration using nine slices with 20% overlap. The best performing model in each bin is highlighted.

Model	0-100 ($n = 72$)	100-200 ($n = 23$)	200-300 ($n = 11$)	300+ ($n = 12$)
Full-image RF-DETR	0.90	0.92	0.81	0.62
SAHI RF-DETR	0.87	0.89	0.82	0.88

Full-image RF-DETR is the stronger choice for plates with fewer than 200 colonies, outperforming SAHI RF-DETR by 0.03 in the 0-100 bin (0.90 vs. 0.87) and by 0.03 in the 100-200 bin (0.92 vs. 0.89). The ranking reverses in denser plates: SAHI RF-DETR leads by 0.02 in the 200-300 bin (0.82 vs. 0.81) and by 0.26 above 300 colonies (0.88 vs. 0.62), where Full-image RF-DETR collapses due to the query ceiling. These results suggest a routing threshold around the 200 range, with Full-image RF-DETR handling sparse plates and SAHI RF-DETR taking over for dense ones. It should however be noted that sample sizes in the 200-300 and 300+ bins are small ($n=11$ and $n=12$ respectively), which limits the strength of conclusions that can be drawn about the precise threshold. A further practical consideration is that the routing threshold in deployment will be determined by a density proxy rather than the ground truth count, introducing additional variance in which model is applied to each image. Setting the threshold at 200 is nonetheless a robust choice: misrouting an image from the 100-200 range to SAHI RF-DETR incurs only a small performance penalty (0.03), and similarly for routing a 200-300 image through Full-image RF-DETR (0.02). By contrast, misrouting a 300+ image to Full-image RF-DETR carries a severe cost (0.26), making it far more important to route dense plates correctly than to optimise the boundary between the two sparse bins. A threshold of 200 therefore provides a comfortable buffer below the 300-colony ceiling, giving the routing signal sufficient margin to absorb estimation error before the large performance drop due to the object query ceiling.

This motivates the final Dual RF-DETR configuration: Full-image RF-DETR below 200 colonies and SAHI RF-DETR from 200 colonies onward.

$$\text{Dual RF-DETR}(x) = \begin{cases} \text{Full-image RF-DETR}, & \text{count}(x) \leq 200 \\ \text{SAHI RF-DETR}, & \text{count}(x) > 200 \end{cases}$$

Evaluation on Val II

Based on the results from the Val I dataset, no single-model configuration dominates across all density regimes. Configurations trained on full-resolution images perform best in the sparse ranges, but collapse above 300 colonies due to the query ceiling, while SAHI-based configurations recover performance above 300 at a modest cost on sparse plates. Dual RF-DETR is the only configuration that avoids this tradeoff by design, pairing the strongest sparse model with the strongest dense model independently. For this reason, Dual RF-DETR is carried forward for evaluation on Val II, alongside YOLO26 as a baseline.

The results are shown in Table 5.11. However results in the 200-300 and the 300+ bin should be

interpreted with caution given the very small sample sizes ($n=2$ and $n=6$ respectively), meaning that conclusions about the dense regimes on this set should be treated as indicative rather than definitive.

Table 5.11: mAP@50 by colony count range on Val II. Values are rounded to two decimals. Dual RF-DETR combines Full-image RF-DETR below 200 colonies with SAHI RF-DETR from 200 colonies onward, using ground-truth counts for routing during evaluation. SAHI RF-DETR denotes the selected configuration using nine slices with 20% overlap. The best performing model is highlighted.

Model	0-100 ($n = 41$)	100-200 ($n = 12$)	200-300 ($n = 2$)	300+ ($n = 6$)
Dual RF-DETR	0.97	0.90	0.94	0.90
YOLO26	0.95	0.86	0.84	0.86

Across the 0-100 range, Dual RF-DETR leads at 0.97, ahead of YOLO26 at 0.95. In the 100-200 range Dual RF-DETR remains ahead at 0.90, with YOLO26 trailing behind at 0.86.

In the 200-300 range, Dual RF-DETR leads at 0.94, ahead of YOLO26 (0.84). This is consistent with the Val I results, where SAHI RF-DETR also outperformed the full-image model in this density range. However, with only $n=2$ plates, this result should not be interpreted beyond noting the direction of the trend.

Above 300 colonies, Dual RF-DETR reaches 0.90, outperforming YOLO26 by a meaningful margin (0.86). Notably, this represents an improvement over Dual RF-DETR’s performance in the same regime on Val I (0.88), and the advantage over YOLO26 widens relative to Val I where the two were essentially tied (0.88 vs. 0.88), suggesting SAHI RF-DETR generalises well to a more diverse and domain-distant setting. These results should nonetheless be treated as indicative rather than definitive given the small sample size ($n=6$), though the high number of colonies per plate (up to 880) lends some stability to the individual AP estimates.

5.4.3 Conclusion

The experiments establish that Dual RF-DETR offers the strongest detection performance across all density regimes. Full-image RF-DETR handles plates with fewer than 200 colonies and SAHI RF-DETR handles denser plates, together achieving a mAP@50 of 0.95 across the 0–300 colony range and 0.90 above 300 colonies on Val II. These figures were produced by routing each plate using its known ground-truth colony count, which is only available during evaluation. In practice the colony count is not known before running the detection model, so the system requires a separate step to decide which pipeline to use for each incoming plate. The following section evaluates three lightweight approaches and assesses whether any can replace this ground-truth shortcut without compromising detection reliability.

5.5 Routing Architecture

For the dual pipeline to work, plates need to be routed to the correct detection path before inference begins.

The simplest approach is to use Full-image RF-DETR as the routing signal. Its count output already

determines which path a plate takes. For sparse plates the routing pass and the detection pass are the same operation, so no additional processing is required. For dense plates it is a different story. A full forward pass runs only to determine that SAHI is needed, after which SAHI runs as a second pass. In a batch with many dense plates that time adds up.

Running both models on every plate is another option, but this doubles processing time regardless of density and is therefore not desirable.

So the question is, can a dedicated lightweight router achieve the same routing reliability as Full-image **RF-DETR** at lower cost? This section evaluates three candidates to find out. The evaluation is intentionally broad rather than deep, selecting one or two representative configurations per method to reveal its feasibility.

Three approaches were selected to span classical image analysis, regression, and detection (Table 5.12).

Table 5.12: Routing candidate approaches and rationale for selection.

Approach	Rationale
Otsu thresholding <i>Classical image analysis</i>	Requires no training data, only threshold tuning. Otsu thresholding has also been used in prior colony counting tools.
ConvNeXt-Atto <i>Count regression</i>	A lightweight learned model that predicts colony count directly from the full image, avoiding the need to localise individual colonies.
YOLOX-Nano <i>Object detection</i>	Already integrated in the pipeline for dish cropping, so it introduces no new framework dependency. Given prior experience with YOLO-based detection in this project, a small YOLOX model is a natural candidate to repurpose for routing.

For this to be worthwhile, any such router must satisfy several practical constraints.

- It must avoid critical routing failures. Plates with more than 300 colonies must not be sent to Full-image **RF-DETR**, which cannot detect beyond its 300-query ceiling regardless of confidence threshold.
- Routing latency must not perceptibly delay the upload workflow on a single CPU core.
- The model must be small enough to ship as a web asset if deployed on the client side.
- The routing threshold must be adjustable without retraining, so that the 200-colony boundary can be revised as training data grows.

Not all routing errors carry the same cost. Sending a plate with more than 300 colonies to Full-image **RF-DETR** produces a structurally incomplete count rather than an imprecise one, because the 300-query ceiling is a hard architectural limit. For this reason we classify routing errors by severity. Plates with ground truth in the range **200-300 represent moderate failures**, as Full-image **RF-DETR** can still attempt a meaningful count. Plates with ground truth above **300 represent**

critical failures, where the count is guaranteed to be truncated. In Val I, 12 of the 23 dense plates have ground truth above 300, so more than half of all possible dense-to-sparse misroutes on this set are critical.

All three estimate colony count and apply a fixed threshold of 200 colonies to produce the routing decision. Plates below this threshold are referred to as sparse and plates above as dense throughout this section. Full-image RF-DETR is included as the deployment baseline, and any lightweight alternative must match its routing performance to be worth using. All inference times are measured as the mean per-image time on Val II, on a single CPU core (Intel Xeon Gold 6226R, one thread, five-image warm-up, disk I/O excluded).

5.5.1 Routing Approaches

The following sections describe how each candidate produces a routing estimate, along with the Full-image RF-DETR baseline.

Classical Image Analysis

Otsu thresholding [22] is a classical image analysis technique that partitions pixel intensities into two classes by minimizing intra-class variance, adapting automatically to varying plate brightness without any learned parameters. Applied to the full plate image, the dominant rim-to-agar contrast dominates the histogram and defeats reliable colony detection. Thresholding is therefore restricted to small circular patches (radius $\approx 17\%$ of the image dimension) extracted from the agar interior, forcing the algorithm to separate colonies from local background rather than plate from rim. Up to five patch positions are evaluated in a cardinal cross layout (centre, north, east, south, west). Connected components with area in the range $[30, 50,000]$ px² are counted as colonies. The lower bound excludes noise pixels and the upper bound excludes merged colony blobs and agar artefacts. Raw patch counts are corrected by a single multiplicative scale factor fitted on the training set to estimate the total plate count from the sampled area. Full pipeline hyperparameters and calibration details are given in Appendix A.8.1. Figure 5.12 illustrates the five patch positions evaluated.

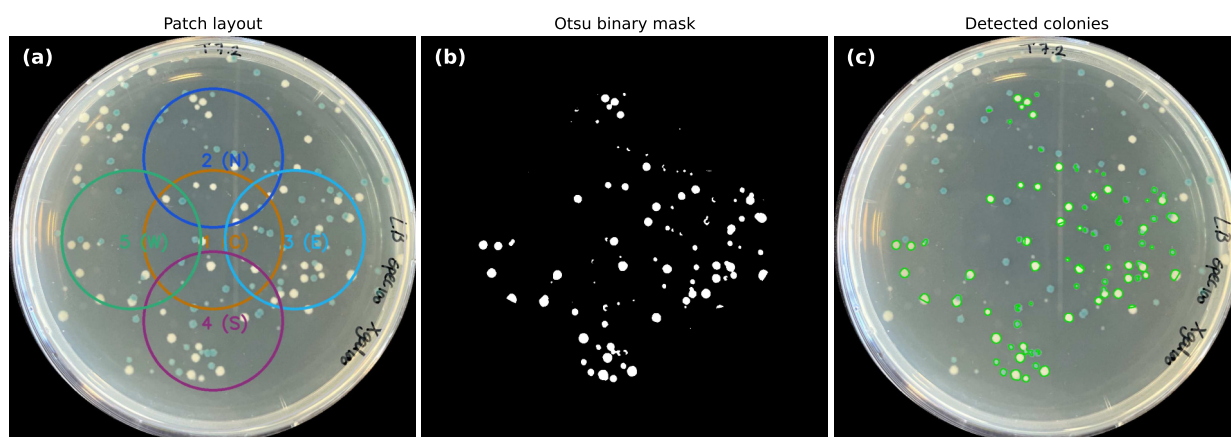


Figure 5.12: The five circular patch positions evaluated for the Otsu router, arranged in a cardinal cross layout (centre, north, east, south, west). The selected 3-patch configuration uses centre, north, and east.

Count Regression

ConvNeXt-Atto regression was evaluated as a lightweight learned alternative that requires no colony localisation. Rather than detecting where each colony is on the plate, it predicts the total count directly from the full plate image. A ConvNeXt-Atto backbone [69] (3.7 M parameters, 13.6 MB ONNX) from the `timm` library [70] is fine-tuned for this purpose. The network predicts $\log(1 + \text{count})$, which compresses the target range and stabilises training across the wide distribution of plate densities. Three input resolutions are evaluated: 224, 384, and 1024 pixels. Training uses Huber loss with differential learning rates to preserve ImageNet backbone features while adapting the regression head. The model is pretrained on the Pretrain dataset and fine-tuned on the Train data. Full architecture and training hyperparameters are given in Appendix A.8.2.

Object Detection

YOLOX-Nano (≈ 0.5 M parameters, 3.6 MB ONNX) is a lightweight single-stage anchor-free detector repurposed here as a colony counter, introducing no new framework dependency beyond the detection pipeline already used in earlier iterations. The model is trained for single-class colony detection, and the routing estimate is the number of detected objects at the selected confidence and NMS thresholds. Three input resolutions are evaluated: 416, 640, and 1024 pixels. For each resolution, a grid sweep over confidence thresholds (0.01-0.50) and NMS thresholds (0.30-0.60) is performed on Val I, selecting the combination that minimises MAE. The model is pretrained on the Pretrain dataset and fine-tuned on the Train dataset. Full architecture, training hyperparameters, and inference configuration are provided in Appendix A.8.3.

Full-image RF-DETR Deployment Baseline

Full-image RF-DETR serves as the deployment baseline. It requires no additional component. The colony count returned by the standard detection forward pass is used directly as the routing signal, forwarding to SAHI RF-DETR when the count approaches the 300-query ceiling. For sparse plates, the routing pass and the detection pass are the same operation and incur no additional cost. The extra processing time therefore scales only with the proportion of dense plates in the submitted batch. The model is run at the confidence threshold that maximises F1 on Val I.

5.5.2 Evaluation on Validation I

The three lightweight approaches are tuned on Val I (118 images: 95 sparse and 23 dense). Per-configuration results for all 11 tested configurations are reported in Appendix A.7. This section focuses on the selected configuration for each approach.

Val I contains 95 sparse and 23 dense plates, a roughly 80/20 split. Standard accuracy is therefore misleading as a routing metric: a router that classifies every plate as sparse would achieve 80% accuracy while failing on every dense plate. Balanced accuracy addresses this by averaging the true positive rate (TPR) and true negative rate (TNR) across both classes, giving equal weight to sparse and dense performance regardless of class size:

$$\text{Balanced accuracy} = \frac{\text{TPR} + \text{TNR}}{2}$$

Figure 5.13 plots routing balanced accuracy against mean CPU inference latency for all 11 configurations. The three approaches occupy distinct performance bands. Otsu configurations plateau at

balanced accuracies around 0.81 regardless of patch count, while Regression and YOLOX configurations reach substantially higher balanced accuracies and improve monotonically with resolution.

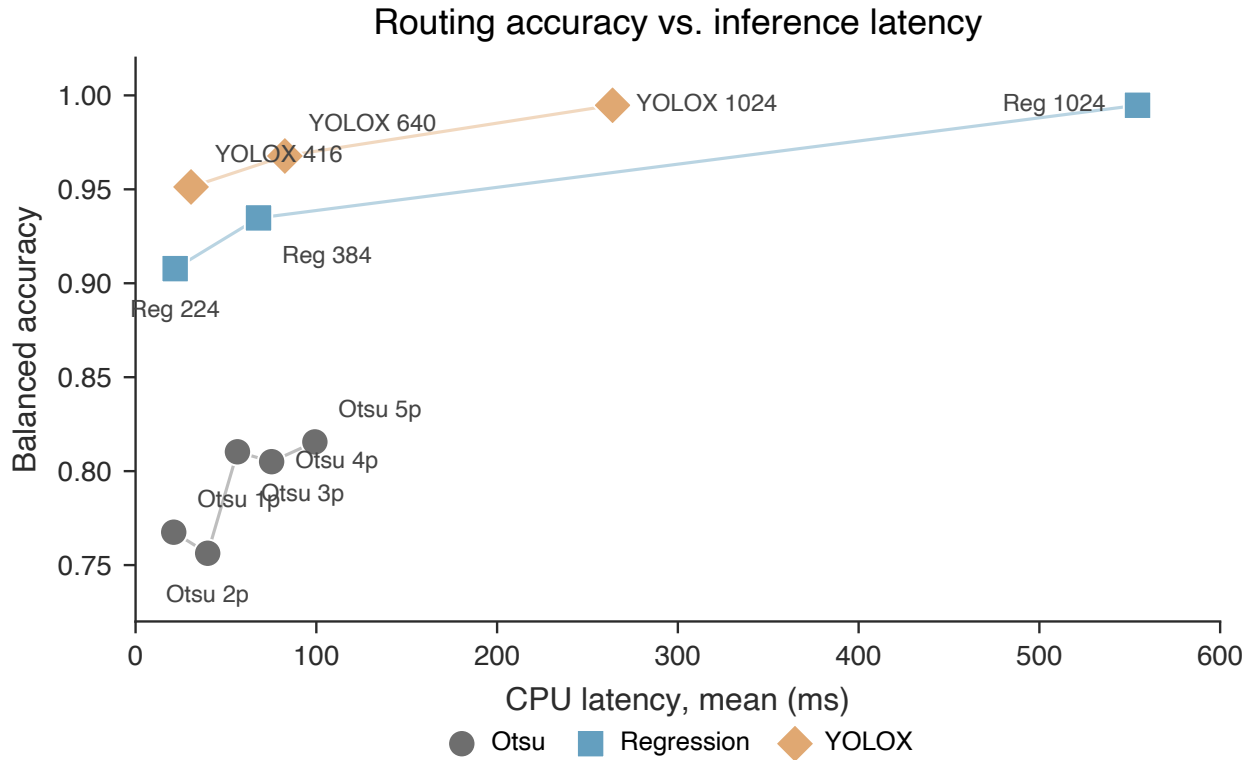


Figure 5.13: Routing balanced accuracy versus mean CPU latency for all 11 Val I configurations. The three approaches occupy distinct performance bands. Selected configurations per approach are marked with an asterisk.

Otsu saturates at a balanced accuracy of approximately 0.81 after three patches. Adding further patches increases latency without improving performance. The structural cause is consistent across all patch counts. At high colony density, colonies merge into large connected regions that the algorithm cannot reliably separate, limiting performance regardless of patch count. As discussed in the background chapter, classical image analysis methods are known to struggle with images outside the distribution they were configured on. Otsu 3-patch is selected as the representative configuration.

Among the Regression configurations, Reg 1024 achieves near-perfect routing accuracy on Val I but at 554 ms per image, roughly eight times the latency of Reg 384. Reg 384 reaches a balanced accuracy of 0.935 at 68 ms and is selected as the representative regression configuration. All Regression configurations avoid critical failures. Their misroutes are confined to plates in the moderate zone (ground truth 200-300).

YOLOX consistently outperforms Regression at comparable latencies. YOLOX 640 achieves a balanced accuracy of 0.968 at 83 ms, outperforming Reg 384 by 3.3 percentage points at similar latency. YOLOX 1024 offers only 2.7 percentage points more at roughly three times the inference cost and is not selected. Like all Regression configurations, all YOLOX configurations avoid critical failures on Val I.



Figure 5.14: Per-plate routing outcome for all 11 Val I configurations. Each row is one configuration and each dot is one plate positioned by its true colony count. Green indicates a correct routing decision, orange a minor misroute (ground truth ≤ 300), and red a critical failure (ground truth > 300 sent to Full-image RF-DETR, which cannot detect beyond its 300-query ceiling). The dashed red line marks Full-image RF-DETR’s 300-colony detection ceiling. Asterisks mark the selected configuration within each approach.

Figure 5.14 shows the per-plate routing outcome for all 11 Val I configurations. Otsu 3-patch produces three critical failures (plates with ground truth 378, 388, and 404 sent to Full-image RF-DETR), while every Regression and YOLOX configuration avoids critical failures entirely, with misroutes confined to plates within 50 colonies of the routing threshold.

5.5.3 Final Evaluation on Validation II

Table 5.13: Selected configuration per routing approach on Val I.

Approach	Config	Bal. Acc.	Recall dense	MAE	Latency
Otsu	3-patch	0.810	0.652	52.9	56 ms
Regression	Reg 384	0.935	0.870	17.9	68 ms
YOLOX	YOLOX 640	0.968	0.957	7.9	83 ms

The three selected configurations (Table 5.13) are evaluated on Val II (61 images: 53 sparse and 8 dense), a held-out set sourced from a different laboratory context than the training data. Full-image RF-DETR is included as the deployment baseline. Val II contains 6 dense plates with ground truth above 300, so any dense-to-sparse misroute on these plates constitutes a critical failure.

Table 5.14: Val II results for the three selected routing approaches and Full-image RF-DETR. Critical failures are plates with ground truth > 300 sent to Full-image RF-DETR.

Approach	Config	Bal. Acc.	Recall High	MAE	Latency	Critical
Otsu	3-patch	0.890	0.875	58.8	67 ms	0
Regression	Reg 384	0.919	0.875	55.0	65 ms	0
YOLOX	YOLOX 640	0.938	0.875	44.3	79 ms	1
Full-image RF-DETR	N/A	1.000	1.000	54.9	3,342 ms	0

In 5.14 we present the results from Val II. YOLOX 640 achieves the highest balanced accuracy among the lightweight approaches on Val II (0.938), with no false over-routes and the lowest MAE (44.3). Five of the six plates with ground truth above 556 colonies are correctly identified as dense. The single exception is the plate with 852 colonies, which the model predicts as 149 and routes to Full-image RF-DETR, constituting a critical failure. The confidence and NMS thresholds tuned on Val I do not generalise to this extreme-density plate, whose colony appearance falls outside the training distribution.

Otsu 3-patch and Reg 384 produce no critical failures on Val II. Otsu improves substantially relative to Val I (0.890 versus 0.810). Its Val I critical failures arose from blob-merging on very dense plates, and the plates in Val II are dense enough that Otsu correctly estimates their counts above 200. Reg 384 misroutes one dense plate near the 200-colony boundary. The Val I ranking (YOLOX > Regression > Otsu) holds on Val II, although the gap between approaches narrows.

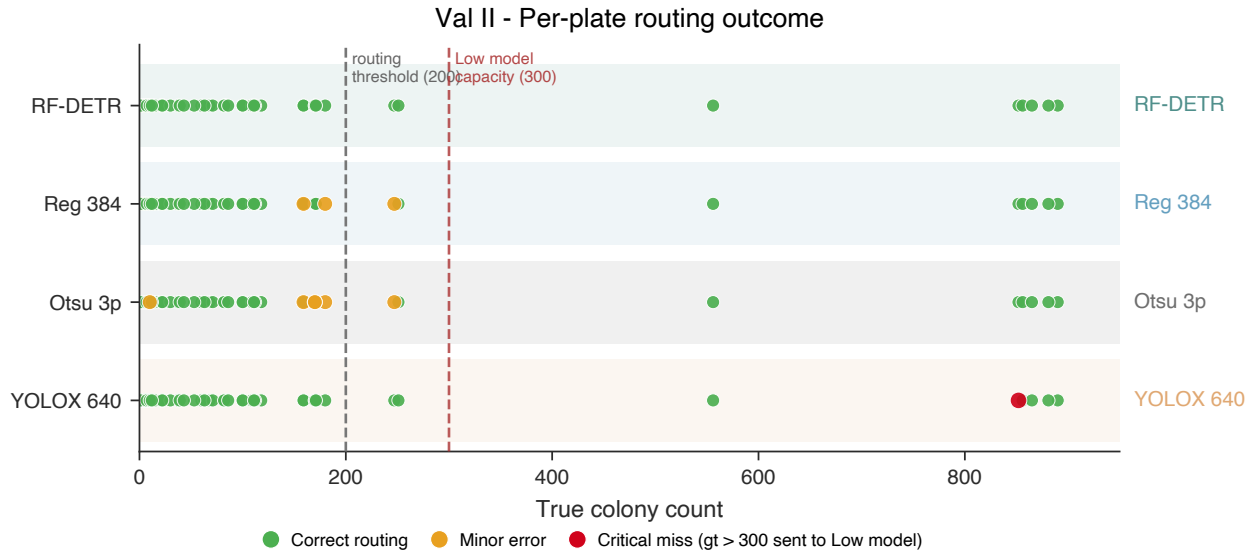


Figure 5.15: Per-plate routing outcome on Val II. Each row is one approach and each dot is one plate positioned by its true colony count. Green indicates correct routing, yellow a minor misroute (ground truth ≤ 300), and red a critical failure (ground truth > 300 sent to Full-image RF-DETR). The dashed grey line marks the 200-colony routing threshold and the dashed red line marks Full-image RF-DETR’s 300-colony detection ceiling.

Full-image RF-DETR achieves perfect routing on all 61 Val II plates. Even on plates with 850-880 colonies, it returns counts of 297-299, nearing its detection ceiling. This correctly exceeds

the 200-colony threshold. Figure 5.15 shows the routing outcomes per-plate in Val II for all four approaches.

5.5.4 Conclusion

Full-image RF-DETR was selected as the routing model due to its perfect routing accuracy across both validation sets.

Among the lightweight approaches, YOLOX 640 achieved the strongest balanced accuracy on Val II, but it exhibited a critical failure by predicting only 149 colonies for a plate containing 852 colonies. This was not a near-miss at the routing threshold, but a severe underestimation caused by confidence and NMS thresholds tuned on Val I failing to transfer to an extreme-density plate from a different domain. The result highlights a fundamental limitation of threshold-tuned detection-based routing: strong validation performance on one distribution does not guarantee robustness against out-of-distribution high-density plates. For a routing component whose primary requirement is avoiding critical failures, even a single unprotected plate class is unacceptable.

Full-image RF-DETR achieved perfect routing accuracy on Val II, but at a significantly higher inference cost. However, the extra time Full-image RF-DETR adds in practice depends only on how many dense plates are in the batch. For sparse plates, which represent the majority of submitted images in practice, the routing pass and the detection pass are the same operation and incur no additional cost. Only dense plates require a second pass through SAHI RF-DETR, and these plates require that second pass regardless of the routing method. Assuming a 13% dense-plate prevalence, identical to the distribution of the Val II set, the expected extra processing time is approximately 434 ms per plate on average. For most plates this introduces no latency penalty, and for the dense minority, routing correctness takes precedence over throughput. Furthermore, Full-image RF-DETR requires no additional model component, integrates directly into the existing API, and leaves the development focus on improving detection rather than maintaining a separate routing model.

5.6 Model Calibration, Post-processing, and Failure Analysis

Having established the Dual RF-DETR pipeline for handling both sparse and dense plates and decided on a routing model, the next step is to further refine the system for deployment. In particular, this involves selecting appropriate confidence thresholds, reducing visible prediction artefacts, and addressing the remaining failure modes observed during inference.

5.6.1 Method

In this section, we first determine optimal confidence thresholds for both models and assess whether a single global threshold is sufficient. We then introduce a lightweight post-processing step to remove systematic artefacts arising from tiled inference. Finally, we analyse failure cases on the diverse Val II set and evaluate whether targeted augmentations improve robustness.

Confidence threshold calibration

To convert model predictions into final detections, we determine an optimal confidence threshold for each model by sweeping threshold values and selecting the point that maximizes the F1 score on Val I. This is done separately for Full-image RF-DETR (0-200 colonies) and SAHI RF-DETR (200+ colonies).

To assess whether calibration varies spatially, predictions are grouped into six radial zones spanning from the plate center to the outer edge. Figure 5.16, A visualizes one of the metrics (mean confidence) for visual reference on how the radial zones are defined. For each zone, we compute the mean confidence, the locally optimal confidence threshold, and the resulting F1 score for the locally optimal confidence threshold and global optimal confidence threshold respectively. This allows us to evaluate whether region-specific thresholds provide any benefit over a single global threshold.

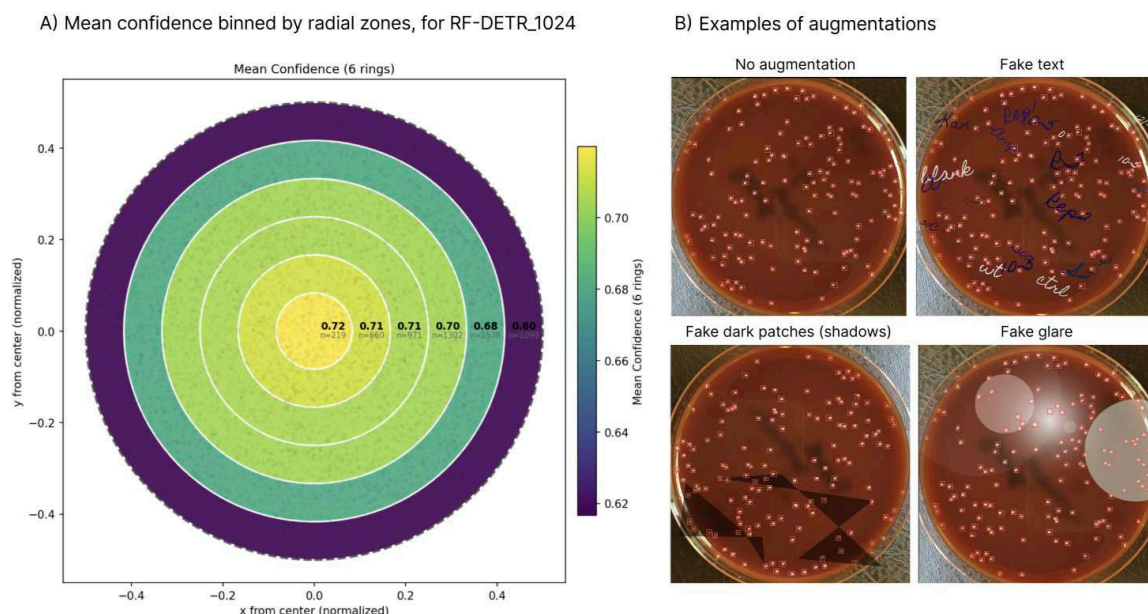


Figure 5.16: A) Mean confidence binned by radial zones for Full-image RF-DETR on Val I (0-200 colonies), B) Examples of augmentations.

Post-processing via aspect ratio filtering

To evaluate artefacts introduced by tiled inference, we examine whether predictions include elongated bounding boxes, which may arise when objects are only partially visible within a tile.

We quantify this effect by computing the aspect ratio of each predicted bounding box, defined as it's largest ratio between any two sides. The impact of filtering detections above a given threshold is then evaluated by sweeping a range of aspect ratio values on Val I and measuring the corresponding F1 score.

Failure analysis and targeted augmentations

To identify remaining failure modes, we evaluate both models on Val II, as this is the most diverse dataset. Performance is analysed across radial zones, with a focus on recall and precision to identify systematic spatial trends.

Based on qualitative inspection, common sources of error include glare, overlaid text, and darker sections (e.g. due to fingers or camera reflections), particularly near the plate edges. To address these issues, we augment the training data with synthetic versions of these artefacts, applying glare, text overlays, and dark patches (shadows) to the input images.

The augmented models are then evaluated against their original counterparts on Val II. In addition to aggregate metrics, we analyse performance across radial zones and perform a targeted evalu-

ation on manually identified text-obscured colonies to assess whether the augmentations improve robustness to specific occlusion types.

Implementation Details

Spatial calibration and failure analysis use six concentric radial zones from the plate centre to the outer rim. Ground-truth boxes and predictions are assigned to zones by the centre of their bounding boxes, and the same assignment is used when computing mean confidence, local confidence thresholds, recall, precision, and F1. Recall, precision, and F1 are computed using the same IoU 0.5 matching criterion as the preceding detection evaluations.

The targeted augmentation models use the same standard [RF-DETR](#) training augmentations described earlier, with three additional augmentations aimed at the observed failure modes. Synthetic handwritten annotations are added by drawing lab-like plate markings, such as dilution labels and sample identifiers at random positions, rotations, sizes, and ink colours. This text augmentation is applied with probability 0.1. Random sun flare and random shadow are implemented using the corresponding Albumentations transformations, each applied with probability 0.05.

5.6.2 Results

The results are presented in the same order as the method section: first we assess confidence-threshold calibration, then post-processing via aspect ratio filtering and finally we look at failure analysis and targeted augmentations.

Confidence threshold calibration

Table 5.15 summarizes the radial calibration analysis for both models, from the centre of the plate to the outer edge. Both models show a clear drop in mean confidence toward the rim, from 0.72 to 0.60 for Full-image [RF-DETR](#) and from 0.73 to 0.57 for SAHI [RF-DETR](#). However, the locally optimal confidence thresholds fluctuate rather than following a stable radial pattern. More importantly, replacing the local thresholds with one global threshold changes the best F1 score by at most 0.01 for either model. This indicates that the lower edge performance reflects a genuine detection limitation near the plate boundary, rather than a calibration problem that can be solved by region-specific thresholding.

Table 5.15: Confidence threshold calibration across radial zones on Val I. The six radial-zone icons are ordered from the plate centre (leftmost filled circle) to the outer rim (rightmost filled ring). F1 loss is the drop from using the selected global threshold instead of the locally optimal radial threshold. The selected global thresholds are 0.45 for Full-image [RF-DETR](#) and 0.48 for SAHI [RF-DETR](#).

Model	Metric						
Full-image RF-DETR	Optimal local conf.	0.54	0.41	0.43	0.51	0.50	0.43
	F1 loss from using global conf. instead of local	0.00	0.00	0.00	0.01	0.01	0.00
SAHI RF-DETR	Optimal local conf.	0.51	0.57	0.51	0.48	0.49	0.40
	F1 loss from using global conf. instead of local	0.01	0.00	0.00	0.00	0.00	0.01

Since radial threshold tuning offers no meaningful benefit, we use a single global threshold for each

model in all remaining evaluations: 0.45 for Full-image RF-DETR and 0.48 for SAHI RF-DETR.

Post-processing via aspect ratio filtering

Looking into the predictions of the model given the above found threshold we see elongated bounding boxes on some colonies on tile boundaries (Figure 5.17, A). This is an artifact of tile-based inference. When a colony falls at the edge of a tile, the model receives only a partial view of the object even with overlap between adjacent tiles, and can produce a prediction that stretches abnormally toward the tile edge rather than enclosing the full colony (Figure 5.17, A). Since colonies are approximately circular, any predicted box with a high aspect ratio is almost certainly an artifact of this kind rather than a true detection. This motivates a straightforward post-processing step: filtering predictions whose bounding box aspect ratio exceeds a fixed threshold, on the assumption that no valid colony detection should deviate substantially from a roughly square bounding box.

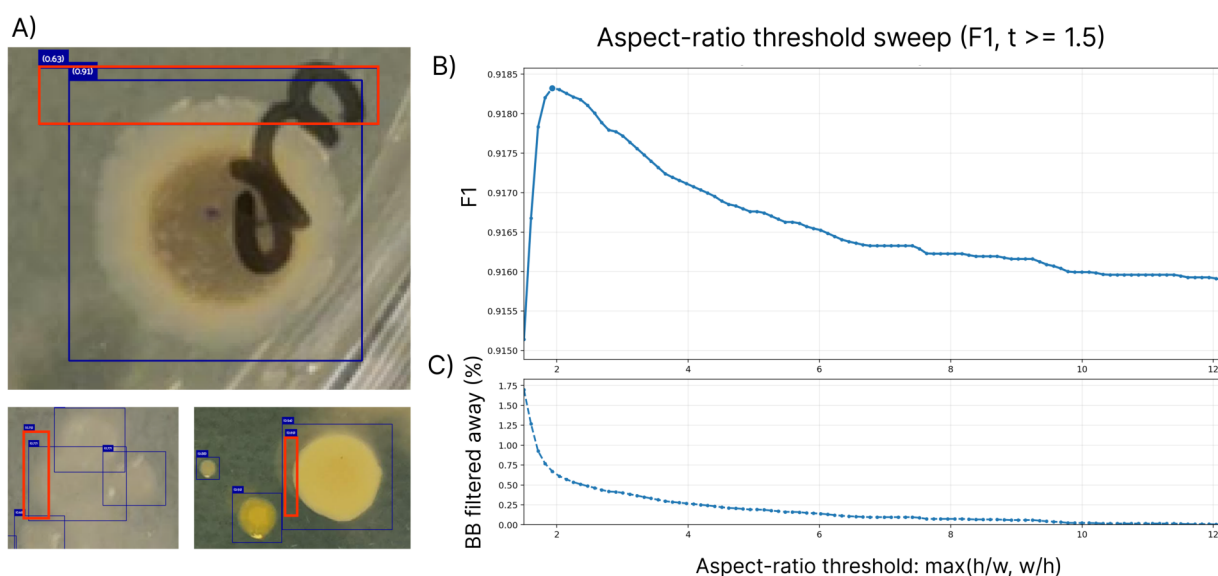


Figure 5.17: A) Examples of artefacts from SAHI B) Effect on F1 of filtering out detections with a max ratio above some threshold, C) Percentage bounding boxes (BB) filtered away when filtering out detections with a max ratio above some threshold.

Figure 5.17, B and C show the result of sweeping the aspect ratio threshold across the range 1.5 to 12 for all images in Val I (to get a larger sample size), filtering out all predicted bounding boxes whose aspect ratio exceeds the threshold. F1 peaks at a threshold of approximately 2 before declining steadily as the threshold relaxes and fewer detections are removed. The absolute gain is small, approximately 0.002 in F1, which is unsurprising given that only around 0.6% of predicted boxes are filtered at that threshold. For the Val II dataset for plates with over 200 colonies setting a threshold at 2.0 ratio, the F1 improves from 0.939 to 0.941 and the mAP@50 from 0.911 to 0.912 with this filter.

While the absolute F1 and mAP@50 gain is small, the value of this filter extends beyond the aggregate metric. Elongated bounding boxes spanning tile boundaries are visually conspicuous artefacts that undermine user trust, a microbiologist seeing a box that clearly fails to bound any real colony would reasonably question the reliability of the entire count. The threshold is therefore retained as a lightweight post-processing step, with the cutoff set to 2.0 based on the F1 sweep.

Failure analysis and targeted augmentations

Based on the optimal confidence threshold identified previously, we use Val II to investigate where and how Dual RF-DETR fails. Throughout the preceding sections, Val I served as the primary dataset for hyperparameter tuning and configuration selection, with Val II used primarily for final model comparisons. Val II is therefore not exposed to low-level tuning decisions and provides a more representative picture of model behaviour. Furthermore, as Val II was collected across different laboratory it provides a more realistic picture of the distribution the model is likely to encounter in practice.

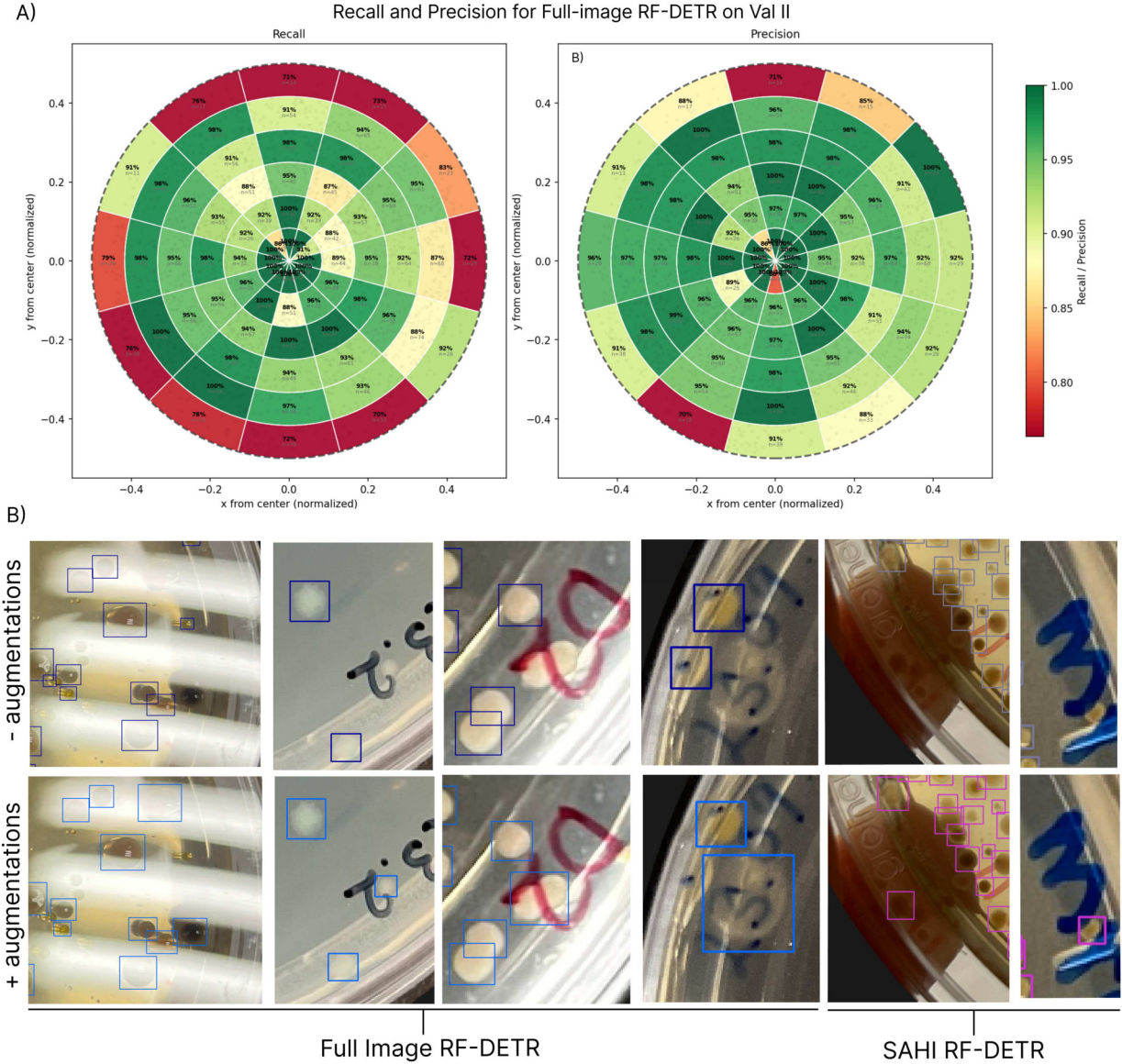


Figure 5.18: A) Recall and precision of different areas of the images for the Full image RF-DETR model. B) Examples of when the two models which are part of Dual RF-DETR fails. "- augmentations" shows predictions of the two models without training with extra targeted augmentations. "+ augmentations" shows the output of the models which are trained with the extra targeted augmentations, showing some colonies which are now detected.

We begin by examining Full-image RF-DETR, looking at recall and precision across radial zones of

the petri dish (see Figure 5.18). The results show that the model performs worse toward the edges of the agar plates, where recall drops the most, which means the model is missing a disproportionate number of colonies near the plate edges. Investigating the entire outer edge, the outer edge has a recall of 0.77 where the rest of the rings have between 0.98 to 0.95 recall. We see a similar trend for SAHI **RF-DETR**, where the outer edge has a recall of 0.71 and the rest of the plate ranges between 0.85 and 0.96.

The plate edges tend to be more visually complex than the central regions. Unlike the centre of the plate, the edges often have a non-uniform background, which alone makes detection harder. On top of this, colonies near the edges are more frequently obscured by external factors such as written text, glare from lighting, or physical objects like fingers changing the background color. As the model relies on visual features to detect colonies, any obstruction or noise in the image makes it more likely that a colony will be missed, which helps explain the drop in recall seen at the edges of the plates. Figure 5.18, B shows examples of colonies that were missed at the edges, highlighting how these factors at the edges affect detection.

Based on these findings, we train new models with additional data augmentations targeting the identified sources of occlusion. Specifically, we add synthetic glare, text, and shadows to the training images, simulating the conditions that were found to cause missed detections at the plate edges. Figure 5.16, B shows examples of how these augmentations appear in practice.

For Full-image **RF-DETR**, adding the augmentations results in a marginal improvement in overall performance, with mAP@50 increasing from 0.901 to 0.905 on images in the 0-200 colony range. Looking at recall and precision across radial zones, however, the augmented model performs almost identically to the original model (Table 5.16). The outer-edge recall remains unchanged at 0.77, indicating that the augmentations do not resolve the main edge-related failure mode.

Table 5.16: Change in recall and precision after targeted augmentations across radial zones on Val II. Values are percentage-point changes from the original model to the augmented model. The six radial-zone icons are ordered from the plate centre (leftmost filled circle) to the outer rim (rightmost filled ring).

Model	Metric						
Full-image RF-DETR	Recall Δ	-1	0	0	0	-1	0
	Precision Δ	0	+1	0	+1	0	+1
SAHI RF-DETR	Recall Δ	-13	-13	-13	-8	-7	-1
	Precision Δ	-5	-7	-7	-3	-3	-11

To investigate this further, we manually annotate colonies that are directly obscured by text, identifying 17 images containing 51 such colonies. Glare, shadows, and other overlapping objects are not quantified, as these are considerably harder to identify and label consistently. Of the 51 text-obscured colonies in the 0-200 colony range, the original model detects 38, while the augmented model detects 40. This means the augmentations only recover 2 additional colonies, suggesting that while the augmentations have some effect, they do not substantially improve the model’s ability to handle text occlusion.

For SAHI **RF-DETR**, adding the augmentations actually results in a drop in overall performance,

with mAP@50 decreasing from 0.912 to 0.833 in the 200–300+ colony range. This is further reflected in the radial zone breakdown (Table 5.16), where the augmented model performs consistently worse than the original across all zones in both recall and precision. The decline is strongest in the inner and middle zones for recall, while precision drops most at the outer edge. Applying the same text-occlusion analysis to SAHI RF-DETR, there are 70 text-obscured colonies in the 200-300+ colony range, of which the original model detects 42 and the augmented model detects 47. While this recovery of 5 additional colonies is slightly larger than the 2 seen for Full-image RF-DETR, it is not enough to offset the broader drop in performance observed across all radial zones.

5.6.3 Summary of Results

The experiments above indicate that both models are well calibrated, as a single global confidence threshold performs comparably to region-specific tuning (0.45 for Full-image RF-DETR and 0.48 for SAHI RF-DETR). This consistency suggests that confidence scores are reliable across spatial regions, and that the reduced performance observed near the plate edges is not a consequence of suboptimal thresholding but rather reflects a genuine limitation in detection under those conditions.

To address visible artefacts, a simple aspect ratio filter was introduced to remove elongated bounding boxes produced by tile boundary effects in SAHI RF-DETR. Although the quantitative improvement is modest, this filtering step removes clearly implausible detections and leads to more trustworthy outputs, which is important for practical use.

Attempts to mitigate edge-related failures through targeted data augmentations including simulated glare, overlaid text, and shadows show limited effect. The Full-image RF-DETR benefits only marginally, while the performance of SAHI RF-DETR decreases slightly, suggesting that these augmentations may introduce additional variability that outweighs their intended benefit.

Taken together, these results indicate that the dominant remaining errors are not easily corrected through thresholding, post-processing, or straightforward augmentation strategies. Instead, they appear to arise primarily from challenging imaging conditions at the plate periphery, such as reduced contrast, occlusions, and optical distortions. Consequently, further improvements are likely to depend more on improvements in image acquisition such as more uniform illumination or better control of edge artefacts than on additional model-side refinements.

5.7 Final Evaluation on Held-Out External Benchmark

This section evaluates our best performing model (Dual RF-DETR) on a held-out external benchmark using our full end-to-end object detection pipeline and compares it against 9 competing colony counting tools, including commercial tools and methods from the literature. The final object detection pipeline, including Dual RF-DETR is shown in Figure 5.19

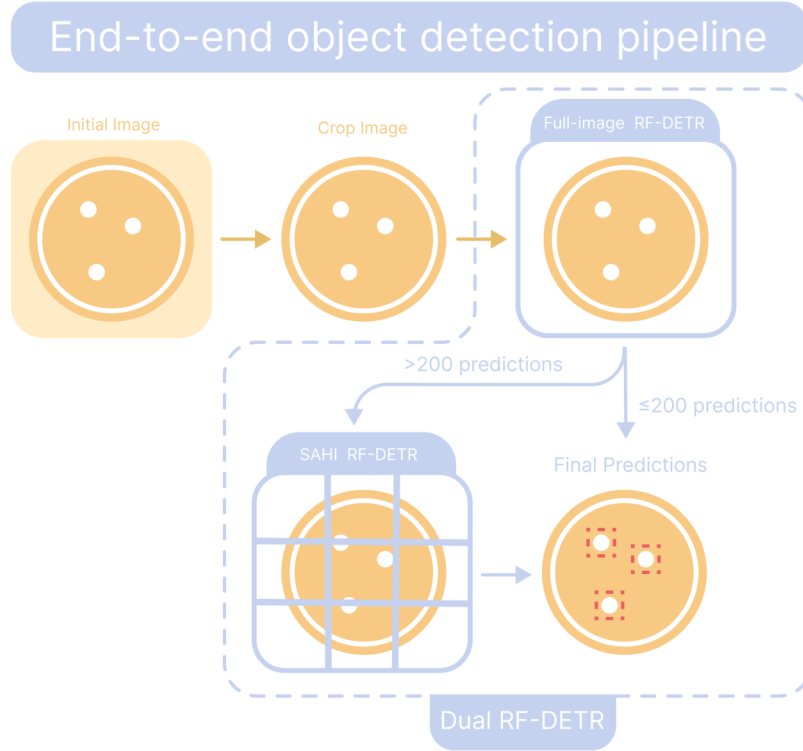


Figure 5.19: The final object detection pipeline. First the image is cropped, the cropped image is then run through Dual RF-DETR to produce the final object detection predictions. Dual RF-DETR first runs Full-image RF-DETR, if it predicts more than 200 object predictions, the image is rerun through SAHI RF-DETR.

5.7.1 Method

The held-out external benchmark dataset we use for this evaluation is the Moucka et al dataset described earlier, which contains 108 agar plates [59]. For each plate, the number of colonies is manually counted, and this manual count serves as the manual reference count for this evaluation.

We evaluate the proposed method against the competing tools shown in Table 5.17. Not all tools could be evaluated on all image subsets because of differences in availability, platform support, and input-resolution constraints.

Evaluation scope is reported with the corresponding result tables and figures.

Dual RF-DETR was evaluated on cropped plate images, because plate localisation is part of the pipeline developed in this thesis. In contrast, the external tools were evaluated on the full input images. To assess whether this difference affected the comparison, the strongest methods were additionally evaluated using both cropped and full-image inputs.

Dual RF-DETR was only evaluated on this benchmark after model development was complete. The held-out external benchmark was not inspected during training, model selection, confidence-threshold calibration, or hyperparameter tuning. The evaluation therefore provides an independent assessment of the final system against both manual reference counts and competing colony counting methods.

Table 5.17: Main tested methods in the external final evaluation. Methods were evaluated on the relevant phone-specific image sets where platform availability and input constraints allowed it. Reported approaches are listed only when publicly documented. Otherwise, the method is treated as a black-box comparator.

Method	Category	Scope	Reported approach
ColonyCounter	Proposed method	Full benchmark	Dual RF-DETR with density-aware tiled inference
Colonia	Mobile app	Full benchmark	YOLO11N
Colony Grounded SAM2	Available model	Full benchmark	Grounding DINO + SAM2
CFU.Ai	Mobile app	iPhone only	Undisclosed
Promega Colony Counter	Mobile app	iPhone only	Undisclosed
APD Colony Counter App Pro	Mobile app	iPhone only	Classical image processing
@BactLAB	Mobile app	Samsung only	Undisclosed
CFUCounter	Desktop application	iPhone + Samsung	Mask R-CNN
openCFU	Desktop application	Samsung subset (n=15)	Classical image processing
online-colony-counter.com	Web interface	Samsung subset (n=15)	Undisclosed

Evaluation metrics

The primary metric we use is practical counting accuracy, denoted $CA@10/\pm 1$. It builds on the $CA@10$ metric proposed by Pichler et al. [71], which reports the percentage of plates with a predicted colony count within $\pm 10\%$ of the manual reference count. A similar mixed-threshold logic is also used by Upfold et al. [72], who define agreement using a $\pm 10\%$ tolerance for plates above 30 CFU and an absolute ± 3 CFU tolerance for plates with 30 CFU or fewer. The rationale is that colony-counting models should be evaluated using task-specific criteria that reflect practical laboratory use, rather than detection metrics or average error alone. In this setting, the key requirement is not necessarily to detect every colony, but to produce a count close enough to the manual reference count to support the same downstream interpretation. This is particularly relevant because the manual reference counts are subject to observer variability, estimated at ± 4.9 colonies, meaning that small deviations from the reference count may fall within the uncertainty of manual counting itself.

We therefore adopt the logic of $CA@10$ but extend it with an absolute ± 1 colony tolerance for plates with 0-9 colonies. This ensures that a one-colony counting error is allowed for low-count plates, where the original $\pm 10\%$ criterion would require an exact match. For plates with more than 9 colonies, the original $\pm 10\%$ criterion is applied. The resulting metric is defined in Equation 5.1.

$$CA_{10/\pm 1} = \frac{1}{N} \sum_{i=1}^N \mathbb{1} \left\{ \begin{array}{ll} |\hat{y}_i - y_i| \leq 1, & 0 \leq y_i < 10, \\ \frac{|\hat{y}_i - y_i|}{y_i} \leq 0.10, & y_i \geq 10. \end{array} \right\} \quad (5.1)$$

where y_i is the manual reference count for image i , $\hat{y}_i$ is the predicted colony count for image i , N is the total number of evaluated images, and $\mathbb{1}\{\cdot\}$ is the indicator function, which equals 1 when the condition inside the braces is true and 0 otherwise. The metric is reported as a percentage by multiplying Equation 5.1 by 100.

MAE is also reported to characterize the average absolute counting error in colony units. Confidence intervals were estimated using non-parametric paired bootstrap resampling at the plate level with

10,000 resamples. To limit multiplicity, formal paired comparisons were restricted to the primary freestyle full-benchmark CA@10/ ± 1 analyses. Paired comparisons were estimated by bootstrap resampling the plate-level CA@10/ ± 1 differences between ColonyCounter and each comparator 10,000 times.

5.7.2 Results

The following results move from the main freestyle benchmark comparison to more detailed analyses of density, agar type, routing errors, cropping effects, and apparatus-based imaging.

Freestyle performance

Table 5.18 shows the overall performance on freestyle images, split by phone. Dual RF-DETR achieves the highest CA@10/ ± 1 on both iPhone and Samsung images. On iPhone freestyle images, Dual RF-DETR reaches 74.1% CA@10/ ± 1 , compared with 55.6% for Colonia and 46.3% for Colony Grounding SAM2. On Samsung freestyle images, Dual RF-DETR reaches 69.4% CA@10/ ± 1 , compared with 58.3% for Colony Grounding SAM2 and 49.1% for Colonia. For MAE, Dual RF-DETR achieves the lowest error on iPhone images with 9.5 colonies, while Colony Grounding SAM2 achieves the lowest error on Samsung images with 12.1 colonies. This difference between CA@10/ ± 1 and MAE indicates that practical accuracy and average absolute error capture related but not identical aspects of performance. CA@10/ ± 1 emphasises whether a count is close enough for practical use, whereas MAE is more sensitive to large errors on difficult plates.

Table 5.18: Overall freestyle performance split by phone. CA@10/ ± 1 values include 95% plate-level bootstrap intervals in grey brackets. Full CA@10/ ± 1 and MAE intervals are provided in Appendix A.9.1.

Smartphone	Method	CA@10/ ± 1 (%)	MAE
iPhone	Dual RF-DETR	74.1 [65, 82]	9.5
	Colonia	55.6 [46, 65]	21.6
	Colony Grounding SAM2	46.3 [37, 56]	14.0
	CFU.Ai	38.9 [30, 48]	36.2
	APD	18.5 [11, 26]	80.8
	Promega	13.0 [7, 19]	53.9
	CFUCounter	11.1 [6, 18]	46.6
Samsung	Dual RF-DETR	69.4 [60, 78]	17.7
	Colony Grounding SAM2	58.3 [49, 68]	12.1
	Colonia	49.1 [40, 58]	31.6
	@BactLAB	15.7 [9, 22]	67.9
	CFUCounter	6.5 [2, 11]	51.9

Paired plate-level bootstrap confidence intervals showed that the CA@10/ ± 1 difference was clearly above zero for Dual RF-DETR versus Colonia on both freestyle phone subsets and for Dual RF-DETR versus Colony Grounding SAM2 on iPhone freestyle images. The Samsung comparison with Colony Grounding SAM2 was also positive, but not clearly separated from zero. Full paired estimates are provided in Appendix A.9.2.

A smaller random 15 image Samsung freestyle subset was used to evaluate OpenCFU and Online-Colony-Counter.com, because full evaluation through these interfaces was not feasible. On this subset, ColonyCounter achieved the highest CA@10/ ± 1 at 53.3%, followed by Colony Grounding SAM2 at 46.7%. Online-Colony-Counter.com reached 20.0%, while OpenCFU reached 6.7%. The

full results are reported in Appendix A.9.7

Together, the freestyle results identify Dual [RF-DETR](#), Colonia, and Colony Grounding SAM2 as the strongest-performing methods overall. They are also among the most recently released tools, with Colonia and Colony Grounding SAM2 both released in March 2026. Notably, all three methods are deep learning-based: Colonia uses a YOLO-based architecture, while Colony Grounding SAM2 combines Grounding DINO for detection with SAM2 for segmentation. Based on these results, the further analysis focuses on these tools.

Performance across colony density

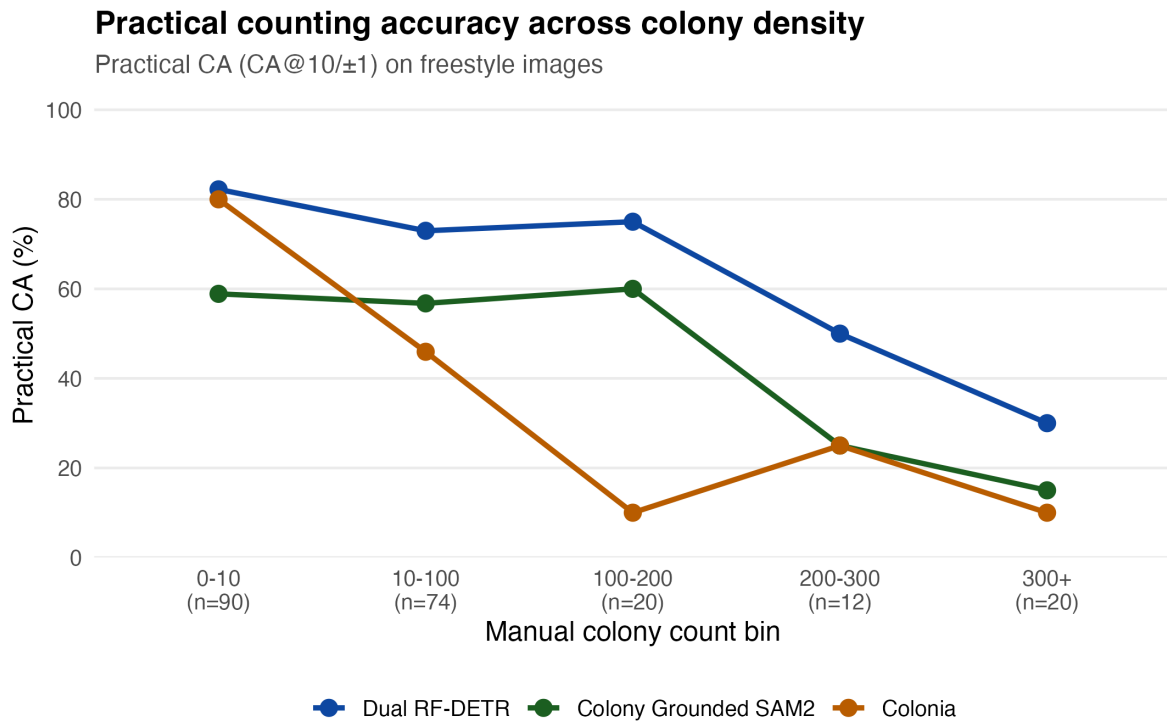


Figure 5.20: Practical counting accuracy (CA@10/±1) across manual colony-count bins for the three strongest freestyle methods. The number of plates in each bin is shown below the x-axis. Exact estimates and bootstrap intervals are provided in Appendix A.9.3.

Figure 5.20 shows practical counting accuracy stratified by manual colony-count range for the three strongest freestyle methods. Dual [RF-DETR](#) achieves the highest CA@10/±1 in every density bin, but performance decreases as colony density increases. This confirms that colony density remains one of the main challenges for automated counting.

In the 0-10 colony range, Dual [RF-DETR](#) and Colonia perform similarly, reaching 82.2% and 80.0% CA@10/±1, respectively. The advantage of Dual [RF-DETR](#) becomes clearer in the 10-100 and 100-200 ranges, where it reaches 73.0% and 75.0% CA@10/±1. In the 200-300 range, performance decreases to 50.0%, but remains above both Colony Grounded SAM2 and Colonia.

Above 300 colonies, all methods decline substantially. Dual [RF-DETR](#) remains the strongest method by practical accuracy, reaching 30.0% CA@10/±1 compared with 15.0% for Colony Grounded SAM2 and 10.0% for Colonia. However, this level of performance is still insufficient for fully

automated counting, supporting the need for manual verification on very dense plates.

Overall, the density-stratified results show that Dual RF-DETR provides the most reliable practical counts across colony densities, especially in the 10-200 range. However, very dense plates remain a clear failure regime even after Dual RF-DETR’s dense-plate handling strategy.

Performance across agar type

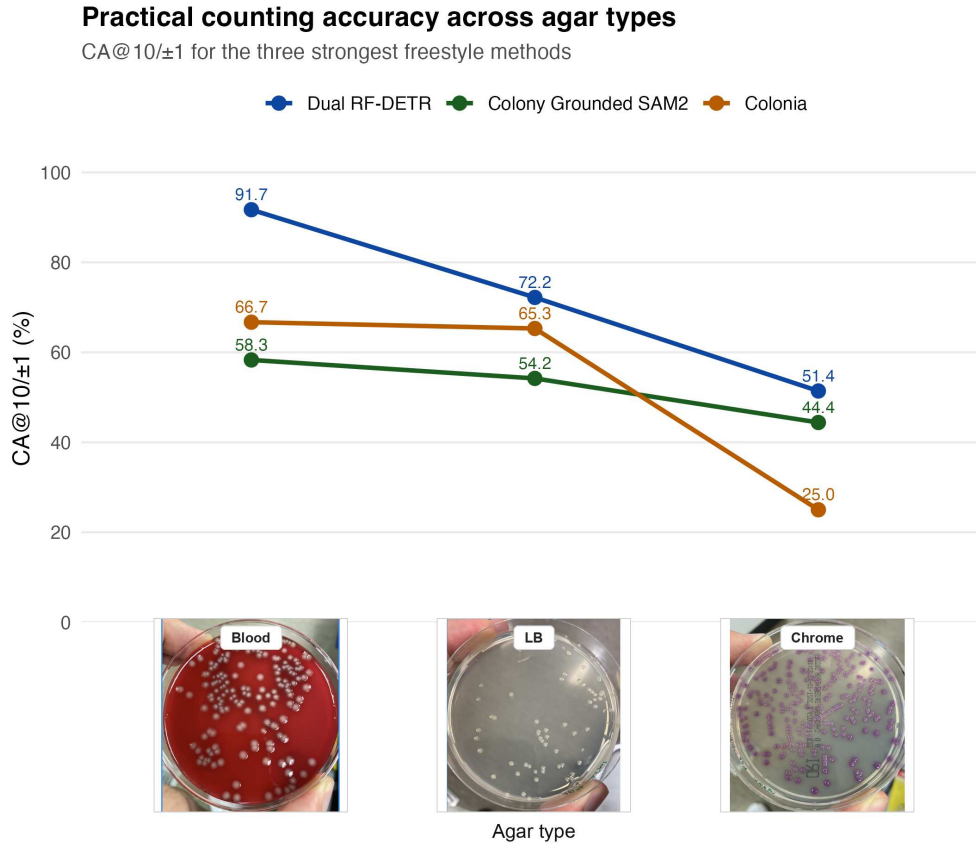


Figure 5.21: Practical counting accuracy (CA@10/±1) across agar types for the three strongest freestyle methods, with representative agar examples shown below the x-axis. Dual RF-DETR remains strongest across all agar types, but all methods decline on Chrome agar, indicating agar-specific domain shift. Exact estimates and bootstrap intervals are provided in Appendix A.9.4.

Figure 5.21 shows practical counting accuracy stratified by agar type for the three strongest freestyle methods, with representative plate examples included below the plotted accuracy values. Dual RF-DETR achieves the highest CA@10/±1 on all three agar types, with performance decreasing from Blood agar (91.7%) to LB agar (72.2%) and Chrome agar (51.4%). All three methods decline on Chrome agar, confirming that agar type is an important source of domain shift.

Overall, Chrome agar represents the clearest agar-specific failure mode by practical counting accuracy, likely due to its distinct background colour and altered colony-agar contrast. The result supports retaining manual verification for Chrome plates, especially when colony density is high.

Routing errors on the held-out benchmark

Because Dual RF-DETR uses separate inference paths for sparse and dense plates, we also assessed routing errors on the held-out benchmark. Images with manual counts above 200 colonies should be

routed to SAHI RF-DETR. Sending them to Full-image RF-DETR is moderate for counts of 200–300 and critical above 300, where the 300-query ceiling can truncate the count.

Table 5.19: Routing errors on the held-out external benchmark. Moderate errors are plates with 200–300 colonies routed to Full-image RF-DETR, while critical errors are plates with more than 300 colonies routed to Full-image RF-DETR.

Plate type	n	Should use SAHI	Routing errors	Moderate	Critical
Overall	432	64	12	8	4
Blood	144	20	0	0	0
Chrome	144	16	7	4	3
LB	144	28	5	4	1

We present the results of the routing errors across the 432 benchmarks images in Table 5.19. Only four errors are critical. The eight moderate errors are less severe because Full-image RF-DETR can still produce a meaningful count below its query ceiling. Critical failures are concentrated on Chrome agar, with none on Blood agar, suggesting that the same agar-specific domain shift observed in the counting results also affects the routing estimate.

Effect of cropping

In the main comparison, Dual RF-DETR is evaluated on cropped plate images because automatic plate localisation is part of the full pipeline developed in this thesis, whereas the external tools are evaluated on the original full images. To assess whether this affects the comparison, the three strongest methods were evaluated with both cropped and non-cropped inputs. Automatic plate localisation failed on one Samsung freestyle image. This failed crop was manually corrected before colony counting, consistent with the intended human-in-the-loop workflow.

Cropping improves performance for all three methods, but the effect is largest for Dual RF-DETR: CA@10/±1 increases from 55.1% to 71.8%, and MAE decreases from 17.5 to 13.6 colonies. Colony Grounded SAM2 also benefits substantially, increasing from 52.3% to 63.4% CA@10/±1, while Colonia shows a smaller improvement from 52.3% to 56.0%. These results show that removing irrelevant background, plate edges, and surrounding artefacts improves counting performance in general, not only for Dual RF-DETR. However, even when the competing tools are given the same cropped input, their performance does not reach that of Dual RF-DETR. Colony Grounded SAM2 improves to 63.4% CA@10/±1 and Colonia to 56.0%, but both remain below Dual RF-DETR at 71.8%. This shows that cropping alone does not explain Dual RF-DETR’s advantage, although it remains an important contributor to the final pipeline performance.

Apparatus performance

Table 5.21 shows the performance on apparatus images, split by phone model. These images were captured under more controlled illumination and positioning than the freestyle images, and therefore provide a useful estimate of how much counting performance improves when image acquisition is standardised.

On iPhone apparatus images, Dual RF-DETR achieves the strongest overall performance. It reaches 91.7% CA@10/±1, with an MAE of 4.7 colonies. This is a substantial improvement over the strongest competing methods: Colonia reaches 64.8% CA@10/±1, while Colony Grounded SAM2

Table 5.20: Effect of cropped and non-cropped inputs for the three strongest freestyle methods. CA@10/ ± 1 values include 95% plate-level bootstrap intervals in grey brackets. Full CA and MAE intervals are provided in Appendix A.9.5.

Method	Cropping	CA@10/ ± 1 (%)	MAE
Dual RF-DETR	cropped	71.8 [64, 79]	13.6
	not-cropped	55.1 [48, 62]	17.5
Colonia	cropped	56.0 [47, 65]	23.4
	not-cropped	52.3 [44, 61]	26.6
Colony Grounded SAM2	cropped	63.4 [56, 70]	10.8
	not-cropped	52.3 [45, 59]	13.0

Table 5.21: Apparatus performance split by phone. CA@10/ ± 1 values include 95% plate-level bootstrap intervals in grey brackets. Full CA@10/ ± 1 and MAE intervals are provided in Appendix A.9.6.

Phone	Method	CA@10/ ± 1 (%)	MAE
iPhone	Dual RF-DETR	91.7 [86, 96]	4.7
	Colonia	64.8 [56, 74]	19.7
	Colony Grounded SAM2	63.0 [54, 72]	17.9
Samsung	Colony Grounded SAM2	78.7 [70, 86]	8.7
	Dual RF-DETR	76.9 [69, 84]	16.2
	Colonia	61.1 [52, 70]	31.3

reaches 63.0%. Dual RF-DETR therefore improves practical counting accuracy by approximately 27 percentage points compared with the closest competitor on iPhone apparatus images.

The Samsung apparatus results are more mixed. Colony Grounded SAM2 achieves the highest practical accuracy, with 78.7% CA@10/ ± 1 and an MAE of 8.7 colonies. Dual RF-DETR performs close behind, reaching 76.9% CA@10/ ± 1 , with an MAE of 16.2 colonies. Although the difference in practical accuracy is small, Colony Grounded SAM2 also has lower average error on Samsung apparatus images, suggesting that it handles this acquisition setting slightly better than Dual RF-DETR. Colonia again performs below the two strongest methods.

Overall, the apparatus results show that improved image acquisition substantially benefits the strongest model-based methods, especially on iPhone images. Dual RF-DETR is the best-performing method on iPhone apparatus images and remains competitive on Samsung apparatus images, where it is only slightly behind Colony Grounded SAM2. At the same time, the results show that standardised imaging does not eliminate all method-dependent differences. The quality of the counting model remains important, and performance still varies by phone model and image characteristics.

Conclusion

Overall, the external benchmark shows that Dual RF-DETR is the strongest method by CA@10/ ± 1 on the full freestyle benchmark. It achieves the highest CA@10/ ± 1 on freestyle images across both phones, all agar types, and all density ranges. Cropping improves all methods, but Dual RF-DETR remains best even when the competing tools are also given cropped inputs, showing that its advantage is not explained by cropping alone. Controlled apparatus imaging further improves performance, especially on iPhone images, where Dual RF-DETR reaches 91.7% CA@10/ ± 1 . How-

ever, on Samsung apparatus images, Colony Grounded SAM2 performs slightly better than Dual RF-DETR, showing that Dual RF-DETR is not universally dominant across all acquisition settings. Performance also still drops on high-density plates and Chrome agar, and critical dense-to-sparse routing errors remain on Chrome and LB plates. These findings support the need for manual verification in difficult cases.

5.8 Conclusion

This iteration addressed the detection-model limitations carried forward from Iteration 0. The four iteration requirements are assessed below.

R2.1: Colony Density Robustness

• Partially Met

The detection model must maintain reliable detection performance across the full range of colony densities encountered in laboratory practice, including high-density plates where colonies are closely packed or partially overlapping.

The dual RF-DETR model (Full-image RF-DETR for sparse plates and SAHI-based RF-DETR for dense plates) improved robustness across density regimes compared earlier models based on the Val I and Val II datasets. On the held-out dataset the system achieved strong performance in low- and mid-density ranges and remained the best-performing approach in higher-density bins. However, accuracy still degraded on high density plates. This requirement is therefore considered partially met.

R2.2: Lighting robustness and annotation occlusion robustness

• Partially Met

The detection model must generalise across the range of lighting conditions encountered in real laboratory environments, including glare, reflections, and shadows. It should also be able to detect colonies occluded by text marker on the plate lid.

The final model was trained and evaluated under a broader range of imaging conditions than the Iteration 0 baseline. Targeted augmentations were also investigated to address the main failure modes identified during error analysis. These augmentations simulated glare, overlaid text, and shadow-like dark patches. However, their impact was limited and inconsistent. They produced only a marginal improvement in mAP@50 for the full-image model, and reduced overall performance for the SAHI model. Although both augmented models detected a small number of additional text-obscured colonies.

Confidence threshold calibration was also investigated. The results indicated that the reduced edge performance was not caused by suboptimal thresholding, as global thresholds performed comparably to region-specific thresholds. Overall, the requirement is therefore considered only partially met.

R2.3: Agar type and morphological robustness

• Partially Met

The detection model must perform reliably across agar types and across colonies that vary in size, shape, colour, and opacity, including colonies located at the rim of the plate.

The model performs well on Blood, and LB agar in the external benchmark. However, Chrome agar where colonies have different morphology remains a persistent failure mode. This requirement is therefore considered partially met.

R2.4: Competitive performance against accessible colony-counting tools • Satisfied

The final pipeline must be evaluated against relevant accessible colony-counting tools and perform competitively under realistic benchmark conditions.

The Dual [RF-DETR](#) pipeline was evaluated against nine competing tools on the held-out external benchmark. It achieved the highest CA@10/ ± 1 on the freestyle dataset and remained consistently competitive across phone types, imaging conditions, agar types, and different colony densities. For controlled-apparatus images, the pipeline outperformed the strongest competing tool (Colony Grounded SAM2) on iPhone images, but performed slightly worse on Samsung images. Overall, the requirement is considered satisfied.

Iteration 2 produced a stronger object-detection pipeline for the human-in-the-loop workflow. It performed competitively against accessible alternatives, and improved robustness especially for high-density plates, but it did not remove the need for user verification. Further improvements are likely to require both model-side refinements and more controlled image acquisition.

6 Imaging apparatus prototype

The initial think-aloud tests conducted at the start of the project revealed a clear need for a standardized imaging setup. This need was reinforced by our own data collection, where freestyle imaging often introduced reflections, shadows, and other acquisition artefacts. Attempts to address these issues through model-side improvements alone were only partially successful, suggesting that more consistent image acquisition was needed. This was further supported by the held-out test set, where the model performed better on apparatus images than on freestyle images.

We therefore developed a low-cost physical imaging prototype to make image acquisition more consistent. The goal of this chapter is to describe the final prototype and explain the main design choices. Its quantitative impact on counting performance and workflow speed is assessed later in the full-system evaluation. Together, the prototype described in this chapter and the full-system evaluation address Q3: whether a purpose-built, low-cost imaging prototype can reduce imaging variability and improve the reliability of the automated colony-counting workflow.

Disclaimer, we are not design, or photonics experts, and the imaging box should be understood as a secondary contribution of this thesis. The prototype was developed through practical, trial-and-error iteration rather than formal design methods. The iterative development process is documented in Appendix A.11.

6.1 The Prototype

The prototype is shown in Figure 6.1. The system consists of seven primary components: a lid containing a centred camera opening,

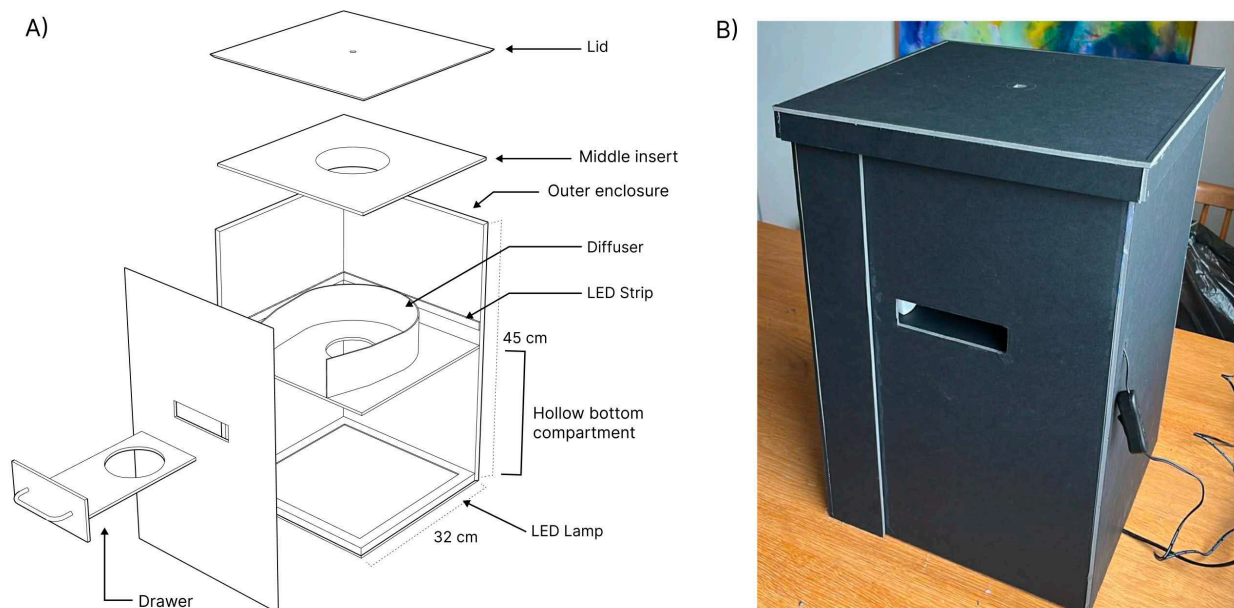


Figure 6.1: Final apparatus prototype shown as both A) an exploded view B) an assembled system.

a light-blocking middle insert, an outer enclosure, a ring of LED strips with a diffuser, a hollow bottom compartment, a bottom LED lamp, and a drawer for inserting the Petri dish.

Materials and cost: The prototype was constructed primarily from black foamboard, which provides a low-reflectance interior surface while remaining sufficiently rigid for repeated use. The material is also inexpensive, lightweight, and easy to cut and fold, making it suitable for rapid prototyping while keeping construction costs low. The final apparatus cost approximately 600 DKK, including the LED panel, LED strip, and diffuser.

Camera and plate positioning: The lid contains a small opening for the smartphone camera, while the Petri dish is inserted through a dedicated drawer. Together, these features constrain the relative position of the camera and sample during image capture. This reduces variation in viewing angle, ensures that the plate is correctly positioned with respect to the illumination, and keeps the plate far enough away from the smartphone camera to allow it to reliably focus on the Petri dish. Because the opening in the lid is small, only a limited part of the phone is exposed above the dish, reducing the risk of the phone being reflected in the Petri dish.

Controlled illumination: The primary illumination source is the side illumination, which consists of a ring of LED strips positioned around the Petri dish. The light passes through a diffuser before reaching the sample, spreading the illumination more evenly and softening the directional light from the LEDs. This provides shallow-angle top illumination that highlights colony morphology, texture, and surface structure while reducing harsh direct reflections from the Petri dish.

A light-blocking middle insert surrounds the Petri dish and separates the illuminated sample area from the rest of the enclosure. The insert prevents direct light from the LED strips from entering the camera and confines the illumination more closely to the sample area.

In addition to the primary side illumination, the apparatus includes a bottom LED panel that can be enabled as a backlight. This secondary mode was included as an experimental configuration to test whether backlighting can improve colony counting by increasing colony–agar contrast.

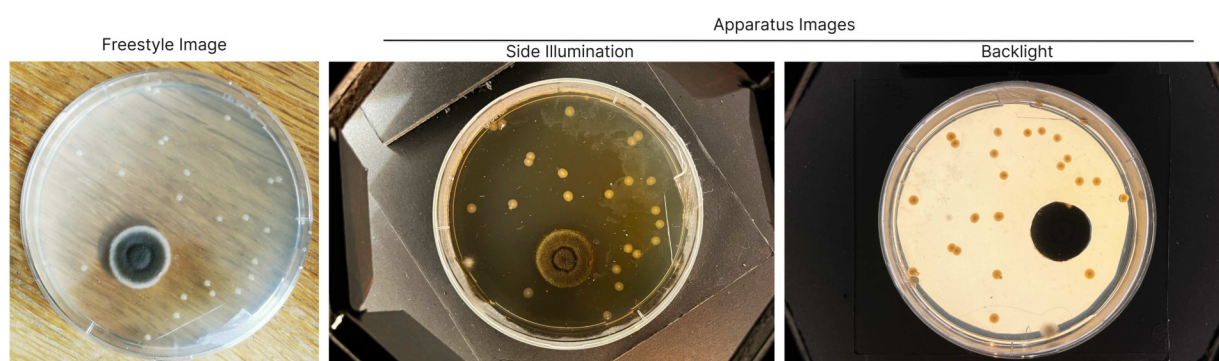


Figure 6.2: Comparison of illumination modes used in the apparatus with a freestyle image.

Figure 6.2 compares freestyle imaging with the two illumination modes supported by the apparatus. The freestyle image shows typical acquisition artefacts, including glare and reflections from the Petri dish surface, which are not present in the apparatus images. When comparing the apparatus

images, the primary side-illumination mode provides clearer information about colony morphology, texture, and surface structure, making features such as filamentous growth and colony texture more visible. In contrast, the backlight increases the visual separation between colonies and the surrounding agar, making colony boundaries easier to identify.

Background control: An important design feature is the hollow compartment located beneath the Petri dish. Rather than allowing transmitted light to illuminate a physical surface directly beneath the sample, the hollow compartment increases the distance to the nearest reflective surface. This reduces the amount of light reflected back toward the camera, creating a darker background beneath the plate and improving contrast between colonies and their surroundings.

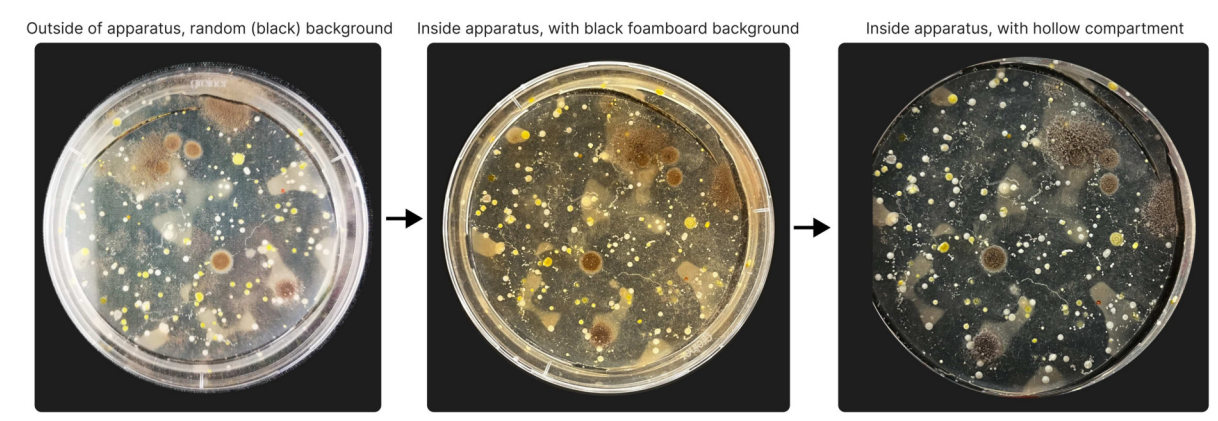


Figure 6.3: Comparison of imaging, we compare an image taken free-style with a random black background (chosen as the closest available alternative without using the apparatus), an image taken inside the apparatus with a black foamboard background and an image taken inside the apparatus with the hollow bottom compartment.

The effect is shown in Figure 6.3. The comparison moves from freestyle imaging with black textile in the background, chosen as the closest available non-apparatus alternative, to apparatus imaging with a black foamboard background, and finally to apparatus imaging with the hollow bottom compartment. The hollow compartment produces the darkest field beneath the plate and makes fine colony structures and textures easier to distinguish.

6.2 Conclusion

The developed prototype provides a low-cost, controlled imaging environment for microbial colony analysis. The combination of diffused illumination, fixed camera and plate positioning, and a hollow bottom compartment reduces visible reflections, limits illumination artefacts, and improves colony-background contrast. Furthermore, the dual illumination system enables both morphology-focused side illumination and high-contrast backlight imaging. The resulting prototype is evaluated quantitatively in the following full-system evaluation.

7 Full System Evaluation

This section evaluates the final ColonyCounter workflow on a controlled set of agar plates collected for this purpose. The goal is to assess the full system in terms of both counting performance and the practical time required to use it, and to compare it with manual counting and the Colonia app.

7.1 Method

The final system evaluation used 12 agar plates with marine samples which were photographed with and without the built apparatus in a DTU Microbiology Laboratory at The Center for Microbial Secondary Metabolites, DTU Bioengineering. (Figure 7.1). As in earlier sections the images taken without the apparatus are referred to as freehand photos while the images taken with the apparatus are referred to as apparatus photos.



Figure 7.1: Overview of the manual counting and image-acquisition conditions used in the final system evaluation. The figure shows manual counting, freehand image acquisition, and apparatus image acquisition, including both the capture setup and the resulting plate image.

The 12 agar plates were derived from 4 marine sample dilution series at 10^4 , 10^5 , and 10^6 , reflecting how dilution series are typically prepared for CFU estimation. As a result, 4 plates were low density (2-4 colonies), 4 medium density (18-30 colonies), and 4 high density (190-250 colonies) (Figure 7.2). The low colony counts both in the low and medium density tiers limit how much these groups can show about counting performance, but they are also a realistic feature of how plates are made for CFU estimation, rather than an artefact of the test design. We (Anton, Frederik and Lasse) each captured one image with and without the apparatus using our phones (iPhone SE 2020, iPhone

13 and Huawei P30 Pro respectively). We then used ColonyCounter to count the plates, recording time both before and after manual correction.

Out of the competing tools evaluated in the external test set of the object detection section, only Colonia was included in this test. This was because Colonia was identified as the strongest competitor with a dedicated user interface (Colony Grounded SAM2 is only available as a CLI model on Github). The entire Colonia app was tested by each of the 3 individuals on freehand images taken with the Colonia app, here the time was also recorded. The Colonia app’s model was also tested on the exact same freehand and apparatus images used by ColonyCounter.

Each of the 3 annotators also manually counted each of the 12 plates and the time spent was recorded. The mean of the 3 annotators counts for each plate is used as the manual consensus for which all counting results are compared. The primary metric used is $CA@10/\pm 1$, and MAE is used as a secondary metric when relevant.

In these tests, the apparatus was used with its standard side-light configuration. To further investigate whether the lighting setup affected image quality and counting performance, the apparatus was also tested using its backlight configuration. For this purpose, an additional set of images was captured for each plate using this configuration, however, unlike the main image dataset, these images were only collected by Frederik, using an iPhone 13.



Figure 7.2: Representative apparatus images from the three colony-density tiers used in the final system evaluation. The full test set contained four plates per tier.

7.2 Results

This section reports counter agreement, primary counting performance, and workflow timing.

7.2.1 Manual reference count agreement

Since the manual consensus is used as the reference for all counting comparisons, its reliability must be established first. Annotator agreement is therefore assessed across the three colony-density tiers.

Manual agreement was high for plates with low and moderate colony densities, indicating that the reference counts were consistent when the colonies were relatively easy to distinguish. For all low-count plates (2–4 colonies), the annotators reported identical counts, showing complete agreement

in this group. A similarly high level of agreement was observed for the middle-density plates (20–30 colonies), where the largest difference between annotators was only one colony (Figure 7.3).

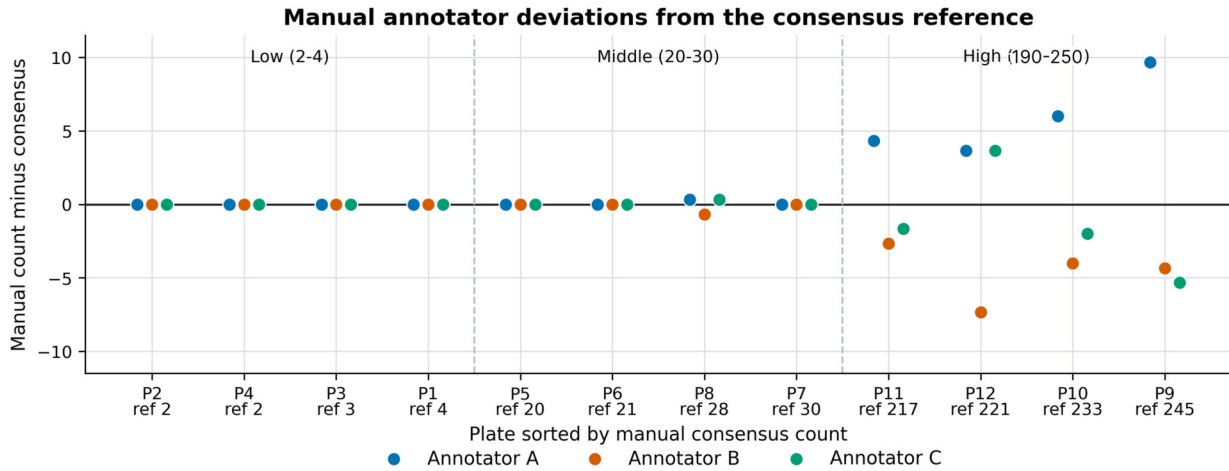


Figure 7.3: Manual annotator deviation from the consensus reference. Plates are sorted by consensus count and split into densities.

In contrast, agreement decreased for the high-density plates (190–250 colonies). For these plates, the colonies were more numerous and therefore more difficult to count consistently, which led to greater variation between annotators. On average, the highest and lowest annotator counts differed by 10.8 colonies, with the largest observed difference for a single plate being 15.0 colonies (Figure 7.3). This indicates that manual reference counts become less certain as colony density increases. However, this uncertainty has limited impact on the $CA@10/\pm 1$ metric, because of the 10% tolerance for these high-count plates.

7.2.2 Primary counting performance by colony density

This subsection compares ColonyCounter’s detection performance against Colonia, evaluated on the same freehand and apparatus images as well as on Colonia’s own app workflow.

Figure 7.4 compares ColonyCounter (detection model only, no manual correction) with the Colonia model evaluated on the same images taken with freehand and apparatus and with the images taken with the entire Colonia app workflow. The figure shows overall $CA@10/\pm 1$ and which colony-density groups contributed the correct counts.

Overall, ColonyCounter achieved the highest $CA@10/\pm 1$ in both acquisition conditions, reaching 97.2% on freehand images and 91.7% on apparatus images. When Colonia was evaluated on the same input images, performance was lower, with $CA@10/\pm 1$ scores of 66.7% on freehand images and 63.9% on apparatus images. The lowest $CA@10/\pm 1$ was observed when Colonia was evaluated using images acquired through its own app, where it reached 44.4%.

All methods tended to undercount colonies on average, as shown by the negative mean signed errors. ColonyCounter had the smallest undercount, with mean signed errors of -5.6 colonies on freehand images and -6.2 colonies on apparatus images. For the Colonia input-matched evaluations, the mean signed errors were larger: -13.3 colonies on freehand images and -8.5 colonies on apparatus images. When using Colonia’s own app-acquired images, the mean signed error was -13.7 colonies.

Primary CA@10/±1 by method and colony density

Each bar totals all evaluated counts; coloured segments are within CA@10/±1 and grey is outside CA@10/±1

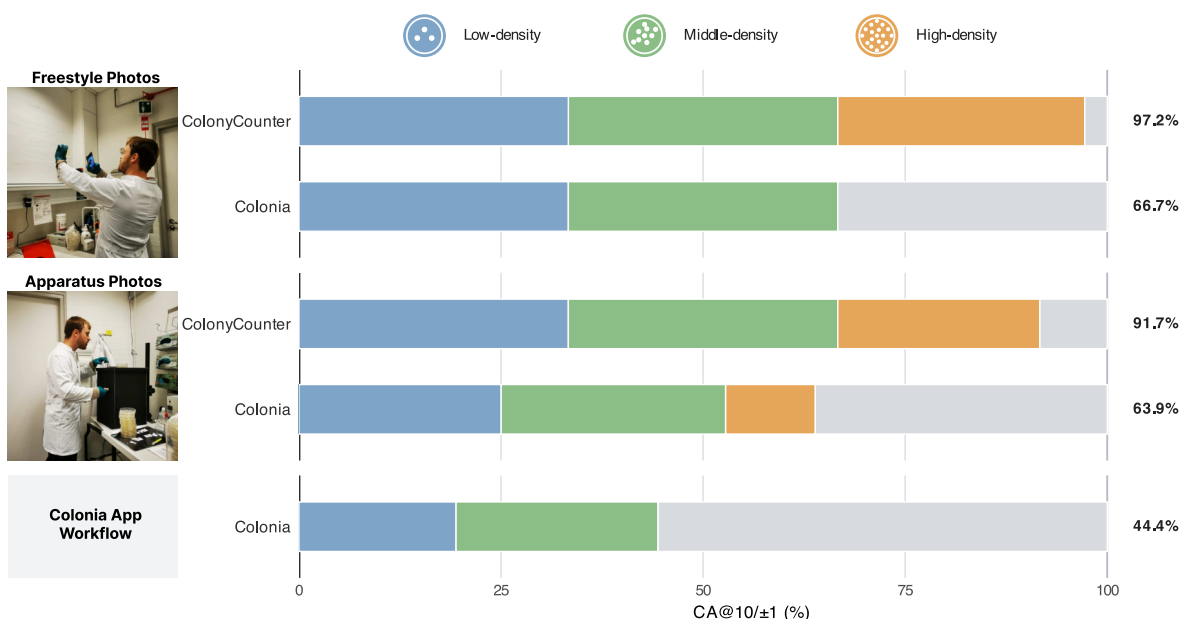


Figure 7.4: Primary CA@10/±1 by method and colony-density group. Each bar represents all evaluated counts for the method. Coloured segments show counts within CA@10/±1 from low, middle, and high-density plates, while grey segments show counts outside CA@10/±1. The Free-hand and Apparatus Colonia rows use Colonia’s model on the same input images. The Colonia app workflow row is the app-based workflow baseline. Exact values are provided in Appendix A.12.2 and Appendix A.12.3.

The clearest separation between methods was observed for the high-density plates. In these images, ColonyCounter maintained strong performance on freehand captures and showed only a modest reduction in accuracy on apparatus-based images, whereas Colonia exhibited substantially lower performance across the same high-density images (Figure 7.4).

7.2.3 Effect of backlighting

Backlighting improved counting performance on the subset where it was tested. Because backlight images were only collected by Frederik using an iPhone 13, this comparison uses the matching Frederik-only standard-apparatus images as the baseline. The values in this subsection therefore differ from the overall apparatus results reported elsewhere, which include all three image takers. For ColonyCounter, CA@10/±1 increased from 83.3% to 91.7%, while MAE decreased from 8.3 to 6.2. Colonia, evaluated on the same images, showed a larger improvement: CA@10/±1 increased from 50.0% to 91.7%, and MAE decreased from 11.3 to 7.5. (Table 7.1). This suggests that backlighting can improve counting performance by increasing colony-agar contrast, although the result should be interpreted cautiously because it was tested only on a single-image-taker subset.

Table 7.1: Effect of backlighting on apparatus images for the Frederik-only paired subset. CA@10/ ± 1 and MAE are shown as standard apparatus $\rightarrow$ backlight apparatus. Δ CA@10/ ± 1 is backlight minus standard apparatus. Green cells indicate an improvement in the target metric.

Method	CA@10/ ± 1 before $\rightarrow$ after	Δ CA@10/ ± 1	MAE before $\rightarrow$ after	Δ MAE
ColonyCounter	83.3 $\rightarrow$ 91.7	+8.3	8.3 $\rightarrow$ 6.2	-2.1
Colonia - input matched	50.0 $\rightarrow$ 91.7	+41.7	11.3 $\rightarrow$ 7.5	-3.8

7.2.4 Effect of manual correction

Manual correction brought both ColonyCounter workflows to complete threshold agreement with the manual consensus. For apparatus images, CA@10/ ± 1 increased from 91.7% before correction to 100.0% after correction, while MAE decreased from 6.8 to 2.1 colonies. (Table 7.2)

For freehand images, CA@10/ ± 1 increased from 97.2% before correction to 100.0% after correction, while MAE decreased from 5.7 to 3.4 colonies. Manual correction improved both threshold-based agreement and mean absolute error for both workflows, with apparatus images benefiting the most from correction and ending up with the lower MAE.

Table 7.2: Manual correction compared with the matching ColonyCounter prediction before correction. CA@10/ ± 1 and MAE are shown as uncorrected model prediction $\rightarrow$ corrected count. Backlight uses one image taker, the other rows include all image takers.

Acquisition	Method	CA@10/ ± 1 before $\rightarrow$ after	Δ CA@10/ ± 1	MAE before $\rightarrow$ after	Δ MAE
Freehand	ColonyCounter	97.2 $\rightarrow$ 100.0	+2.8	5.7 $\rightarrow$ 3.4	-2.3
Apparatus	ColonyCounter	91.7 $\rightarrow$ 100.0	+8.3	6.8 $\rightarrow$ 2.1	-4.7

7.2.5 Workflow timing

This subsection compares the total time required to count all 12 plates across the different workflows.

Figure 7.5 summarises the time it took to count all 12 plates for the different approaches. For ColonyCounter, both freehand and apparatus workflows are split into the time used to 1) take the images, 2) upload, crop the images and run the model, and 3) manually correct the annotations. The Colonia app workflow is split into 1) the time used for taking an image with the app and running the model and 2) the time for the mandatory advertisement break after counting 10 colonies.

Manual counting took 8.2 minutes on average for the full twelve-plate session. Without manual correction, ColonyCounter was substantially faster: acquisition plus upload and model inference took 3.3 minutes for freehand images and 3.1 minutes for apparatus images, corresponding to approximately 60% less time than manual counting. Apparatus image acquisition was slower than freehand acquisition, increasing from 2.1 to 2.7 minutes, but upload, cropping, and model inference decreased from 1.2 to 0.4 minutes, due to more reliable automatic cropping on apparatus images, which reduced the need to fix crop failures before running the model.

With manual correction included, the freehand workflow took 6.9 minutes, corresponding to a 16% reduction relative to manual counting, while the apparatus workflow took 7.5 minutes, corresponding to a 9% reduction. Contrary to expectations, the freehand workflow was faster than the

apparatus workflow for the complete session. The difference originated in the correction phase: free-hand correction took 3.6 minutes compared with 4.4 minutes for apparatus correction. The clearer images produced by the apparatus exposed more detections for review, including false positives from small particles in the agar media that were less visible in freehand images. The model was also trained predominantly on freehand-style images, which likely contributed to fewer prediction errors on freehand input and therefore less manual correction.

Colonia was the fastest recorded workflow before adding the advertisement time. The app workflow took 2.2 minutes, compared with 3.3 minutes for AI-only ColonyCounter on freehand images and 3.1 minutes on apparatus images. After adding the 70 second advertisement delay, the Colonia workflow took 3.4 minutes, making it similar to the AI-only ColonyCounter workflows but still faster than the corrected workflows (6.9 minutes for freehand and 7.5 minutes for apparatus). This speed advantage comes both from the lack of petri dish cropping and that image acquisition and inference happen directly on the phone, with no separate upload step.

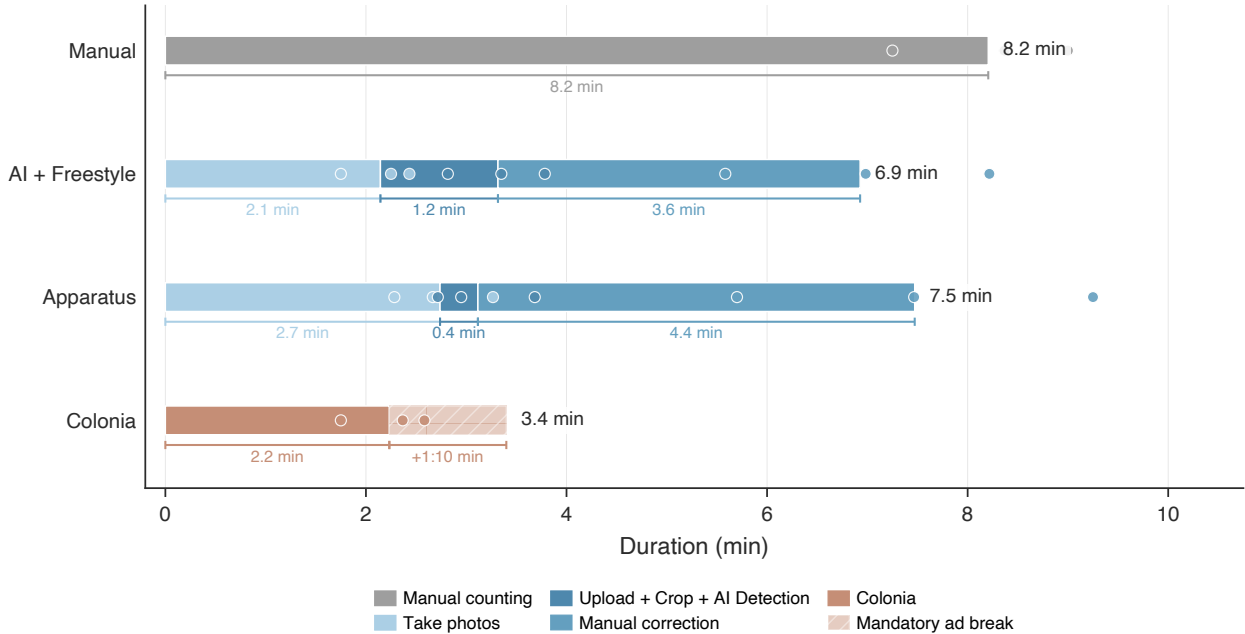


Figure 7.5: Total session time for the twelve-plate counting session. The two ColonyCounter workflows are split into acquisition, upload and model-assisted counting, and correction. Manual counting and Colonia are shown as total session times.

Table 7.3 shows the average end-to-end per-plate time (including manual correction) for each workflow for low density, medium density and high density plates. The speed advantage of the ColonyCounter workflows over manual counting was limited to high-density plates. At low and middle density (up to 30 colonies), manual counting remained faster, taking approximately 5 and 14 seconds per plate respectively, compared with around 20 and 24 to 27 seconds for the ColonyCounter workflows, as the fixed acquisition and upload overhead dominated over the short correction time. At high density the relationship reversed, with manual counting taking 104 seconds per plate compared with 59 seconds for ColonyCounter using freestyle images and 64 seconds for ColonyCounter with apparatus images, corresponding to reductions of 43% and 38% respectively.

Table 7.3: Mean end-to-end counting time per plate by colony density and counting method. Times include image acquisition, upload, model inference, and manual correction.

Method	Low Density (2–4 colonies)	Medium Density (20–30 colonies)	High Density (190–250 colonies)
Manual	5 s	14 s	104 s
AI, Freestyle	21 s	24 s	59 s
AI, Apparatus	20 s	27 s	64 s

7.2.6 Accuracy-time tradeoff

The fastest workflow was not the most accurate. After adding the 70 second advertisement delay used in this comparison, Colonia took 3.4 minutes and reached 44.4% CA@10/±1, as shown in Figure 7.6. The uncorrected ColonyCounter predictions gave a stronger automatic-counting tradeoff, with freehand images reaching 97.2% CA@10/±1 in 3.3 minutes, while apparatus images reached 91.7% in 3.1 minutes.

The corrected workflows show the effect of manual verification. Corrected freehand counting reached 100.0% CA@10/±1 and took 6.9 minutes, while corrected apparatus counting reached 100.0% CA@10/±1 and took 7.5 minutes, compared with 8.2 minutes for manual counting. Both corrected ColonyCounter workflows therefore reached complete threshold agreement faster than manual counting. The strongest AI-only result was obtained on freehand images.

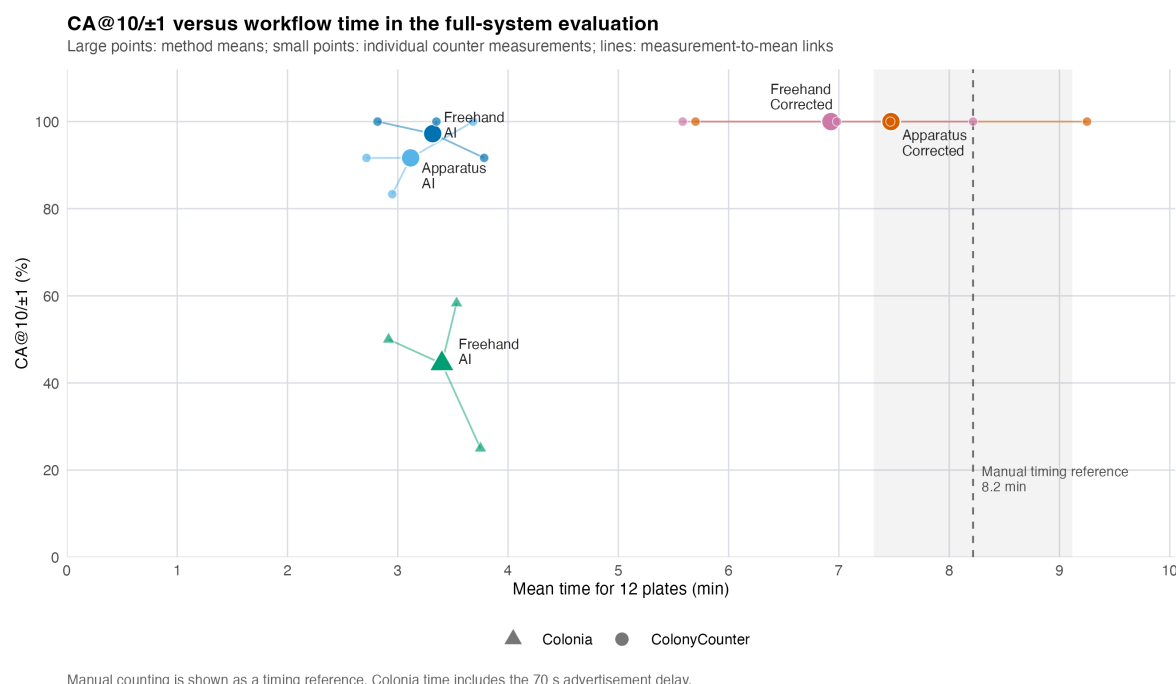


Figure 7.6: CA@10/±1 versus mean workflow time in the full-system evaluation. Large points show method means, small points show the three individual counter measurements, and semi-transparent lines connect each individual measurement to its method mean. Point shape separates ColonyCounter from Colonia. Manual counting is shown only as a timing reference because the manual counts define the reference used for CA.

7.3 Conclusion

Overall, the full-system evaluation shows that ColonyCounter provides the strongest counting performance among the evaluated workflows. The uncorrected ColonyCounter predictions reached higher CA@10/ ± 1 than both Colonia baselines in both acquisition conditions, with the largest separation on high-density plates.

ColonyCounter without correction performed better on freehand images than on apparatus images, reaching 97.2% compared to 91.7% CA@10/ ± 1 . This was unexpected, since the apparatus images were more standardised. One possible explanation is that the final model was trained and optimized mainly on freehand-style images, so the apparatus did not improve uncorrected model accuracy directly in this test. Instead, its main contribution was to reduce upload and cropping time from 1.2 to 0.4 minutes through more reliable automatic cropping. However, the clearer apparatus images also revealed more agar fragments and small particles in the media, which the model flagged as additional false positives, adding to the correction effort and resulting in a longer correction phase than for freehand images.

Both corrected ColonyCounter workflows reached 100.0% CA@10/ ± 1 and were faster overall than the 8.2 minutes required for manual counting, with the corrected freehand workflow taking 6.9 minutes and the corrected apparatus workflow taking 7.5 minutes while reaching the lower MAE of 2.1 colonies compared with 3.4 for freehand. This time advantage was however limited to high-density plates. At low and medium densities, manual counting remained faster, since the fixed acquisition and upload overhead of the ColonyCounter workflows outweighed the short correction time needed on these simpler plates.

Colonia was faster than both ColonyCounter workflows, but its lower accuracy meant that speed alone did not make it the strongest complete workflow.

8 Future Work and Preliminary Investigations

Throughout the project, several alternative approaches and future development directions were considered alongside the primary system development. Some of these were explored experimentally and produced preliminary results, while others emerged from observed limitations in the final system and are included as prospective directions for further work. This chapter begins by discussing laboratory-specific models as a potential strategy for adapting the system to new data distributions, followed by a discussion of our preliminary investigations into synthetic data generation and colony classification, as well as possible future directions for both.

8.1 Laboratory Specific Models

Across the detection experiments, performance was strongest when the colony appearances, agar media, density ranges, and imaging conditions in the test images were well represented in the training data. This was most clearly demonstrated in the held-out external benchmark, where the final pipeline performed well overall but remained weaker on Chrome agar. A similar pattern was observed in the imaging box evaluation. Although the apparatus produced more standardised images, these images differed from the freestyle images used during training, which led to reduced detection performance. In both cases, the limitation was therefore not only related to model architecture, but also to a mismatch between the training distribution and the images encountered at test time.

Moving toward laboratory-specific models could reduce this problem. Many laboratories repeatedly work with the same organisms, agar media, imaging setup, and approximate density ranges. Rather than expecting one general-purpose model to perform equally well across all possible microbiological conditions, deployment could begin with a smaller calibration dataset from the target laboratory. Images from that laboratory could be annotated or corrected and then used to fine-tune the detector before routine use. This would make the model better matched to local colony appearances, plate artefacts, lighting conditions, and camera setup.

The human-in-the-loop workflow also makes this transition feasible after deployment. Each corrected plate can act as a labelled example from the laboratorys actual data distribution. These corrections could then be used for periodic retraining or fine-tuning, allowing the detector to adapt as the laboratory introduces new agar types, bacterial strains, or imaging conditions. In this setup, unfamiliar cases would initially require more manual correction, but that correction effort would also generate the data needed to reduce the correction burden over time.

8.2 Synthetic Data Generation

Synthetic data generation was explored as a potential method for increasing dataset diversity and reducing the need for manual annotations. Beyond simply producing more training images, a robust synthetic data pipeline could enable controlled generation of difficult or underrepresented cases, such as dense colony growth, overlaps, low-contrast colonies, rim artifacts, lighting variation, and different agar appearances. This could support targeted augmentation, model stress testing, or

pretraining before fine-tuning on real annotated plates. Two separate approaches were investigated during the project, although neither was integrated into the final training workflow.

8.2.1 Overlap Augmentation

One explored direction focused on generating artificial overlaps directly from existing colony detections. Individual colonies were selected and transformed using randomized geometric and photometric variations before being placed in partially overlapping configurations and blended together. Figure 8.1 illustrates the overall augmentation process.

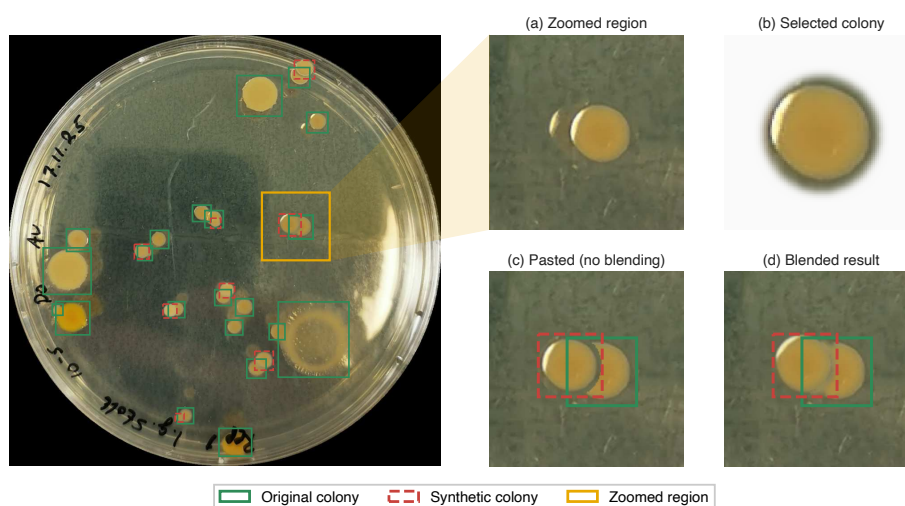


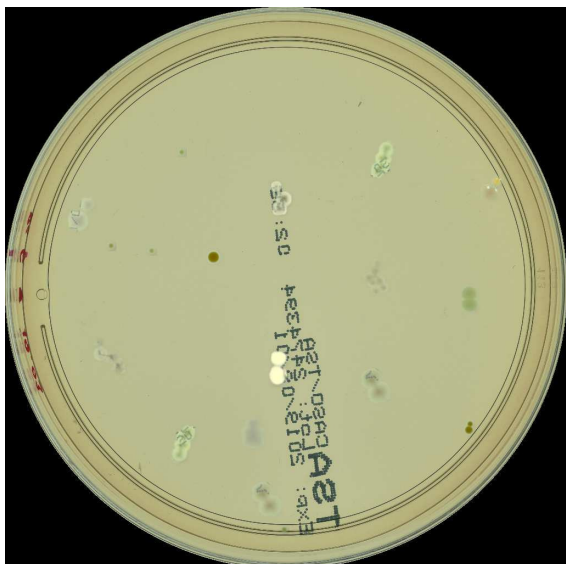
Figure 8.1: Example pipeline for generating synthetic overlapping colonies. A detected colony is selected, transformed, pasted into a nearby region and blended. Although blending reduces hard seams, artifacts may remain around colony boundaries and in the overlap region.

The generated samples were visually convincing in some cases, particularly for isolated colonies with clear boundaries and strong contrast. However, the pipeline also revealed several limitations. Since colonies were generated by transforming and pasting existing detections, the realism depended heavily on the quality of the extracted colony crop. Crops containing halos, shadows or agar texture could transfer these artifacts into the synthetic image.

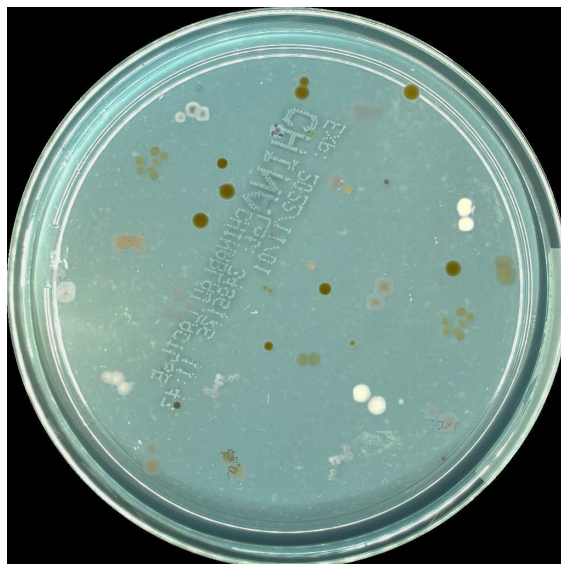
In addition, even with blending, the overlap region sometimes appeared artificial: colony edges could become overly smooth, visible seams could remain, and the pasted colony did not always interact naturally with the underlying colony or agar background. This made some samples resemble layered image composites rather than true biological colony overlaps. Because these artifacts could bias the detector toward synthetic visual cues, the method at its present state was not integrated into the final training workflow.

8.2.2 Full Plate Synthesis

We also explored full-plate synthesis, where segmented colonies from existing datasets were placed onto empty plate backgrounds using randomized spatial sampling to approximate natural colony distributions. Different blending techniques were explored to preserve colony translucency and texture, and optional neural style transfer methods were tested to increase visual diversity between generated samples. Examples of the synthesized plates are shown in Figure 8.2.



(a) Example of full plate synthesized data



(b) Second example of full plate synthesized data

Figure 8.2: Examples of full plate synthesized data.

The synthesized plates demonstrated that large-scale artificial dataset generation was technically feasible, but the generated images remained visibly synthetic. The main limitation was imperfect blending between the pasted colonies and the underlying plate background, which made many colonies appear inserted rather than naturally present on the agar surface. A limited preliminary training experiment using a YOLOX Nano model showed that the model could learn basic colony-like structures from the synthetic data, but did not generalize well to the broader appearance variation present in real-world agar plates. Given these results and the time constraints of the project, the pipeline was not considered mature enough to include in the final training workflow in its current form.

8.2.3 Summary and outlook

Overall, synthetic data generation showed promise as a complementary tool for increasing dataset diversity and creating controlled examples of challenging colony appearances. However, both overlap augmentation and full-plate synthesis introduced visible artifacts. Future work could improve this direction through more realistic colony-background interaction, and improved colony-colony interaction for overlapping colonies. In addition, generative models such as diffusion models or GANs could be explored to produce more realistic synthetic data. Another possible direction is simulation-based synthetic data generation, where plate geometry, agar appearance, colony morphology, lighting, and camera conditions are modeled to produce more realistic and controllable training images.

8.3 Colony Classification

Colony classification was investigated as an exploratory track to determine whether different colony types could be separated automatically within individual agar plate images. The core pipeline we investigated first extracts features from the detected colonies and then applies an unsupervised clustering method, in this case k-means, to construct colony clusters. We evaluated three different

feature sets on a test dataset consisting of 12 manually annotated real-world images. For the first two feature sets, blob detection was applied within each bounding box to isolate the colony region before feature extraction. For more details about our approach see A.10.

Feature set 1, LAB colours: The first feature set serves as a baseline and consists only of the three LAB colour channels. LAB was chosen because its distances more closely reflect human-perceived colour differences than e.g.. RGB, making it suitable for a clustering task intended to approximate manual colony classification.

Feature set 2, synthetic data based: The second feature set uses a synthetic dataset for feature selection (using backwards selection) based on 74 candidate image features, including LAB colours. The synthetic dataset was constructed by compositing colony crops from single-species source plates across 15 bacterial species onto shared 2048×2048 canvases, yielding 17,816 colony instances with known ground-truth clusters across 500 synthetic images.

Feature set 3, embeddings: The third feature sets consists of deep embeddings extracted from the DINOv2 transformer backbone from the full-image RF-DETR detection model. Colony bounding boxes were projected into the intermediate 256-channel feature map, ROI-aligned, spatially pooled to 3×3 , and averaged to yield a 256-dimensional embedding per colony, which was further reduced to 11 features through the same selection procedure as described in "Feature set 2, Synthetic data based".

Clustering performance was evaluated using the Adjusted Rand Index (ARI), which measures agreement between the predicted clusters and the manual labels. Based on the test set evaluation, the synthetic data based feature set (ARI 0.089) and the DINOv2 embeddings (ARI -0.016) showed no meaningful clustering signal, with scores close to 0 indicating near-random agreement. The LAB colour baseline feature set achieved the highest clustering performance (ARI 0.408); however, compared with the maximum ARI score of 1, this still reflects limited agreement with the manual labels. For each final evaluation, k-means was run with k set to the ground-truth number of classes, reflecting the assumption that application users will know the number of classes in advance.

Several factors could have contributed to these results. Firstly, the synthetic data used for feature selection did not fully capture the variability present in real Petri dish images. Although the synthetic dataset provided controlled ground-truth labels, it was generated by compositing colonies from single-species source plates onto shared canvases. This likely preserved species-specific colour and morphology under relatively clean conditions, but did not reproduce important sources of variation in real plates, such as lighting differences, changes in growth stage, and interactions between different bacterial types on the same plate. As a result, features selected on the synthetic data did not generalise well to the manually annotated real-world test images.

Secondly, the DINOv2 embeddings were extracted from the RF-DETR detection backbone, which was optimised for colony detection rather than fine-grained colony classification. The ROI-aligned embeddings therefore encode information useful for localising colony-like objects, but not necessarily the subtle colour, texture, and morphological differences needed to separate visually similar colony types.

Thirdly, the task itself carries inherent ambiguity: colonies of the same bacterial species can exhibit

markedly different morphologies and colours depending on their growth phase, and reasonable annotators can disagree on whether visually distinct colonies of the same species should be grouped together.

These results suggest that the tested unsupervised feature representations were not sufficient for reliable general colony classification. Achieving more general classification would likely require labelled real-world data or more targeted feature learning than was feasible within this project. Although the LAB colour feature set achieved only moderate clustering performance (ARI 0.408), it remained useful for simpler classification scenarios where the expected colony types differ primarily in colour. This is often the case in two-colour assays, such as white-blue colony screening, where the goal is not general species classification but separation of visually distinct colony groups. A LAB colour scale classifier was therefore integrated into the platform and is actively used in the laboratory of group leader Mogens Kilstrup for assay work (Figure 8.3).

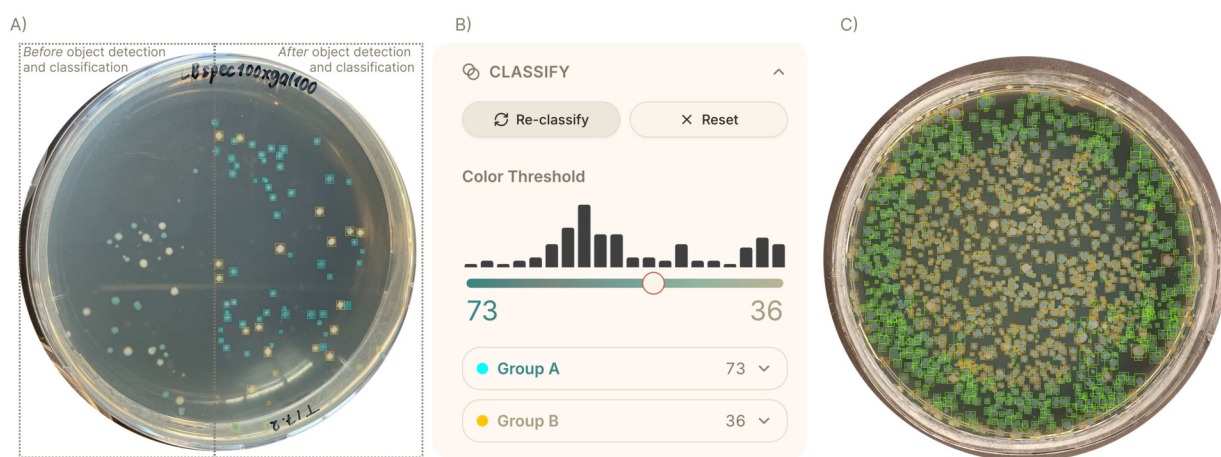


Figure 8.3: A) Shows a white-blue colony assay before and after running the object detection and classification model using LAB colours. B) shows the UI interface for changing the colour threshold, and seeing the counts for each class. C) This shows an example of classification failure caused by shadows around the plate rim, which prevent the white and blue colonies from being grouped correctly.

8.3.1 Summary and outlook

Overall, colony classification was explored as an extension of colony detection, but the tested unsupervised approaches showed limited performance on real-world images. The LAB colour baseline performed best and was integrated into the platform for simpler colour-based assays, such as white-blue colony screening. However, the weaker performance of the synthetic feature set and DINOv2 embeddings suggests that general colony classification likely requires labelled real-world data and more task-specific feature learning. Future work could explore supervised classification on labelled real colony crops or contrastive learning methods that learn embeddings where visually similar colonies are grouped together. Another practical direction is interactive human-in-the-loop classification, where users select a few example colonies from each class and the system classifies the remaining colonies by similarity. In the longer term, improved synthetic data generation could also support these approaches by providing additional labelled examples and more controlled variation in colony appearance.

9 Discussion and Conclusion

This chapter discusses and concludes the thesis. It first recaps the work across the iterations, then revisits the candidate features against their delivery status, evaluates each research question in turn in order to draw an overall conclusion. The recap below is organised by iteration in the order they were carried out, alongside a separate imaging-hardware chapter.

Iteration 0: Baseline and user grounding Iteration 0 began with a user study to identify researchers needs and system constraints. A central finding was that researchers do not trust fully automated counts for scientific documentation, which made interactive verification the design anchor for the rest of the project. A baseline prototype was then evaluated in a controlled counting study, which confirmed that AI-assisted counting with manual verification was more reliable than AI-only counting, with the improvement concentrated on high-density plates. The study also exposed two gaps that drove the iterations that followed. End-to-end processing including the FastSAM-based dish detection was slow, and the colony detection model had decreased performance on dense plates and had not yet been evaluated under challenging and diverse real-world conditions.

Iteration 1: Deployment and workflow Iteration 1 addressed the system-side research question by closing the deployment, performance, and usability gaps exposed by the baseline. Here, we aimed to get a fully functional platform up and running. The FastSAM dish cropping model was replaced with a browser-side YOLOX-Nano running in ONNX format, which removed the slow server upload identified in Iteration 0. The colony detection model was converted to OpenVINO format and the upload workflow restructured, which together shortened the end-to-end processing time per image from about 20 seconds to under 6 seconds. The system was deployed as a publicly hosted service at colonycounter.dk with email-verified authentication, per-user rate limiting, and multi-format batch export. A think-aloud evaluation with relevant end users drove the addition of project-based organisation, an onboarding tour, a dot annotation tool, and a right-click context menu.

Iteration 2: Detection model Iteration 2 addressed the detection research question, which the baseline system had identified as the primary technical bottleneck. Two new datasets were constructed to support more robust training, NatureDTU covering 24 bacterial species and ocean environmental samples across diverse imaging conditions and CrossLab (Val II) assembled from images collected across different laboratory settings. A detection-head ablation on YOLO26 and an evaluation of RF-DETR as a transformer-based alternative pointed toward RF-DETR as the stronger model for plates with up to 300 colonies. Its hard detection ceiling at 300 object queries was addressed through a dual-model setup, pairing a full image RF-DETR model for sparse plates with a 9-slice SAHI configuration for dense plates, routed by the full image RF-DETR count itself. The combined model was evaluated on a held-out external benchmark of 108 plates against Colonia, Colony Grounded SAM2, and several other tools.

Imaging hardware A purpose-built imaging box was developed as a separate attempt to overcome image variability that model-side improvements alone could not remove. The synthetic glare, text, and shadow augmentations tested in Iteration 2 gave only a marginal benefit to the full-image model and reduced the performance of the dense-plate SAHI model, which pointed toward image acquisition rather than further model tuning as the next lever. The foamboard prototype combines shallow-angle side LED lighting with an optional bottom backlight to provide diffuse and repeatable illumination.

9.1 Feature Delivery

The user study at the start of Iteration 0 identified twenty candidate features and placed them in an effort-impact matrix (Figure 3.1). In Figure 9.1 we have re-rendered that matrix with the delivery status of each feature at the close of the project.



Figure 9.1: Effort-impact matrix re-rendered with delivery status at the end of the project. Green denotes delivered, yellow partially delivered, red attempted without a working approach, and gray not started.

The Quick Wins quadrant was fully delivered, which is consistent with the original prioritisation. • **High-Quality Counting**, • **Filters**, • **Data Persistence**, • **Export** and • **Interactive Tools** all shipped in Iteration 0 or Iteration 1. • **Colour Classification**, also a Quick Win, was delivered as a simple LAB colour scale classifier with the caveat that the result is limited to colour.

The Major Projects quadrant produced the most interesting mix of outcomes. • **Medium-Quality Counting** was delivered through the additional training datasets with more diversity and the Dual RF-DETR Model. • **Low-Quality Counting** is partially possible, but often still requires manual adjusting on the platform for accurate results. The • **Imaging Box** prototype was built in foamboard and tested in the Final Test evaluation, where it successfully increased image quality and achieved the lower MAE after correction, though the clearer images required a slightly longer manual correction phase than freestyle. The • **Mobile UI** was not developed, however the website on a mobile interface still supports image upload. • **Morphology Classification** was the only Major Project attempted without delivering a working approach, with both the synthetic data based feature set and the DINOv2 deep embeddings underperforming the simpler LAB colour features.

The Fill-Ins quadrant was mostly delivered. An embedded YOLOX-Nano model provides • **Auto Cropping** on the platform, and • **Project Management** and • **Custom Styling** were both delivered. The • **CFU Calculator** shipped as a useful but incomplete solution, and • **Growth Curves** for multi-plate replicates and more detailed growth analysis were not built.

The Avoid quadrant remained almost entirely unbuilt as planned. • **Outlier Detection** was not explored, as the problem closely overlaps with the unsuccessful morphology classification work. • **Multi-Image Anti-Glare** was eliminated as a need rather than addressed, since the imaging box reduced the practical case for software-side glare removal. • **Benchling** was not prioritised, while initial research in • **OCR** found that needed models would be unfeasibly large for our use case.

9.2 Research Questions in Retrospect

The three sub-questions and the main research question are addressed below. Each box restates the original question and provides a direct answer grounded in the evidence gathered across all iterations.

Q1: Detection

To what extent can a deep learning detection pipeline be developed and optimised for robust, generalisable colony counting across colony morphologies, densities, and imaging conditions encountered in laboratory practice?

The final object detection pipeline was developed by improving the data, model, and inference strategy together.

On the held-out external benchmark, the final pipeline achieved the highest CA@10±1 among all tested tools, including the newly released Colonia and Colony Grounded SAM2. However, performance was not uniform across all conditions. Accuracy remained lower on underrepresented cases, particularly Chrome agar and very dense plates. Failure analysis further showed that errors often occurred near plate edges, where glare, shadows, and overlaid text obscured colonies. Targeted augmentations for these artifacts, together with additional calibration, produced mixed results and did not fully eliminate these failure modes.

These results suggest that the remaining failure modes are difficult to solve through model-side tuning alone. Further improvements are more likely to come from expanding the training data to cover the missing agar types, colony appearances, and density ranges, or from improving image acquisition so that the detector receives cleaner and more consistent inputs.

The research question can therefore be answered positively, but only to a limited extent. A deep learning detection pipeline can be developed and optimised to substantially improve colony counting in practical laboratory settings. However, the results do not support treating it as a robust, fully general automatic counter across all colony morphologies, densities, and imaging conditions.

Q2: Platform

How can a platform for a semi-automated colony counting system be built to support an efficient and accessible counting workflow for researchers and laboratory practitioners?

The platform question was addressed by treating semi-automated colony counting as a workflow problem rather than only a model-deployment problem. For the system to be useful to researchers, it had to support trust, efficiency, accessibility, and documentation.

Trust was addressed by making interactive verification central to the workflow, allowing users to inspect, filter, correct, and annotate predictions before accepting a count. Efficiency was addressed by making Petri dish cropping fast enough for interactive use through a browser-side YOLOX-Nano model, and by optimizing the upload and backend pipeline. Accessibility was addressed by deploying the system as a publicly hosted browser-based service at colonycounter.dk, with authentication, rate limiting, and persistent projects.

Finally, laboratory documentation was supported through project organisation, CFU calculation, custom styling, and structured multi-format export. The answer to the platform research question is therefore that an efficient and accessible platform must combine automatic detection with the surrounding tools needed to review, organize, and export results, rather than simply present model predictions to the user.

Q3: Imaging Apparatus

To what extent does a purpose-built, low-cost imaging prototype reduce imaging variability and improve the reliability of the automated colony counting workflow?

The imaging hardware question was addressed by developing a low-cost foamboard prototype with diffuse side-mounted LED lighting and an optional bottom backlight.

In the final evaluation, the apparatus improved the reliability of automatic cropping, suggesting that the controlled setup made plate localisation more consistent. However, the apparatus did not improve uncorrected detection performance. Apparatus images achieved a CA@10/ ± 1 of 91.7% before manual correction, compared with 97.2% for freestyle images. This suggests that while the

apparatus reduced some acquisition variability, it also changed the image distribution. Since the model was trained primarily on freestyle images, the more standardised apparatus images likely differed enough from the training data to reduce detection accuracy.

After manual correction, apparatus images achieved a lower MAE than freestyle images, at 2.1 versus 3.4 colonies. This indicates that the apparatus made colonies easier to inspect and correct manually, even if it did not improve the fully automatic workflow. An exploratory one-person subset showed that the bottom backlight improved performance by 8.3 percentage points, suggesting that although the apparatus did not improve uncorrected detection overall, improved contrast from lighting remains a relevant design factor.

The research question can be answered in two parts. The prototype reduced acquisition variability and improved the corrected workflow through more consistent dish positioning, more reliable cropping, and lower post-correction MAE. However, it did not improve uncorrected automatic detection in its current form. The apparatus should therefore be viewed as a promising support component rather than a standalone solution to imaging variability. Further gains would likely require training or fine-tuning the detector on apparatus images.

Main Research Question

Building on an established proof-of-concept, how can a semi-automated colony counting system be improved to achieve robust and generalisable colony detection within an accessible workflow, across diverse conditions encountered in real laboratory environments?

The thesis provides a semi-automated colony counting system by addressing three research questions, each covering one part of the same counting workflow. **Q1** addressed detection, where a dual RF-DETR setup replaced the original YOLO baseline. **Q2** addressed the platform, deployed as a publicly hosted website that takes a user from image upload through interactive correction to structured export. **Q3** addressed acquisition, where a low-cost foamboard prototype was built to standardise how plates are photographed.

These three parts should not be seen as independent but rather as three tightly connected pillars of one workflow. **Image Acquisition** is how the plate is photographed and how closely the image matches the model's training data, **Detection** is how well the dual RF-DETR model finds the colonies in the image, and **Platform** is where the user uploads the image, reviews the detections, and corrects the count before export. Reaching a reliable colony count requires work across all three. An image closer to the model's training data eases detection, a stronger detector eases correction, and careful correction absorbs the errors the detector leaves behind.

The final system combines the dual RF-DETR model with an interactive web interface where users inspect, correct, and export colony counts, turning the model output into part of a practical counting workflow rather than a standalone prediction. It was evaluated on 12 plates spanning three density ranges, counted by three annotators. Although small, the evaluation showed that combining the model detections with interactive correction brought the workflow to 100% practical counting accuracy (CA@10/ ± 1) for both freestyle and apparatus images completing the sessions

in 6.9 minutes and 7.5 minutes respectively, against 8.2 for manual counting. The time advantage was concentrated on the dense plates where the workflow reduced time with 43.3% and 38.5%. These findings suggest that a semi-automated approach can effectively balance accuracy and efficiency, leveraging automation where it performs well while retaining user oversight where it remains necessary. The main research question can therefore be answered as follows: robust and generalisable semi-automated colony counting is best achieved by distributing the counting task across acquisition, detection, and user-guided correction.

Bibliography

- [1] Christopher J Petzold and Aindrila Mukhopadhyay. “From bench to biofactory: high-throughput technologies and automated workflows to accelerate biomanufacturing”. In: *Current Opinion in Biotechnology* 94 (2025), p. 103320. ISSN: 0958-1669. DOI: <https://doi.org/10.1016/j.copbio.2025.103320>. URL: <https://www.sciencedirect.com/science/article/pii/S0958166925000643>.
- [2] Laura Elizabeth Lansdowne. “Automating Research to Improve Reproducibility and Throughput”. In: *Technology Networks* (July 2019). Accessed: 2026-03-05. URL: <https://www.technologynetworks.com/drug-discovery/articles/automating-research-to-improve-reproducibility-and-throughput-321482>.
- [3] Carla Fontana et al. “Laboratory Automation in Microbiology: Impact on Turnaround Time of Microbiological Samples in COVID Time”. In: *Diagnostics* 13.13 (2023). ISSN: 2075-4418. DOI: 10.3390/diagnostics13132243. URL: <https://www.mdpi.com/2075-4418/13/13/2243>.
- [4] Kritikos Antonios, Antony Croxatto, and Karissa Culbreath. “Current State of Laboratory Automation in Clinical Microbiology Laboratory”. In: *Clinical Chemistry* 68.1 (Dec. 2021). Review, pp. 99–114. DOI: 10.1093/clinchem/hvab242.
- [5] EILEEN GARRY et al. “ENUMERATING CHROMOGENIC AGAR PLATES USING THE COLOR QCOUNT AUTOMATED COLONY COUNTER”. In: *Journal of Rapid Methods & Automation in Microbiology* 17.1 (2009), pp. 46–54. DOI: 10.1111/j.1745-4581.2008.00150.x. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1745-4581.2008.00150.x>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1745-4581.2008.00150.x>.
- [6] Silvio D. Brugger et al. “Automated Counting of Bacterial Colony Forming Units on Agar Plates”. In: *PLOS ONE* 7.3 (Mar. 2012), pp. 1–6. DOI: 10.1371/journal.pone.0033695. URL: <https://doi.org/10.1371/journal.pone.0033695>.
- [7] Parker Smith and Martin Schuster. “Inexpensive Apparatus for High-Quality Imaging of Microbial Growth on Agar Plates”. In: *Frontiers in Microbiology* 12 (2021). ISSN: 1664-302X. DOI: 10.3389/fmicb.2021.689476. URL: <https://www.frontiersin.org/journals/microbiology/articles/10.3389/fmicb.2021.689476>.
- [8] Reshape Biotech. *IMAGINE Device Rack*. Accessed: 2026-05-26. 2026. URL: <https://reshapebiotech.myshopify.com/en/products/imagine-device-rack?variant=58668005261661>.
- [9] *Microbiology of the food chain — General requirements and guidance for microbiological examinations*. Identical to ISO 7218:2024. Danish Standards, 2024.
- [10] Robert S. Breed and W. D. Dotterer. “The Number of Colonies Allowable on Satisfactory Agar Plates”. In: *Journal of Bacteriology* 1.3 (1916), pp. 321–331. DOI: 10.1128/jb.1.3.321-331.1916.
- [11] James L. Fowler et al. “Analyst Variation in Doing the Standard Plate Count as Described in Standard Methods for the Examination of Dairy Products^{1,2}”. In: *Journal of Food Protection* 41.1 (1978), pp. 4–7. ISSN: 0362-028X. DOI: <https://doi.org/10.4315/0362-028X-41.1.4>. URL: <https://www.sciencedirect.com/science/article/pii/S0362028X23023165>.
- [12] John T. Fruin et al. “Accuracy and Speed in Counting Agar Plates”. In: *Journal of Food Protection* 40.9 (Sept. 1977), pp. 596–599. ISSN: 0362-028X. DOI: 10.4315/0362-028X-40.9.596.

- [13] *Heathrow Scientific Hand Tally Counter LCD | Buy Online | Heathrow Scientific | Fisher Scientific*. [Online, accessed 2026-02-24]. URL: <https://www.fishersci.co.uk/shop/products/hand-tally-counter-1/11359403#?keyword=>.
- [14] *Cole-Parmer Stuart Colony Counter 1Ea | Buy Online | Cole-Parmer | Fisher Scientific*. [Online, accessed 2026-02-24]. URL: <https://www.fishersci.nl/shop/products/stuart-colony-counter/11983044>.
- [15] M.H. Brodsky, B.W. Ciebin, and D.A. Schiemann. “A Critical Evaluation of Automatic Bacterial Colony Counters”. In: *Journal of Food Protection* 42.2 (1979), pp. 138–143. ISSN: 0362-028X. DOI: <https://doi.org/10.4315/0362-028X-42.2.138>. URL: <https://www.sciencedirect.com/science/article/pii/S0362028X2302570X>.
- [16] Interscience. *Scan^o 4000 Ultra-HD automatic colony counter and inhibition zone reader*. Accessed: 2025-11-20. 2025.
- [17] schuett-biotec GmbH. *schuett colonyQuant automatic colony counting system*. Accessed: 2025-11-20. 2021.
- [18] Reshape Biotech. *Automated Microbial Colony Counting*. Accessed 27 Nov 2025. 2025. URL: <https://www.reshapebiotech.com/>.
- [19] Anna Lewcock. *See Spots (But in a Good Way) with Automated Colony Counters*. Biocompare: The Buyer’s Guide for Life Scientists. Accessed 2026-02-24. May 2014. URL: <https://www.biocompare.com/Editorial-Articles/161464-See-Spots-But-in-a-Good-Way-with-Automated-Colony-Counters/>.
- [20] Mandel Scientific Company. *Interscience Scan^o 1200 HD Automatic Colony Counter*. Listed at CA\$27,675. 2025. URL: <https://www.mandel.ca/isl-437000> (visited on 03/03/2025).
- [21] Elisa Heuser, Karsten Becker, and Evgeny A. Idelevich. “Evaluation of an Automated System for the Counting of Microbial Colonies”. In: *Microbiology Spectrum* 11.4 (Aug. 2023). Ed. by Krisztina M. Papp-Wallace. DOI: 10.1128/spectrum.00673-23. URL: <https://pubmed.ncbi.nlm.nih.gov/37395656/>.
- [22] Nobuyuki Otsu. “A Threshold Selection Method from Gray-Level Histograms”. In: *IEEE Transactions on Systems, Man, and Cybernetics* 9.1 (1979), pp. 62–66. DOI: 10.1109/TSMC.1979.4310076.
- [23] Quentin Geissmann. “OpenCFU: A New Free and Open-Source Software to Count Cell Colonies and Other Circular Objects”. In: *PLoS ONE* 8.2 (2013), e54072. DOI: 10.1371/journal.pone.0054072.
- [24] D.H. Ballard. “Generalizing the Hough transform to detect arbitrary shapes”. In: *Pattern Recognition* 13.2 (1981), pp. 111–122. ISSN: 0031-3203. DOI: 10.1016/0031-3203(81)90009-1. URL: <https://www.sciencedirect.com/science/article/pii/0031320381900091>.
- [25] J M Bewes, N Suchowerska, and D R McKenzie. “Automated cell colony counting and analysis using the circular Hough image transform algorithm (CHiTA)”. In: *Physics in Medicine & Biology* 53.21 (Oct. 2008), p. 5991. DOI: 10.1088/0031-9155/53/21/007. URL: <https://doi.org/10.1088/0031-9155/53/21/007>.
- [26] Areesha Rehman et al. “Automated vs. Manual Counting of Bacterial Colonies Using Hough Circle Transform: A Comparative Study for E. coli, S. aureus, and P. aeruginosa”. In: *Journal of Computer Science* 21.10 (Dec. 2025), pp. 2312–2322. DOI: 10.3844/jcssp.2025.2312.2322. URL: <https://thescipub.com/abstract/jcssp.2025.2312.2322>.

- [27] Serge Beucher and Christian Lantuéjoul. “Use of Watersheds in Contour Detection”. In: vol. 132. Jan. 1979.
- [28] Caroline A Schneider, Wayne S Rasband, and Kevin W Eliceiri. “NIH Image to ImageJ: 25 years of image analysis”. In: *Nature Methods* 9.7 (June 2012), pp. 671–675. DOI: 10.1038/nmeth.2089. URL: <https://pubmed.ncbi.nlm.nih.gov/22930834/>.
- [29] G Pau. “EBImage—an R package for image processing with applications to cellular phenotypes”. In: *Bioinformatics (Oxford, England)* 26.7 (2010). DOI: 10.1093/bioinformatics/btq046. URL: <https://pubmed.ncbi.nlm.nih.gov/20338898/>.
- [30] Online Colony Counter. *Online Colony Counter*. Accessed 27 Nov 2025. 2024. URL: <https://online-colony-counter.com/>.
- [31] Oculyz. Oculyz Colony counter website, accessed 27 Nov 2025. URL: <https://www.oculyze.net/colony-counter/how-it-works/>.
- [32] cchamikara. Colony Counter: Bacteria, Playstore, accessed 27 Nov 2025. URL: https://play.google.com/store/apps/details?id=com.cchamikara.colony_counter_app.
- [33] M. Khan et al. “AutoCellSeg: robust automatic colony forming unit (CFU)/cell analysis using adaptive image segmentation and easy-to-use post-editing techniques”. In: *Scientific Reports* 8.1 (May 2018), pp. 7302–. DOI: 10.1038/s41598-018-24916-9. URL: <https://www.nature.com/articles/s41598-018-24916-9#citeas>.
- [34] L. Zhang. “Machine learning for enumeration of cell colony forming units”. In: *Visual Computing for Industry, Biomedicine, and Art* 5.1 (2022), p. 26. DOI: 10.1186/s42492-022-00122-3.
- [35] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. “Deep learning”. In: *Nature* 521.7553 (May 2015), pp. 436–444. DOI: 10.1038/nature14539. URL: <https://www.nature.com/articles/nature14539>.
- [36] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation”. In: *CoRR* abs/1505.04597 (2015). arXiv: 1505.04597. URL: <http://arxiv.org/abs/1505.04597>.
- [37] Libo Cao et al. “U2-Net and ResNet50-Based Automatic Pipeline for Bacterial Colony Counting”. In: *Microorganisms* 12.1 (Jan. 2024), p. 201. DOI: 10.3390/microorganisms12010201. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10820204/>.
- [38] Tanguy Naets et al. *A Mask R-CNN approach to counting bacterial colony forming units in pharmaceutical development*. 2021. arXiv: 2103.05337 [cs.LG]. URL: <https://arxiv.org/abs/2103.05337>.
- [39] Joseph Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection”. In: *CoRR* abs/1506.02640 (2015). arXiv: 1506.02640. URL: <http://arxiv.org/abs/1506.02640>.
- [40] Ross B. Girshick et al. “Rich feature hierarchies for accurate object detection and semantic segmentation”. In: *CoRR* abs/1311.2524 (2013). arXiv: 1311.2524. URL: <http://arxiv.org/abs/1311.2524>.
- [41] Joseph Redmon and Ali Farhadi. “YOLO9000: Better, Faster, Stronger”. In: *CoRR* abs/1612.08242 (2016). arXiv: 1612.08242. URL: <http://arxiv.org/abs/1612.08242>.
- [42] Rejin Varghese and Sambath M. “YOLOv8: A Novel Object Detection Algorithm with Enhanced Performance and Robustness”. In: *2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS)*. 2024, pp. 1–6. DOI: 10.1109/ADICS58448.2024.10533619.

- [43] Ao Wang et al. “Yolov10: Real-time end-to-end object detection”. In: *Advances in neural information processing systems* 37 (2024), pp. 107984–108011.
- [44] Glenn Jocher and Jing Qiu. *Ultralytics YOLO26 Documentation*. Accessed: 2026-03-05. Ultralytics. 2025. URL: <https://docs.ultralytics.com/models/yolo26/>.
- [45] Ranjan Sapkota et al. *YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection*. 2026. arXiv: 2509.25164 [cs.CV]. URL: <https://arxiv.org/abs/2509.25164>.
- [46] Qiang Chen et al. “Lw-detr: A transformer replacement to yolo for real-time detection”. In: *arXiv preprint arXiv:2406.03459* (2024).
- [47] Yian Zhao et al. “Detrs beat yolos on real-time object detection”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2024, pp. 16965–16974.
- [48] Yansong Peng et al. “D-FINE: Redefine regression task in DETRs as fine-grained distribution refinement”. In: *arXiv preprint arXiv:2410.13842* (2024).
- [49] Isaac Robinson et al. “RF-DETR: neural architecture search for real-time detection transformers”. In: *arXiv preprint arXiv:2511.09554* (2025).
- [50] Nicolas Carion et al. “End-to-end object detection with transformers”. In: *European conference on computer vision*. Springer. 2020, pp. 213–229.
- [51] Xizhou Zhu et al. “Deformable detr: Deformable transformers for end-to-end object detection”. In: *arXiv preprint arXiv:2010.04159* (2020).
- [52] Maxime Oquab et al. “Dinov2: Learning robust visual features without supervision”. In: *arXiv preprint arXiv:2304.07193* (2023).
- [53] Peter Robicieux et al. *Roboflow100-VL: A Multi-Domain Object Detection Benchmark for Vision-Language Models*. 2025. arXiv: 2505.20612 [cs.CV]. URL: <https://arxiv.org/abs/2505.20612>.
- [54] Michal iatka, Radim Burget, and Jan Karásek. “Machine-learning Approach to Microbial Colony Localisation”. In: *2022 45th International Conference on Telecommunications and Signal Processing (TSP)*. 2022, pp. 206–211. URL: <https://api.semanticscholar.org/CorpusID:251762352>.
- [55] Michal iatka. *BRUKERAGAR-DATASET*. <https://github.com/michalcikatka/BRUKERAGAR-DATASET>. Brno University of Technology. License: CC BY-NC-ND 4.0. 2023.
- [56] Michal Ciatka et al. “Increasing segmentation performance with synthetic agar plate images”. In: *Heliyon* 10.3 (Feb. 2024), e25714. DOI: 10.1016/j.heliyon.2024.e25714. URL: [https://www.cell.com/heliyon/fulltext/S2405-8440\(24\)01745-6?_returnURL=https%3A%2F%2Flinkinghub.elsevier.com%2Fretrieve%2Fpii%2FS2405844024017456%3Fshowall%3Dtrue](https://www.cell.com/heliyon/fulltext/S2405-8440(24)01745-6?_returnURL=https%3A%2F%2Flinkinghub.elsevier.com%2Fretrieve%2Fpii%2FS2405844024017456%3Fshowall%3Dtrue).
- [57] Sylwia Majchrowska et al. “AGAR a microbial colony dataset for deep learning detection”. In: *CoRR* abs/2108.01234 (2021). arXiv: 2108.01234. URL: <https://arxiv.org/abs/2108.01234>.
- [58] László Makrai et al. “Annotated dataset for deep-learning-based bacterial colony detection”. In: *Scientific Data* 10 (2023), p. 497. DOI: 10.1038/s41597-023-02404-8. URL: <https://www.nature.com/articles/s41597-023-02404-8>.
- [59] Michael Moucka et al. “Performance of four bacterial cell counting apps for smartphones”. In: *Journal of Microbiological Methods* 199 (2022), p. 106508. DOI: 10.1016/j.mimet.2022.106508.
- [60] ThisIsMySwamp. *Colony Counter*. Roboflow Universe dataset, accessed 24 Nov 2025. 2025.
- [61] Unknown. *Colony RF2PS*. Roboflow Universe dataset, accessed 24 Nov 2025. 2025.

- [62] Xu Zhao et al. “Fast segment anything”. In: *arXiv preprint arXiv:2306.12156* (2023).
- [63] Chien-Yao Wang and Hong-Yuan Mark Liao. *YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information*. 2024. arXiv: 2402.13616 [cs.CV]. URL: <https://arxiv.org/abs/2402.13616>.
- [64] Zheng Ge et al. “YOLOX: Exceeding YOLO Series in 2021”. In: *arXiv preprint arXiv:2107.08430* (2021).
- [65] Tsung-Yi Lin et al. “Microsoft COCO: Common Objects in Context”. In: *CoRR* abs/1405.0312 (2014). arXiv: 1405.0312. URL: <http://arxiv.org/abs/1405.0312>.
- [66] Parwit Chutichaimaytar et al. “An improved small object detection CTB-YOLO model for early detection of tip-burn and powdery mildew symptoms in coriander (*Coriandrum sativum*) for indoor environment using an edge device”. In: *Smart Agricultural Technology* 12 (2025), p. 101142. ISSN: 2772-3755. DOI: 10.1016/j.atech.2025.101142. URL: <https://www.sciencedirect.com/science/article/pii/S2772375525003740>.
- [67] Ultralytics. *Data Augmentation using Ultralytics YOLO*. <https://docs.ultralytics.com/guides/yolo-data-augmentation/>. Accessed: 2026-02-25. 2025.
- [68] Vox51. *FiftyOne*. 2025. URL: <https://github.com/voxel51/fiftyone>.
- [69] Zhuang Liu et al. “A ConvNet for the 2020s”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2022, pp. 11976–11986.
- [70] Ross Wightman. *PyTorch Image Models*. <https://github.com/rwightman/pytorch-image-models>. 2019. DOI: 10.5281/zenodo.4414861.
- [71] Henrik Pichler, Janis Keuper, and Matthew Copping. “Assessing Foundation Models for Mold Colony Detection with Limited Training Data”. In: *DAGM German Conference on Pattern Recognition*. Springer. 2025, pp. 337–352.
- [72] Jennifer Upfold et al. “Validation of an AI-Powered Automated Colony Analysis Platform Across Eight ISO Microbiological Methods: A Multi-Pathogen, Multi-Matrix Performance Study”. In: *bioRxiv* (2026). Preprint posted May 9, 2026. DOI: 10.64898/2026.05.08.723721.
- [73] Jia Du et al. “High-Resolution Colony Images of Clinically Isolated Bacteria for Automated Detection and Deep Learning”. In: *Scientific Data* (Mar. 2026). ISSN: 2052-4463. DOI: 10.1038/s41597-026-07095-5. URL: <https://doi.org/10.1038/s41597-026-07095-5>.

A Appendix

A.1 Contributions

Generally all members have contributed to all parts of the project and report. To optimize the use of time we have delegated tasks and therefore each member has had an increased focus on some parts. A table of members and focus areas can be seen below in Table A.1

Table A.1: Contribution overview

Frederik Valdemar Holck Reimert (s203520)	Lasse Schnell Danielsen (s203512)	Anton Egeskov Grier (s203557)
• Web Platform • Database-backend security • Background section • Routing model section • Synthetic data • Speed part of the Full System Evaluation • Imaging Apparatus Building	• DETR and RF-DETR background section • RF-DETR section • Extending RF-DETR Beyond 300 Colonies section • Model calibration, Post-processing and failure mode analysis • Final evaluation on held-out external benchmark • Counting accuracy part of the Full System Evaluation	• YOLO architecture • YOLO heads experiment • Data statistics • Imaging box section • Cropping model • Classification experiment • Imaging Apparatus Building

A.2 Statement of AI use

We have used generative AI tools, specifically large language models (LLMs), throughout the development of this thesis. All AI-generated content was critically reviewed, edited, and approved by the authors, who take full responsibility for the final work. Generally we used frontier models as some were released during the work.

Writing assistance Claude Sonnet, ChatGPT, and Gemini were used to assist with the writing process. These tools helped formulate sentences, rephrase paragraphs for clarity, and maintain consistent terminology and style across chapters written by three authors. All generated text was reviewed and revised by at least one author before inclusion.

Code generation and experiment setup GitHub Copilot and Claude Opus were used for code scaffolding, generating boilerplate, writing Python scripts for data processing and model evaluation, and configuring training experiments. All generated code was tested, debugged, and validated by the authors. No code was deployed or used to produce results without human verification.

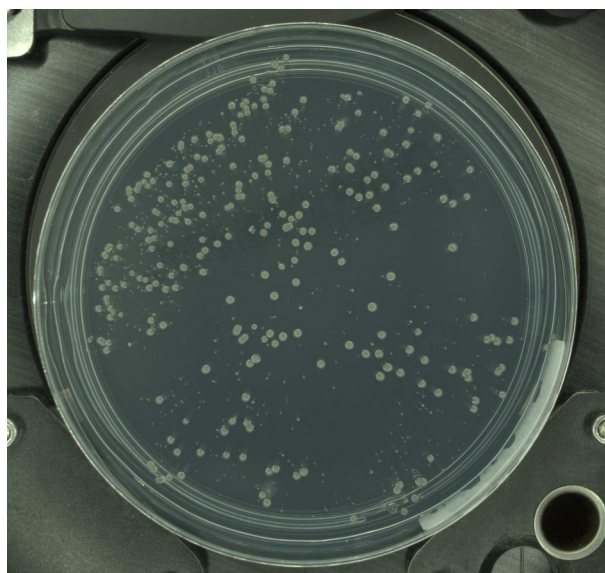
Literature and data analysis AI were used to summarise academic papers, assist in identifying

relevant references, and support the analysis of experimental data. They were also used to generate scripts for producing figures, tables, and plots. All summaries were verified against the original sources, and all visualisations were inspected and adjusted by the authors before inclusion.

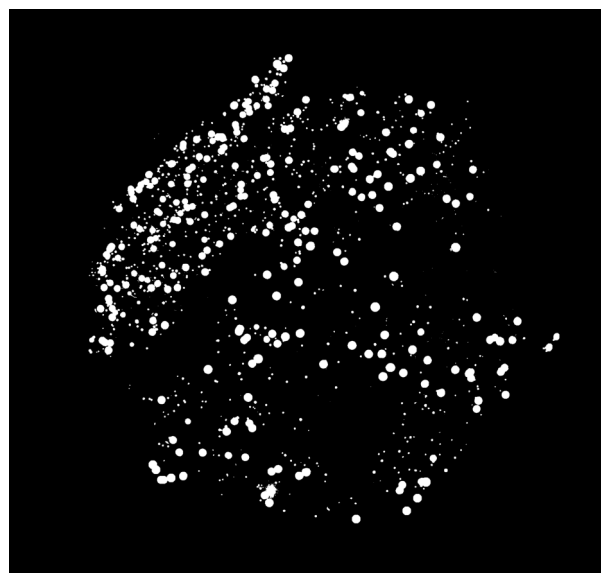
A.3 Data

This section has examples from the unused datasets

A.3.1 BRUKER Family



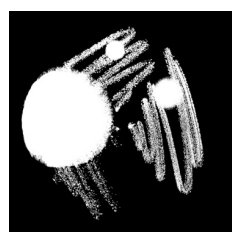
(a) Original agar plate image.



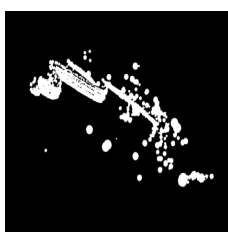
(b) Corresponding segmentation mask.



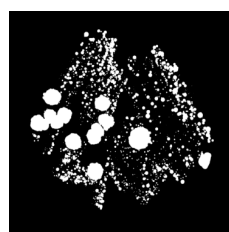
(c)



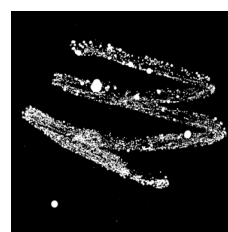
(d)



(e)



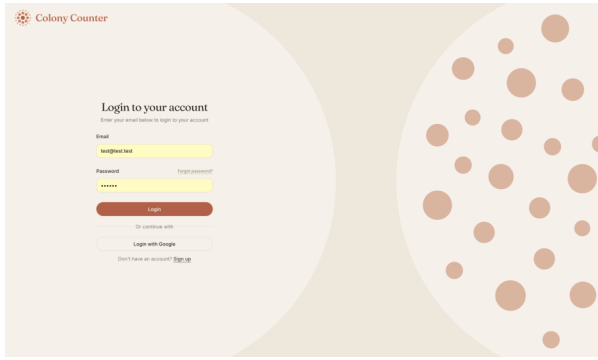
(f)



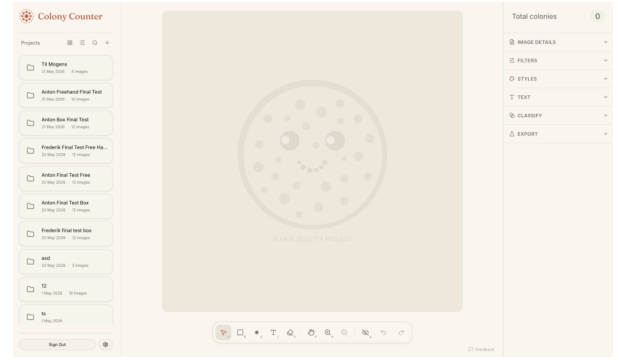
(g)

Figure A.1: Overview of the BRUKERDUST dataset. The top row demonstrates a typical raw image (a) and its manually annotated semantic mask (b). The bottom row (c-g) provides further examples of the varied segmentation masks present in the dataset.

A.4 Website Examples



(a) Front page of web app before login



(b) Front page of web app after login

Figure A.2: Web application interface before and after authentication

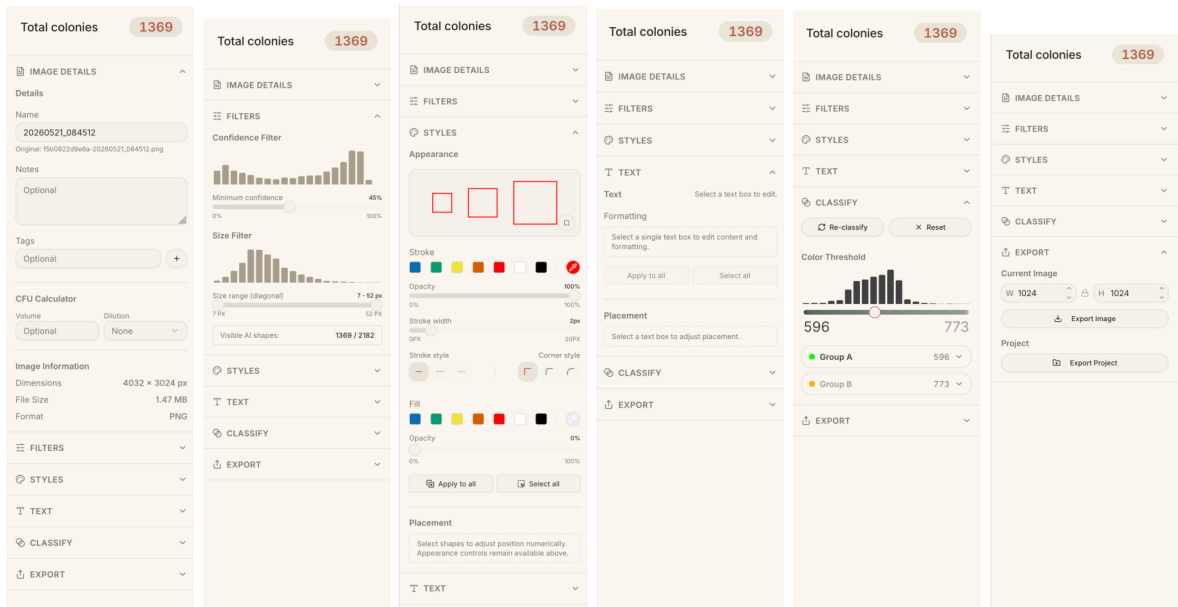


Figure A.3: Sidebar interface

A.5 Image Augmentations and Hyperparameters: YOLO

Specific image augmentations and hyperparameters for the YOLO26

A.5.1 Colony Detection

Table A.2: Image Augmentation Hyperparameters.

Parameter	<i>hsv_h</i>	<i>hsv_s</i>	<i>hsv_v</i>	<i>degrees</i>	<i>translate</i>	<i>scale</i>	<i>shear</i>	<i>persp.</i>	<i>mosaic</i>
Value	0.5	0.6	0.5	90	0.2	0.2	15	0	0

Table A.3: General Training Hyperparameters.

Parameter	<i>epochs</i>	<i>imgsz</i>	<i>batch</i>	<i>nbs</i>
Value	100	1024	18	64

Table A.4: Data augmentation parameters we used during training. HSV augmentation randomly perturbs the hue, saturation, and value of the input images to improve robustness to lighting variations. Horizontal flipping mirrors images with a given probability to increase invariance to orientation. Mosaic augmentation combines multiple images into one during training, improving generalisation and small object detection. MixUp blends pairs of images and their labels to regularise the model. Rotation applies random rotations up to the specified degrees, while translation shifts images horizontally and vertically. Shear introduces geometric distortion by slanting the image, improving robustness to viewpoint changes.

Augmentation	Value
HSV augmentation probability	0.7
Horizontal flip probability	0.5
Mosaic augmentation probability	0.8
MixUp augmentation probability	0.1
Rotation (degrees)	10.0
Translation	0.1
Shear	2.0

A.6 Hybrid RF-DETR SAHI Experiment

Hybrid RF-DETR refers to the mixed-training model evaluated as a single-model selective-SAHI alternative. It was trained on full images for cases with fewer than 250 ground truth colonies and crops generated using four slices for cases with 250 or more ground truth colonies. The threshold was set below 300 to retain headroom for low-confidence detections near the query ceiling. Four slices were used because the distribution difference was expected to be smaller between full images and images sliced in four than between full images and images sliced in nine.

The results for training Hybrid RF-DETR on a mixed dataset of full images and crops generated using four slices are shown in Figure A.4, where it is compared against Full-image RF-DETR and a model trained on crops generated using four slices, each evaluated with and without SAHI inference using four slices, alongside YOLO26 as a fixed baseline. We use a 10% overlap between tiles for all SAHI evaluations, as this performs best when using four slices in the main SAHI comparison. The results are shown for the 0-300 colony range and for the 300+ colony range.

In the 0-300 colony range, the effect of SAHI inference depends strongly on what the model was trained on. The model trained on crops generated using four slices gains slightly from SAHI inference (+0.015), as expected given that sliced tiles match its training distribution exactly. Both Full-image RF-DETR and Hybrid RF-DETR suffer a modest drop when SAHI is applied (-0.019 and -0.030 respectively), consistent with a mismatch between their training distribution and the cropped tiles presented at inference. For Hybrid RF-DETR, the majority of training examples are full images, with sliced crops only introduced for plates with 250+ colonies.

Hybrid RF-DETR nonetheless achieves the highest base performance in this range at 0.899, slightly above Full-image RF-DETR (0.892). While the difference is small and unlikely to be meaningful in practice, it is consistent with Hybrid RF-DETR being trained on a greater variety of plates, including dense ones represented as sliced crops, giving it broader exposure to different colony densities and appearances during training.

In the 300+ colony range, the picture changes substantially. Without SAHI, all three RF-DETR configurations collapse to a similar low baseline ($\text{mAP}_{50} \approx 0.620\text{-}0.631$), as the query ceiling is a fundamental bottleneck for dense images regardless of training strategy. SAHI inference rescues performance across all three configurations, with gains of +0.209, +0.238, and +0.221 for Full-image RF-DETR, the model trained on crops generated using four slices, and Hybrid RF-DETR respectively. The model trained on crops generated using four slices achieves the highest mAP_{50} among the RF-DETR configurations (0.866), followed closely by Hybrid RF-DETR (0.852) and Full-image RF-DETR (0.829). However despite these large gains, all three configurations with SAHI remain below the YOLO26 baseline of 0.878.

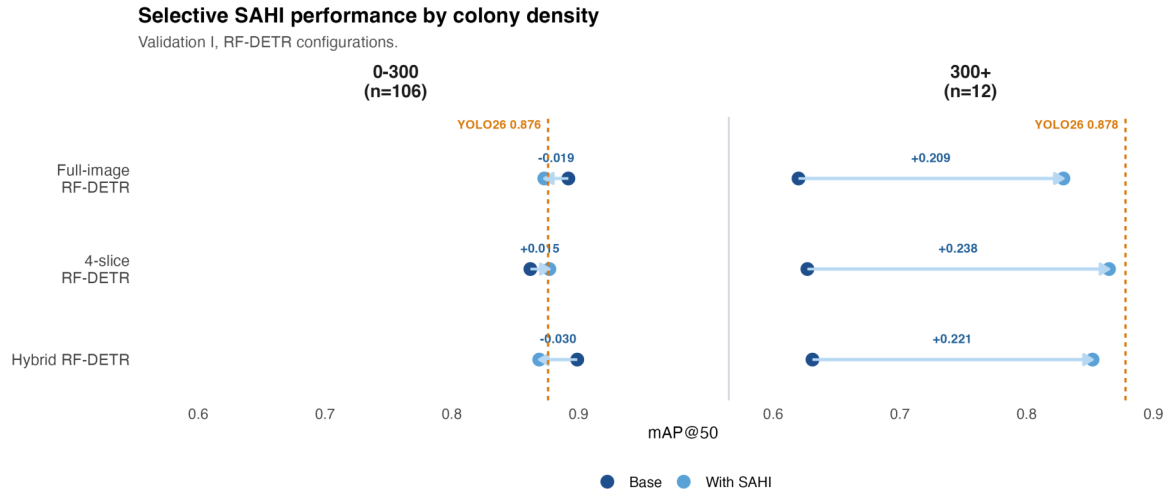


Figure A.4: One model, selective SAHI on validation I. Arrows show the change from base inference to SAHI inference for each RF-DETR configuration, with labels reporting the mAP_{50} difference. The dashed line marks the YOLO26 baseline. The model trained on crops generated using four slices used 576x576 slices originating from images resized to 1100x1100.

A.7 Routing Model: Full Val I Results

The following table reports the 11 routing configurations evaluated on Val I (118 images: 95 sparse < 200 colonies, 23 dense ≥ 200 colonies). Configurations selected for final evaluation on Val II are marked in bold. Bal. Acc. is balanced accuracy, Recall Dense is the fraction of dense plates correctly identified, and MAE is mean absolute error on count prediction.

Table A.5: Routing configurations evaluated on Val I.

Approach	Config	Bal. Acc.	Recall Dense	MAE	Latency (ms)
Otsu	1-patch	0.768	0.609	64.2	21
Otsu	2-patch	0.756	0.565	59.1	40
Otsu	3-patch	0.810	0.652	52.9	56
Otsu	4-patch	0.805	0.652	50.4	75
Otsu	5-patch	0.816	0.652	49.3	99
Regression	224	0.908	0.826	19.1	22
Regression	384	0.935	0.870	17.9	68
Regression	1024	0.995	1.000	14.0	554
YOLOX	416	0.951	0.913	8.9	31
YOLOX	640	0.968	0.957	7.9	83
YOLOX	1024	0.995	1.000	6.2	264

A.8 Routing Model: Implementation Details

A.8.1 Otsu Router

Table A.6: Otsu router pipeline hyperparameters.

Parameter	Value
Input resolution	1024×1024 px
Colour conversion	BGR to grayscale
Smoothing	Gaussian, 11×11 kernel, $\sigma = 0$
Plate mask radius	$0.47 \times \min(H, W)$
Patch radius	$0.17 \times \min(H, W)$
Patch positions	C (0.50, 0.50), N (0.50, 0.30), E (0.70, 0.50)
Otsu scope	Per-patch, masked pixels only
Polarity selection	Invert if foreground occupies more than half of patch
Connected components	8-connectivity
Area filter	$[30, 50,000]$ px ²
Calibration	$\hat{y} = a \cdot \hat{x}$, $a = \sum y / \sum \hat{x}$
Routing threshold	200 colonies

A.8.2 ConvNeXt-Atto Regression Router

Table A.7: ConvNeXt-Atto regression router architecture and training hyperparameters.

Parameter	Value
Backbone	<code>convnext_atto.d2_in1k</code>
Parameters	≈ 3.7 M
Head	Single linear unit
Input resolution	384×384 px
Input normalisation	ImageNet statistics
Target transform	$\log(1 + \text{count})$
Loss	Huber (SmoothL1), $\delta = 1.0$
Optimiser	AdamW
LR (backbone)	1×10^{-5}
LR (head)	3×10^{-4}
Weight decay	0.05
LR schedule	Cosine decay with 3-epoch warmup
Dropout	0.4
Batch size	32
Epochs	40

Table A.8: ConvNeXt-Atto training data augmentation.

Transform	Parameters
Resize	384×384
Random horizontal flip	$p = 0.5$
Random vertical flip	$p = 0.5$
Random 90° rotation	$0^\circ/90^\circ/180^\circ/270^\circ$, uniform
ColorJitter	brightness = 0.1, contrast = 0.1, saturation = 0.05, hue = 0.02

A.8.3 YOLOX-Nano Router

Table A.9: YOLOX-Nano architecture. All resolution variants share the same architecture.

Component	Configuration
Model family	YOLOX-Nano
Depth multiplier	0.33
Width multiplier	0.25
Backbone	CSPDarknet (depthwise separable convolutions)
Neck	YOLOPAFPN
Head	Decoupled YOLOXHead (cls + reg + obj), depthwise
Classes	1 (colony)
Detection scales	3 (stride 8, 16, 32)

Table A.10: YOLOX-Nano training hyperparameters per resolution variant.

Parameter	416	640	1024
Input size (px)	416×416	640×640	1024×1024
Batch size	64	32	—
Optimiser	SGD	SGD	SGD
Momentum	0.9	0.9	0.9
Weight decay	5×10^{-4}	5×10^{-4}	5×10^{-4}
Total epochs	150	150	150
No-aug epochs	15	15	15
Warmup epochs	5	5	5
LR schedule	cosine + warmup		
Pretrain init	fullplate-416	fullplate-640	—

Table A.11: YOLOX-Nano inference configuration (selected YOLOX 640 variant).

Parameter	Value
Input resolution	640×640 px
Preprocessing	Letterbox resize
Confidence threshold	0.20
NMS threshold (IoU)	0.55
Colony count	Number of detections
Routing threshold	200 colonies

A.9 External Final Evaluation Supplementary Tables

The full performances of the external final evaluation. Results are more fine grained than in the report and include bootstrapped confidence intervals

A.9.1 Freestyle smartphone-image performance

Table A.12: Freestyle smartphone-image performance with full bootstrap intervals.

Phone	Method	CA@10/ ± 1 (%)	CA 95% CI	MAE	MAE 95% CI
iPhone	Dual Model	74.1	64.8–82.4	9.5	5.6–14.1
iPhone	Colonia	55.6	46.3–64.8	21.6	13.6–30.8
iPhone	Colony Grounding SAM2	46.3	37.0–55.6	14.0	9.4–19.2
iPhone	CFU.Ai	38.9	29.6–48.1	36.2	23.9–50.7
iPhone	APD	18.5	11.1–25.9	80.8	57.5–106.4
iPhone	Promega	13.0	7.4–19.4	53.9	38.6–70.5
iPhone	CFUCounter	11.1	5.6–17.6	46.6	36.4–58.4
Samsung	Dual Model	69.4	60.2–77.8	17.7	9.8–27.5
Samsung	Colony Grounding SAM2	58.3	49.1–67.6	12.1	7.8–16.8
Samsung	Colonia	49.1	39.8–58.3	31.6	20.1–44.8
Samsung	@BactLAB	15.7	9.3–22.2	67.9	49.5–87.9
Samsung	CFUCounter	6.5	1.9–11.1	51.9	37.7–68.8

A.9.2 Paired freestyle bootstrap comparisons

Table A.13: Paired plate-level bootstrap comparisons on freestyle images. Differences are Dual Model minus comparator in CA@10/ ± 1 percentage points.

Phone	Comparator	Dual Model CA (%)	Comparator CA (%)	Difference (pp)	95% CI	n
iPhone	Colonia	74.1	55.6	18.5	9.3–27.8	108
iPhone	Colony Grounding SAM2	74.1	46.3	27.8	16.7–38.9	108
Samsung	Colonia	69.4	49.1	20.4	11.1–29.6	108
Samsung	Colony Grounding SAM2	69.4	58.3	11.1	0.0–22.2	108

A.9.3 Freestyle performance by colony-count bin

Table A.14: Freestyle performance by manual colony-count bin with full bootstrap intervals.

Manual count bin	Method	CA@10/ ± 1 (%)	CA 95% CI	MAE	MAE 95% CI
0–10	Dual Model	82.2	73.3–90.0	0.8	0.6–1.0
0–10	Colonia	80.0	68.9–90.0	0.9	0.5–1.3
0–10	Colony Grounding SAM2	58.9	50.0–67.8	1.7	1.3–2.2
10–100	Dual Model	73.0	59.5–85.1	3.9	2.3–6.2
10–100	Colony Grounding SAM2	56.8	45.9–67.6	5.1	3.8–6.5
10–100	Colonia	45.9	31.1–60.8	10.5	6.4–15.4
100–200	Dual Model	75.0	50.0–100.0	18.4	6.0–37.1
100–200	Colony Grounding SAM2	60.0	40.0–80.0	14.2	10.3–17.9
100–200	Colonia	10.0	0.0–25.0	39.3	22.1–60.4
200–300	Dual Model	50.0	16.7–83.3	45.5	13.0–78.9
200–300	Colony Grounding SAM2	25.0	8.3–41.7	39.4	29.4–49.4
200–300	Colonia	25.0	0.0–58.3	90.8	31.4–152.9
300+	Dual Model	30.0	5.0–55.0	82.9	46.4–128.2
300+	Colony Grounding SAM2	15.0	0.0–35.0	76.6	58.8–93.6
300+	Colonia	10.0	0.0–25.0	150.8	104.0–199.7

A.9.4 Freestyle performance by agar type

Table A.15: Freestyle performance by agar type with full bootstrap intervals.

Agar type	Method	CA@10/ ± 1 (%)	CA 95% CI	MAE	MAE 95% CI
Blood	Dual Model	91.7	83.3–97.2	7.4	2.3–14.8
Blood	Colonia	66.7	51.4–80.6	24.5	9.3–43.2
Blood	Colony Grounding SAM2	58.3	47.2–69.4	13.6	6.3–22.4
LB	Dual Model	72.2	59.7–84.7	9.5	4.3–15.9
LB	Colonia	65.3	51.4–77.8	14.5	6.1–24.5
LB	Colony Grounding SAM2	54.2	43.1–65.3	15.6	7.3–26.1
Chrome	Dual Model	51.4	37.5–65.3	23.9	8.9–42.3
Chrome	Colony Grounding SAM2	44.4	33.3–55.6	10.0	5.3–15.8
Chrome	Colonia	25.0	12.5–38.9	40.8	19.3–67.1

A.9.5 Cropping comparison

Table A.16: Effect of cropping on the three leading methods with full bootstrap intervals.

Cropping	Method	CA@10/ ± 1 (%)	CA 95% CI	MAE	MAE 95% CI
cropped	Dual Model	71.8	64.4–78.7	13.6	7.7–20.7
cropped	Colony Grounding SAM2	63.4	56.5–70.4	10.8	7.0–15.0
cropped	Colonia	56.0	47.2–64.8	23.4	14.9–33.2
not-cropped	Dual Model	55.1	47.7–62.5	17.5	10.2–26.4
not-cropped	Colony Grounding SAM2	52.3	45.4–58.8	13.0	8.9–17.9
not-cropped	Colonia	52.3	43.5–61.1	26.6	16.9–38.0

A.9.6 Apparatus-image performance

Table A.17: Apparatus-image performance for the strongest methods with full bootstrap intervals.

Phone	Method	CA@10/ ± 1 (%)	CA 95% CI	MAE	MAE 95% CI
iPhone	Dual Model	91.7	86.1–96.3	4.7	2.6–7.2
iPhone	Colonia	64.8	55.6–74.1	19.7	12.2–28.2
iPhone	Colony Grounding SAM2	63.0	53.7–72.2	17.9	11.6–25.2
Samsung	Colony Grounding SAM2	78.7	70.4–86.1	8.7	5.1–12.9
Samsung	Dual Model	76.9	68.5–84.3	16.2	9.5–24.1
Samsung	Colonia	61.1	51.9–70.4	31.3	19.5–44.7

A.9.7 OpenCFU and Online-Colony-Counter.com subset baselines

Table A.18: Samsung freestyle subset baseline results for OpenCFU and Online-Colony-Counter.com. These results are contextual subset baselines only and are not part of the full-benchmark ranking.

Method	CA@10/ ± 1 (%)	CA 95% CI	MAE	MAE 95% CI	n
Dual Model	53.3	26.7–80.0	23.1	7.5–44.4	15
Colony Grounding SAM2	46.7	20.0–73.3	15.5	6.9–26.5	15
Colonia	26.7	6.7–53.3	44.9	17.0–80.9	15
Online-Colony-Counter.com	20.0	0.0–40.0	41.8	23.0–63.2	15
CFUCounter	13.3	0.0–33.3	55.5	27.1–96.0	15
OpenCFU	6.7	0.0–20.0	89.5	49.9–136.0	15
@BactLAB	0.0	0.0–0.0	104.5	60.8–151.1	15

A.10 Colony Classification

One explored direction in the project was unsupervised colony classification within individual agar plate images. The goal was to investigate whether different colony types could be separated automatically based on visual appearance, without relying on predefined bacterial species labels.

To explore this, both handcrafted image descriptors and deep visual features extracted from a DINOv2-based backbone were evaluated together with clustering-based methods. The experiments provided insight into the challenges of distinguishing colony populations using feature-based representations alone.

A.10.1 Methods

Foreground Isolation and Blob Detection

To reduce background influence, each crop was first processed using a simple blob-detection pipeline. Images were converted to grayscale and thresholded using Otsu’s method to generate an initial foreground mask. Small isolated noise regions were removed.

Connected-component analysis was then applied. If the resulting foreground occupied at least half of the crop, the detected blob mask was retained. Otherwise, a centered elliptical fallback mask was used to ensure a valid foreground region in all cases. The final mask was applied to the RGB crop, setting background pixels to zero.

For deep feature extraction, RGB values were preserved. For handcrafted features, crops were normalized and converted into CIELAB space.

Data

A key challenge in this task is the lack of suitable real-world datasets for *within-image clustering*. Most existing datasets, such as those in the dataset from Nature, assign class labels across images (e.g., one species per class), whereas the present task focuses on separating heterogeneous colony populations within a single sample.

To address this mismatch, a synthetic dataset was constructed based on a recently released dataset [73], supplemented with 12 manually annotated images from the Val II dataset. These images were selected due to their higher colony density and clearer visual separation between colony types, making them suitable for evaluating clustering behaviour.

Synthetic Source Dataset

The synthetic dataset was derived from 950 agar plate images containing single-species colony growth across 19 bacterial species. Each image contained colonies from only one species, ensuring consistent ground-truth identity per source image.

Across the dataset, 118,442 annotated colony instances were available via bounding boxes, forming the basis for synthetic episode generation.

Synthetic Episode Generation

Synthetic clustering episodes were constructed by compositing colony crops from multiple source images into a single artificial scene. Each episode defines an independent clustering task.

For each episode:

1. A random number of source images ($k \in [1, 8]$) was selected.
2. Each source image was sampled from a distinct bacterial species.
3. Between 1 and 16 colony crops were randomly extracted per source image.
4. Crops were placed at random non-overlapping positions on a 2048×2048 canvas.
5. Colonies inherited cluster labels from their source image.

A total of 500 synthetic episodes were generated, containing 17,816 colony instances.

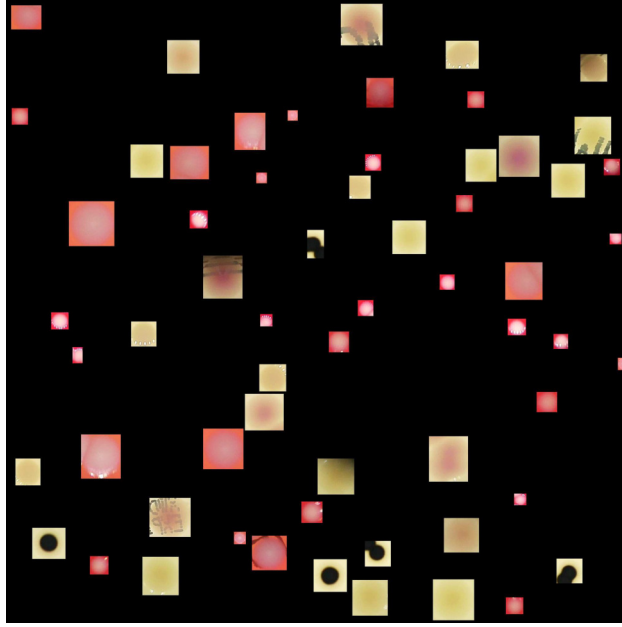


Figure A.5: Example synthetic episode containing multiple colony types within a single image.

Table A.19: Synthetic dataset statistics.

Property	Value
Generated episodes	500
Total colony annotations	17,816
Mean colonies per episode	35.6
Colony count range per episode	1–103
Source images per episode	1–8
Source species represented	15
Canvas size	2048×2048 px

Handcrafted Feature Categories

A large set of handcrafted descriptors was constructed to capture morphology, colour, intensity structure, edges, texture, and spatial variation. These features were largely generated in an automated fashion (with assistance from large language models) and subsequently grouped into interpretable categories.

The full feature list is extensive and can be seen in Table A.21. In total, 74 handcrafted features were initially considered, spanning:

Deep Feature Extraction with DINOv2

In parallel, deep features were extracted using a DINOv2-based transformer backbone from the RF-DETR model trained in the iteration 2.

Each colony bounding box was projected into the backbone feature map space. The intermediate feature map has dimensions $256 \times 64 \times 64$, located between encoder and decoder components. Bounding boxes were rescaled from 1024×1024 to 64×64 , then ROI-aligned and pooled to 3×3 ,

resulting in a $256 \times 3 \times 3$ representation. This was averaged spatially to produce a 256-dimensional embedding per colony.

The final representation combined local colony descriptors with a global image descriptor, followed by L2 normalization and feature selection.

Feature Selection

To reduce redundancy and assess feature importance, a backward feature selection strategy based on permutation importance was applied separately to handcrafted and deep features.

For each feature, clustering performance was evaluated using Adjusted Rand Index (ARI). The feature was then permuted within each image, and clustering was repeated five times. The mean drop in ARI relative to baseline was used as the importance score.

Features below a threshold of 0.01 were removed iteratively one by one. KMeans clustering was run with the number of clusters fixed to the ground-truth value, reflecting an assumed realistic setting where the number of colony types is known a priori.

A.10.2 Results

Handcrafted Feature Selection

Starting from 74 features, iterative pruning reduced the set to 30 features after 49 iterations. Baseline ARI improved from 0.342 to 0.406 over the selection process.

The final feature subset retained representative descriptors across morphology, colour, intensity, texture, and spatial structure.

DINOv2 Feature Selection

The original 256-dimensional deep representation was reduced to 11 features. However, clustering performance remained limited. Mean ARI improved modestly from 0.259 to 0.348 after selection, indicating weak alignment between learned embeddings and the clustering objective.

Final Evaluation

A final comparison was performed using the selected handcrafted features, selected deep features, and two simple baselines.

Table A.20: Clustering performance across feature representations.

Feature Set	# Features	Images	ARI Mean	ARI Std	Acc Mean	Acc Std
Colour + Area	4	12	0.275	0.279	0.703	0.160
Colour	3	12	0.408	0.237	0.772	0.154
DINO	11	12	-0.016	0.027	0.548	0.086
Handcrafted	30	12	0.089	0.136	0.636	0.084

The simplest representation, based solely on the LAB colour, achieved the strongest clustering performance. Handcrafted features provided only marginal gains over weak baselines, while deep DINOv2 features showed no meaningful clustering signal on the test set.

Overall, the results suggest that while colour carries strong discriminative power for colony grouping in this setting, neither handcrafted nor deep feature representations fully capture the structure

required for robust unsupervised colony clustering. The task remains partially unsolved and would benefit from more targeted feature learning or interaction-guided clustering strategies.

A.10.3 Implementation

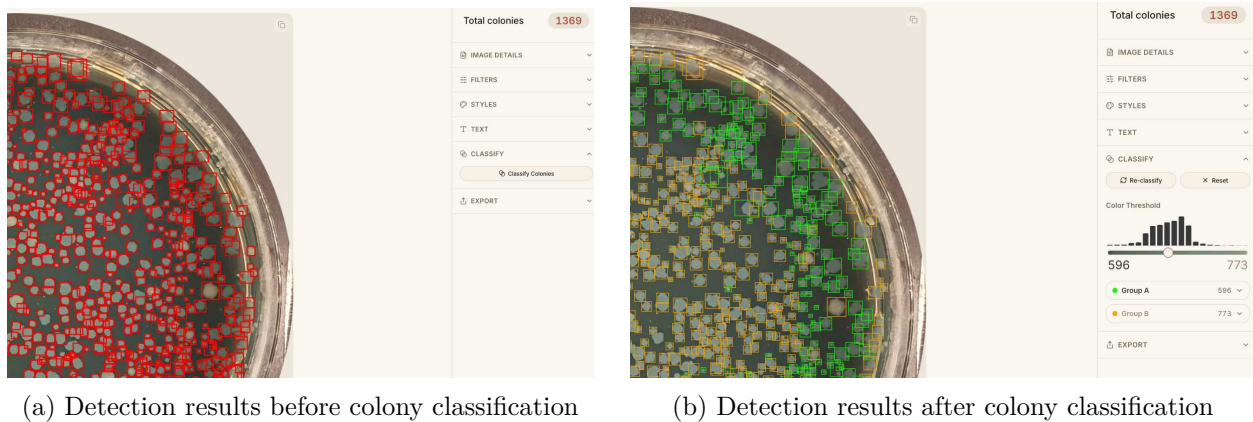


Figure A.6: Comparison of colony detection results before and after classification.

On the website the clustering algorithm was implemented just as described above using the LAB colour. For each detection we first run the blob detection using the Otsu algorithm, and then we use Kmeans to cluster the samples. We limit the number of clusters to 2 since this allows us to construct a custom threshold. The Kmeans algorithm finds two centroids which we fit a straight line through and find the distance from each annotation to the line. As each point is placed on this line, we can move the threshold and which every group is on the left or right of the threshold

A.10.4 Summary and Outlook

This experiment demonstrated that simple colour statistics dominated more complex representations in this setting, and that learned deep features did not yet align well with the clustering objective. The performance on the synthetic dataset could not be replicated in a real world example, demonstrating that the used technique for data generation was insufficient in representing realistic data. The objective in itself is prone to large bias from the human side, so a proper ground truth annotated data set would have been a difficult and cumbersome task to carry out. One colony can have multiple morphologies and colours during its growth phase, introducing variance based on the interpretation of the annotator as to whether the same bacterial species should be classified as the same group despite having different colours, and if not, at what hue they should be classified as the same. More exploratory work and data gathering would be needed to provide a proper feature representation. For now it seemed that clustering by colour was most optimal. In the current implementation of the classifying implementation, we lack a method of annotating each individual detection. Right now the user only relies on the threshold, which has its own limitations. As the clustering is solely based on colour, the algorithm is heavily influenced by imaging. As can be seen in figure A.6 (b), the plate is brighter in colour around the edge which leads to misclassification of some colonies around the edge and in the centre. This particular image was captured in our imaging box, which implies that the lighting inside the box needs to be further optimized to provide even lighting across the plate.

Table A.21: Complete handcrafted feature set grouped by descriptor category.

Feature Domain	Category	Features
Shape and Morphology	Size and geometry	Area, perimeter, equivalent diameter, bounding-box area, extent
	Convexity and compactness	Convex hull area, convex hull perimeter, solidity, convexity, circularity, compactness, perimeter-to-area ratio
	Axis-based properties	Major axis length, minor axis length, aspect ratio, eccentricity
	Structural complexity	Internal holes, skeleton length, radial distance statistics, boundary roughness, fractal dimension
Colour	Per-channel statistics	Mean, standard deviation, median, interquartile range for L^* , a^* , and b^*
	Derived colour metrics	$a^*/ b^* $ ratio, colourfulness index, coefficient of variation, joint colour entropy
Intensity	Brightness statistics	Mean, standard deviation, minimum, maximum, range
	Distribution shape	Entropy, skewness, kurtosis
Gradient and Edge	Gradient statistics	Mean gradient magnitude, standard deviation, maximum gradient
	Edge structure	Edge density
Texture	GLCM	Contrast, dissimilarity, homogeneity, energy, correlation, angular second moment
	LBP	Mean LBP value, standard deviation, histogram uniformity
Spatial Intensity Structure	Centre-edge comparison	Central intensity mean, edge intensity mean, centre-to-edge ratio
	Radial structure	Radial intensity mean, radial intensity standard deviation

A.11 Imaging Box Process

A.11.1 Methods

The imaging system was designed to minimize optical artifacts, primarily reflections, glare, and non-uniform illumination, while preserving sufficient contrast for downstream analysis of colonies. The following considerations guided the design.

Box design

External light sources can introduce uncontrolled reflections and intensity variations. Therefore, the prototype should be designed to be light-tight. A small hole will be introduced at the top for the camera lens. This opening should be minimized to prevent stray light from entering the system.

A more rounded or continuous inner surface is expected to reduce directional reflections and prevent the formation of hotspots caused by multiple reflections between flat surfaces. In practice, a curved interior should promote a more uniform redistribution of light within the chamber.

Lighting direction. Light sources will be positioned at shallow angles relative to the surface of the Petri dish. This is expected to reduce the likelihood that reflections from the plastic lid or agar surface are directed back into the camera. At steeper angles, reflected rays tend to align more closely with the axis of the camera, potentially producing strong glare. Using shallow angles, reflected light should instead be redirected away from the lens, improving the visibility of the sample.

Diffused illumination. Diffusers will be introduced between the light sources and the sample. Diffusion transforms a directional light source into a broader, more uniform emitter by scattering incoming rays across a range of angles. This is expected to reduce high-intensity peaks and soften shadows.

Diffusion is expected to be effective because it reduces the directionality of the incident light. Instead of a single dominant reflection direction, which produces sharp glare, the reflected light will be distributed over many directions with lower intensity per direction. As a result, fewer rays should align with the camera, reducing specular highlights. Additionally, diffusion is expected to minimize contrast differences caused by refractive index transitions at the plastic-air-agar interfaces, leading to more homogeneous illumination across the dish.

Light intensity will be kept as low as possible while maintaining a sufficient signal-to-noise ratio. Lower intensity is expected to reduce saturation in reflective regions and help preserve detail. Neutral or white light sources will be preferred to avoid introducing color bias, which could affect feature extraction or segmentation tasks. Consistent spectral characteristics should ensure that observed variations are due to the sample rather than illumination differences.

First stage

The first stage of our prototype was largely based on heuristics and empirical testing. We acquired a Petri dish with agar and different colonies growing on it, which we used for testing. Each choice we took was tested with the same plate.

Materials We chose to make the first prototype out of cardboard. The material could be acquired for free in abundance, and is easily manipulated into shapes. Cardboard quickly wears down, so it could not be used for any lasting construction, but for use in a minimal viable product, it was optimal. The prototype was stitched together with gaffer tape. Two light sources were used. A 1 meter plain white light LED strip and a Kindle. The Kindle was used to test whether a diffused backlight could enhance the imaging, while the LED strip was used for the main light on the Petri dish.

First iteration We started very simple with the first design, by stitching together a square box in which we could place the Petri dish and capture an image. We placed the LED strip 2 cm above the dish to get a shallow angle of light and placed a lid with a small hole for the camera. The prototype can be seen in A.7.

While the setup worked somewhat, it still left reflections and glare on the image. To try and mitigate this we used a Kindle as a backlight. The Kindle has a diffused light which enters the screen from the sides which we hoped could provide a nice lighting effect for the colonies without being too bright. An image captured at this point can be seen in Figure A.8a. The dish is illuminated and



Figure A.7: The first prototype created during development of imaging setup.

the colonies are clear. Problematically there are yet some artifacts present. Looking closely at the colonies we can see that they reflect the LED lights from one angle. This led us to try and diffuse the light. To do this we used simple parchment paper used for baking. The resulting image can be seen in Figure A.8b. This reduced the reflections that occurred on the individual colonies, but the overall contrast in the image was heightened due to lower light intensity from the top.



(a) Backlight and LED strips.

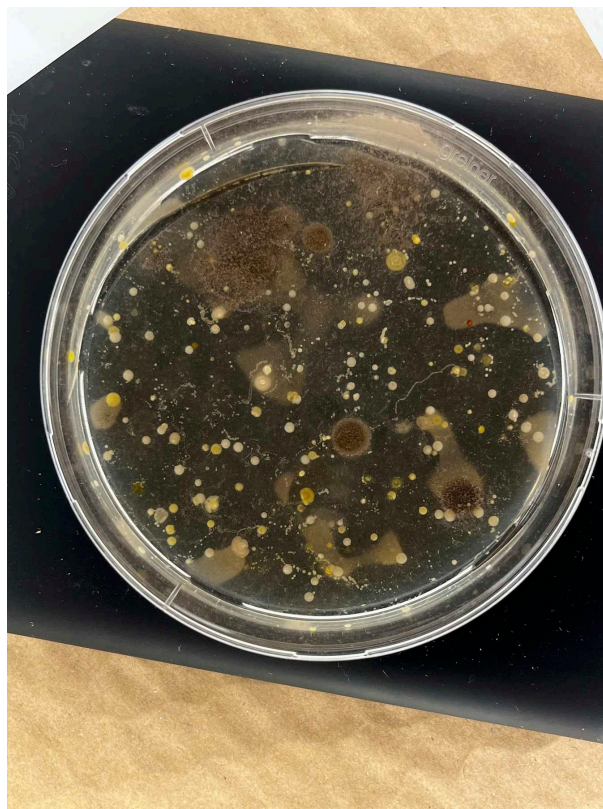


(b) Parchment paper diffusion.

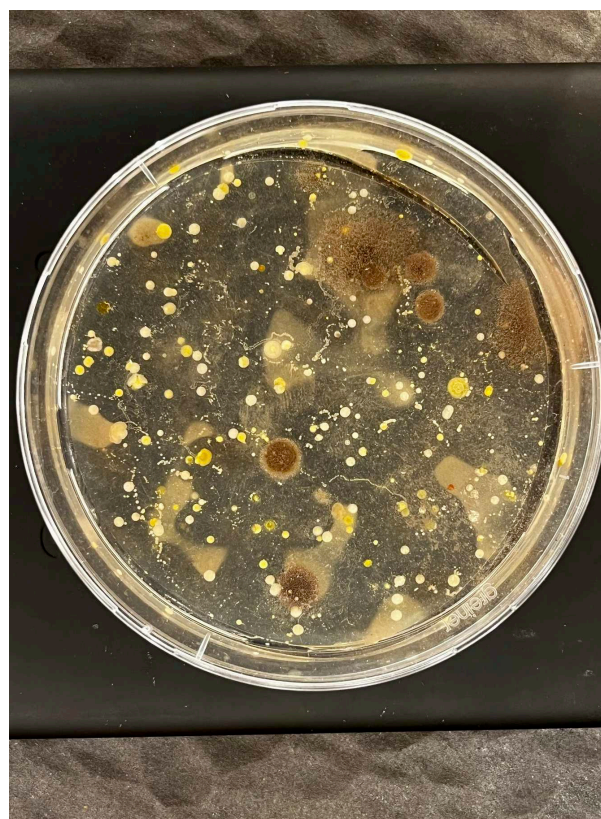
Figure A.8: Comparison of Petri dish lighting setups with and without diffusion.

Following this we pivoted away from the backlight and chose to explore how images would look if we applied a black background, while keeping the parchment paper for diffusion. In Figure A.9a we flipped the Kindle and used the black backside as the background. The background around the kindle was regular brown cardboard which we thought could obstruct the camera from focusing on

the plate. We therefore tried to paint the cardboard black to hopefully get a clearer picture. The results can be seen in A.9b. Reducing the light intensity around the subject seemed to enhance the light intensity of the subject, but the Kindle background then also got lighter, so the background inside the dish seems less black.



(a) No backlight with black background.



(b) No backlight with black background and black cardboard

Figure A.9: Experimental comparison of top-lighting effects on dark backgrounds.

After these observations we decided to rebuild the entire setup to make it more robust, find a way to make the background even darker, and to make the spread of the lighting more uniform. To do this we constructed a hexagonal prototype.

Second Iteration The imaging enclosure consists of several modular components (Fig. A.10). The outer structure is formed by a hexagonal shell (Fig. A.10a) with a black cardboard base (Fig. A.10b). Inside, a dedicated platform positions the Petri dish (Fig. A.10c), while a middle insert reduces ambient light around the sample (Fig. A.10d). Finally, a lid with a centred hole allows controlled camera access (Fig. A.10e).

The light blocking middle part was made to try and block the light from the LED so all the parts around the subject would remain black. While we recognize that part of our system crops the Petri dish out of the image so artifacts around should not be a concern, we believe that limiting the light around the subject allows the camera to better focus on the light on the Petri dish. During testing we found that the rounder shape provided more uniform lighting. We also experimented with removing the bottom cardboard (fig /A.10b) and instead introduce a large empty space beneath



(a) Outer hexagonal shell



(b) Black cardboard bottom



(c) Petri dish platform



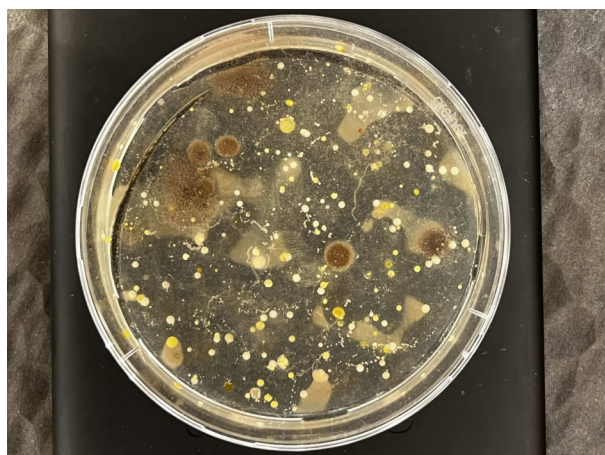
(d) Light-blocking middle part



(e) Lid with camera opening

Figure A.10: Components of the second iteration imaging prototype

the Petri dish. As light would travel through the dish and illuminate whatever platform was underneath, if we increased the space underneath, such that all light would be absorbed, we could achieve a truly black background for the Petri dish. A comparison with and without the hollow bottom compartment can be seen in Figure A.11. It is clear to see that the background plays a very large role in image quality. The textures of the colonies are much more refined, and the background is as dark as it can get.



(a) Image captured with black background



(b) Image captured with hollow bottom compartment

Figure A.11: Comparison of imaging with and without the hollow bottom compartment

With the addition of the hollow bottom we were satisfied with the imaging results. We then acquired a new Petri dish with agar and colonies on it from a source at the Copenhagen University. The agar medium was substantially thicker than the previous Petri dish. We tested the setup again on the new plate. The result can be seen in Figure A.12. To our dismay, the setup caused reflections of individual colonies to appear on the image of this plate. We believed that this could be due to the plastic platform on which the Petri dish is placed.



Figure A.12: Newly acquired agar plate. Prominent reflections of individual colonies present.

A.12 Full-System Evaluation Supplementary Material

More finegrained results for the Full-system evaluation

A.12.1 Manual reference count agreement

Table A.22: Manual reference count agreement by colony-density group. Plates were sorted by the manual consensus count and split into three groups of four plates.

Count group	Plates	Exact plates	Mean range	Max range	Mean abs. dev.	Max abs. dev.
Low (2–4 colonies)	2, 4, 3, 1	4	0.0	0.0	0.0	0.0
Middle (20–30 colonies)	5, 6, 8, 7	3	0.3	1.0	0.1	0.7
High (217–245 colonies)	11, 12, 10, 9	0	10.8	15.0	4.6	9.7

Table A.23: Annotator-level deviation from the manual consensus reference.

Annotator	Mean bias	Mean abs. dev.	Max abs. dev.	Within 1 colony (%)
Annotator A	2.0	2.0	9.7	66.7
Annotator B	-1.6	1.6	7.3	66.7
Annotator C	-0.4	1.1	5.3	66.7

A.12.2 Primary counting performance

Table A.24: Primary counting performance values corresponding to Figure 7.4. ColonyCounter and Colonia - input matched use the same input images. The Colonia app workflow is the app-workflow baseline. CA SD is calculated across the three image takers. Each row contains $n = 36$ counts.

Acquisition	Method	CA@10/ ± 1 (%)	CA SD	MAE	Bias
Freehand	ColonyCounter	97.2	4.8	5.7	-5.6
Freehand	Colonia - input matched	66.7	0.0	13.3	-13.3
Freehand	Colonia app workflow	44.4	17.3	15.6	-13.7
Apparatus	ColonyCounter	91.7	8.3	6.8	-6.2
Apparatus	Colonia - input matched	63.9	17.3	10.0	-8.5
Apparatus	Colonia app workflow	44.4	17.3	15.6	-13.7

A.12.3 Density subgroup performance

Table A.25: Density subgroup performance. CA values are CA@10/ ± 1 in percent. Dense MAE is included because this subgroup drives most remaining error. Each subgroup contains four plates.

Acquisition	Method	Sparse CA	Middle CA	Dense CA	Dense MAE
Freehand	ColonyCounter	100.0	100.0	91.7	16.5
Freehand	Colonia - input matched	100.0	100.0	0.0	39.3
Freehand	Colonia app workflow	58.3	75.0	0.0	43.9
Apparatus	ColonyCounter	100.0	100.0	75.0	19.3
Apparatus	Colonia - input matched	75.0	83.3	33.3	27.7
Apparatus	Colonia app workflow	58.3	75.0	0.0	43.9

Technical
University of
Denmark

Richard Petersens Plads, Building 324
2800 Kgs. Lyngby
Tlf. 4525 1700

www.compute.dtu.dk/